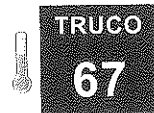


ponibles el intruso no será capaz de verlos para identificar cualquier parte no monitorizada de su sistema de ficheros. Segundo, es una buena idea cambiar los ficheros de política y de configuración de TripWire, de tal manera que su base de datos se almacene en alguna forma de medio de sólo lectura, tal como un CD. Esto evita que alguien sea capaz de crear su base de datos de nuevo con modificaciones, escondiendo así puertas traseras u otro tipo de programa dañino. Y finalmente, no espere a que su máquina esté expuesta a Internet para instalar y configurar TripWire. Le servirá mejor cuando se instala en una máquina limpia y que es capaz de seguir la pista de su sistema de ficheros desde una instalación reciente. De esta manera se asegurará de que no está monitorizando un equipo que ya ha estado en peligro.

Mientras podría parecer a primera vista que TripWire es demasiado abrumador para molestarle por él, no es exactamente el caso. El fichero de políticas es bueno asustando a la gente, y los valores por defecto y la configuración inicial pueden generar mucho ruido y extraños mensajes de error. Sin embargo, con un poco de trabajo y alguna exploración de su propio sistema de ficheros, podrá aprender bastante sobre cómo opera su sistema mientras configura TripWire. Además, TripWire tiene muchos usos fuera del ámbito de la seguridad. Por ejemplo, puede usar TripWire para asegurar que una aplicación desinstala todos sus componentes o para identificar todos los cambios hechos cuando instala un RPM. Los posibles usos para TripWire son infinitos, y una vez que lo domine, puede ser una herramienta increíblemente potente para monitorizar y mantener sus sistemas.

-Brian Warshawsky



TRUCO

67

Verificar la integridad de los sistemas de ficheros con Afick

Monitoree la integridad de sus sistemas de ficheros con esta herramienta fácil de usar.

La preocupación sobre la seguridad *online* crece día a día según van apareciendo nuevos virus y gusanos. Por esto, es ahora más importante que nunca el monitorizar el sistema de ficheros de su servidor para buscar signos de peligro. Anteriormente tratamos TripWire y presentamos sistemas de detección de intrusos y la utilización del comprobador de integridad de sistemas de ficheros TripWire para monitorizar la multitud de cambios que ocurren dentro de su sistema de ficheros. TripWire es una herramienta excelente, pero para mucha gente la empinada curva de aprendizaje es un gran repelente para utilizarlo. Si por cualquier razón TripWire no es para usted, hay otros comprobadores de integridad disponibles. ¡Después de todo, esto es Linux! Afick (*Another File Integrity Checker*, Otro

Comprobador de Integridad de Ficheros) es una de estas herramientas que proporciona numerosos métodos de configuración, incluyendo una interfaz gráfica en perl/tk y un módulo de administración por Web. Este truco le tendrá listo y funcionando con Afick mientras sus otros amigos administradores están todavía leyendo el manual de TripWire:

Instalar Afick

Hay unas pocas dependencias implicadas en el despliegue de Afick. Puesto que Afick está escrito en Perl, necesitará obviamente tener instalado tanto Perl como sus librerías. Aparte de esto, simplemente tiene que descargar el código fuente desde <http://afick.sourceforge.net>, desempaquetarlo en su ubicación de compilación preferida y ejecute la instalación como sigue:

```
# perl Makefile
```

Si no quiere instalar la interfaz gráfica perl/tk, puede ignorar cualquier advertencia que pueda ver sobre módulos perl/tk no encontrados.

Una vez que Perl ha terminado de procesar el Makefile, ejecute el siguiente comando para realmente instalar el software:

```
# make install
```

Ahora que hemos compilado e instalado Afick, vamos a configurarlo y a ponerlo en funcionamiento.

Configurar Afick para que armonice con su equipo

El primer paso para adaptar Afick a su sistema es editar su fichero de configuración, el cual determina a qué atributos de su sistema debe prestar atención Afick cuando escanea, y así saber cuando alertarle de cambios específicos. Afick proporciona un fichero de configuración por defecto, pero como cada sistema es diferente, debería no depender de él para mantener su servidor seguro. En última instancia, afinar Afick para que funcione de acuerdo a su sistema de ficheros será un proceso de ensayo y error.

Para comenzar este proceso, eche un vistazo primero al fichero de configuración de Afick, que se llama `linux.conf` y se encuentra en el directorio en el que desempaquetó Afick. Este fichero contiene varias secciones, dos de las cuales tienen un interés especial para nosotros. El fichero está presentado y dispuesto de una manera muy amigable, haciendo las secciones muy fáciles de diferenciar.

La primera sección en la que estamos interesados es la sección "alias". En esta sección, configuraremos las distintas combinaciones de comprobaciones de fi-

cheros que Afick puede llevar a cabo. Aplicaremos más tarde los alias definidos aquí a tipos específicos de ficheros y directorios. He aquí algunos alias comunes:

```
# alias :
#####
DIR = p+i+n+u+g
ETC = p+d+i+u+g+s+md5
Logs = p+n+u+g
MyRule = p+d+i+n+u+g+s+b+md5+m
```

La primera parte de cada directiva es simplemente el nombre del alias que se está definiendo. Usará esto más tarde para asignar estos alias a ficheros y directorios específicos. La segunda parte es una lista de las comprobaciones a llevar a cabo en el sistema de ficheros, separadas por signos "+". Una lista de estas opciones se presenta en la tabla 7.1 para su consulta.

Tabla 7.1. Opciones Afick de comprobación del sistema de ficheros.

Opción	Comprobación asociada
md5	Verifica la suma de comprobación de contenidos de fichero md5
sha1	Verifica la suma de comprobación de contenidos de fichero sha1
d	Verifica el número mayor y menor de dispositivo
i	Verifica el número de i-nodo
p	Verifica los permisos del fichero
n	Verifica el número de enlaces
u	Verifica la propiedad del fichero (usuario)
g	Verifica la propiedad del fichero (grupo)
s	Verifica el tamaño del fichero
b	Verifica el número de bloques destinados al fichero
m	Verifica el tiempo de la última modificación (mtime)
c	Verifica el tiempo del último cambio (ctime)
a	Verifica el tiempo del último acceso (atime)

La segunda parte del fichero de configuración en la que estamos interesados es la sección "Files to Scan" (ficheros a escanear). En esta sección puede definir qué comprobaciones individuales de Afick o qué combinaciones de ellas definidas como

alias se llevarán a cabo en ficheros y directorios específicos de su sistema de ficheros. He aquí algunos ejemplos para que los use al iniciar el proceso de afinar su configuración.

```
/etc/adjtime ETC
/etc/aliases.db ETC -md5
/etc/mail/statistics ETC -md5
/etc/dhcpd.conf c+sha1+s+p
!/etc/cups/certs/0
```

Este extracto deja ver mucho sobre la sintaxis del fichero de configuración. Cada uno de los tres primeros ficheros utiliza el alias ETC predefinido para especificar qué atributos deberían comprobarse. Sin embargo, el segundo y el tercero utilizan la directiva md5 para decirle a Afick que use el alias ETC menos la opción de comprobación md5. Esta postura es útil si quisiera especificar un alias genérico desde el que trabajar con una pequeña modificación para diferentes ficheros. La cuarta entrada comprueba sólo el tiempo de la última modificación, la suma sha1, el tamaño de fichero, y los permisos del fichero /etc/dhcpd.conf. La entrada final listada arriba utiliza la opción "!" (*obang*, para los usuarios *nix de la vieja escuela), que le dice a Afick que no compruebe el fichero o directorio especificado en absoluto. Esta opción debería usarse pocas veces, y sólo cuando es verdaderamente necesario.

Ejecutar Afick

Tras tomarse unos minutos para ajustar el fichero de configuración para que se adapte a su sistema de ficheros, ya está preparado para ejecutar Afick por primera vez. Afick opera creando una instantánea de su sistema de ficheros en forma de base de datos. Cuando ejecuta Afick por primera vez, esta base de datos será inicializada, almacenada, y usada como base para comparación en comprobaciones de integridad posteriores. Para crear la base de datos, ejecute el siguiente comando:

```
# afick -c /path_to_linux.conf/linux.conf -i
```

La directiva "-c" le dice a Afick dónde encontrar el fichero de configuración que debería usar, mientras que "-i" le dice que cree una base de datos inicial. Esta operación puede llevar unos cuantos minutos, pero cuando se complete encontrará la base de datos en la ubicación especificada en la primera directiva dentro de su fichero linux.conf.

Una vez que la base de datos inicial está creada, espere unos momentos y ejecute de nuevo Afick, esta vez con la opción "-k":

```
# afick -c /path_to_linux.conf/linux.conf -k
```

La opción "-k" le dice a Afick que compare el sistema de ficheros existente con la instantánea en la base de datos, e informe de cualquier error. Es llegado a este punto que da comienzo la fase de ensayo y error de su configuración Afick. Según se le informa de los cambios y errores, clasifíquelos y modifique su fichero de configuración como corresponda.

Mientras no esté cambiando cosas, y su sistema se encuentre en un estado tranquilo, lo que mostrará son cosas en su sistema que están probablemente en constante cambio. En algunos casos será apropiado continuar monitorizando atributos tales como la propiedad y los i-nodos, pero no los valores de "mtime" o "atime". Experimente con su fichero de configuración y ajústelo como corresponda. Una vez que pueda ejecutar Afick sin que devuelva un chorro de alertas, está preparado para añadirlo al crontab del súper-usuario para automatizar su ejecución periódica. Para hacer esto ejecute el siguiente comando como súper-usuario:

```
# crontab -e
```

Esto abrirá el crontab del súper-usuario en su editor de texto por defecto. Añada la siguiente línea sustituyendo la ruta apropiada:

```
0*/8 * * * root /ruta_a_afick.cron/afick.cron
```

Esto programará Afick para que se ejecute cada ocho horas, mandando correos al súper-usuario con cualquier cambio que ocurra.

Proteger Afick

Una vez que ha alcanzado este punto en su configuración, debería considerar mover su base de datos a un medio de almacenamiento de sólo lectura. Según mi experiencia, un disco zip viejo es una elección excelente (aunque también puede usar un CD-R o un DVD).

Para mover su base de datos a un disco zip, primero monte la unidad zip y después ejecute el siguiente comando:

```
# mv /var/lib/afick/afick.pag /mnt/zip/afick.pag
```

Una vez hecho esto, asegúrese de que modifica su fichero de configuración para que apunte a su recién trasladada base de datos usando una entrada "database := /ruta/a/base_de_datos".

Puede entonces mover también su fichero de configuración al disco zip, y utilizar la pestaña en la parte posterior de éste para marcarlo como de sólo lectura. Haciendo esto, está protegiendo su base de datos y su fichero de configuración de ser modificados por cualquiera que no tenga acceso físico al servidor.

Actualizar su base de datos

Cuando haga cambios en su sistema de ficheros, necesitará actualizar su base de datos. Puede hacer esto ejecutando el siguiente comando:

```
# afick -c /ruta_a_linux.conf/linux.conf -u
```

Una vez que el comando termina su ejecución, su base de datos está actualizada. Debería llevar a cabo una actualización cada vez que modernice una aplicación. Aplique nuevos parches de software o del núcleo de sistema, o realice cualquier otra actividad que altere su sistema de ficheros.

Conclusión

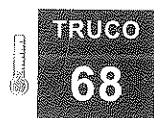
Como probablemente dirá, Afick es una versión menos complicada de Tripwire. Las dos aplicaciones comparten muchas similitudes, pero encuentro Afick la más útil y amigable de las dos.

En mi experiencia con Afick, he encontrado unos pocos usos más, aparte de asegurar que mi sistema no corra peligro. Entre estos usos adicionales están asegurar que las aplicaciones se desinstalen adecuadamente, así como llevar la cuenta de los cambios exactos hechos ejecutándolas. Estos y otros muchos usos que pueden encontrarse para éste y otros comprobadores de integridad, y tan solo un poco de experimentación le garantiza encontrar uno o dos que sean importantes para usted.

Véase también

- <http://afick.sourceforge.net>

-Brian Warshawsky



TRUCO

68

Buscar rootkit y otros ataques

Deje que chkrootkit compruebe automáticamente sus máquinas orientadas al exterior para buscar rootkit y otros ataques.

Un rootkit es un paquete de software que permite a una persona no autorizada obtener privilegios administrativos o de súper-usuario en una máquina. Se instalan normalmente explotando un problema de seguridad conocido. Una vez instalados, pueden capturar contraseñas, monitorizar el estado del sistema, enviar información de autenticación a otros equipos, e incluso ejecutar programas en intervalos programados.

Mientras que los *rootkit* son bastante interesantes conceptualmente, ser "rooteado" (el término que define estar en peligro porque gente no autorizada tiene acceso de súper-usuario en su sistema) no lo es. Afortunadamente, así como hay gran cantidad de *script* que automatizan la instalación de *rootkit*, también hay algunos paquetes estupendos de software que los detectan y que identifican sistemas y aplicaciones en peligro. Algunos paquetes, tales como Tripwire y Afick, generalmente monitorizan tamaños de ficheros y firmas, y le hacen saber si ha cambiado algo que no debería. Este truco explora *chkrootkit*, uno de los paquetes de software más potentes y populares, para realmente detectar *rootkit*, y trata sobre cómo instalarlo y usarlo para localizar y cerrar invasiones.

Confesiones

Hola, mi nombre es Bill, y uno de mis sistemas fue "rooteado" una vez. ¿Qué mejor sitio para confesar mis indiscreciones como administrador que un libro que con un poco de suerte será leído por millones de personas?

Hace años, mucho antes de que las puertas de enlace caseras y los equipos NAT (*Network Address Translation*, Traducción de Dirección de Red) fueran dispositivos electrónicos de consumo por debajo de los 80 euros, construí mi propia puerta de enlace casera, como la mayoría de los fanáticos de Linux. Poniendo una tarjeta Ethernet extra en una vieja máquina Pentium y escribiendo unas cuantas reglas *ipfwadm* (el ancestro del *ipchains* de ayer y del *iptables* de hoy), podía hacer NAT y enmascaramiento de mis sistemas internos a través de mi interfaz de red externa. Esto funcionó bien 24x7 durante bastante tiempo, módulo los fallos de corriente ocasionales, y lo utilizaba para ejecutar mi servidor de nombres casero y dirigirlo a servidores DNS externos. Nunca actualicé el software de ese equipo, basándome en la regla del "no lo arregles si no está roto" (que es una regla muy mala a seguir por administradores de sistemas cuando se trata de actualizaciones de seguridad). Un día, inicié sesión en el equipo para comprobar algo y me di cuenta de que la salida de mi invocación favorita al comando *ps* no mostraba la información de la misma manera que solía hacerlo. Así que revisé */var/log/messages* y encontré unos cuantos mensajes que indicaban que alguien había estado probando mi servidor DNS, intentando inducir un desbordamiento de la memoria intermedia. Hurgué un poquito y, sin sorpresa, descubrí que mi máquina había sido asaltada y el *rootkit* t0rn había sido instalado (<http://www.sans.org/y2k/t0rn.htm>). Mi reacción a esto fue diferente que la de la mayoría. Puesto que ninguna de mis máquinas de casa habían sido asaltadas (lo comprobé) y sentía curiosidad, cambié todas mis contraseñas en los sistemas que podía haber

contactado desde que se instaló el *rootkit* (desde el trabajo, por supuesto, no desde casa), y dejé de hacer nada en casa que necesitara de una contraseña remota por unos cuantos días. Entonces puse un fichero *README.txt* en */usr/src/.puta*, que es donde t0rn pone la mayoría de sus ficheros, diciendo algo como "Hola, enhorabuena, ¿cómo has entrado?" Recibí un correo en un día más o menos del tipo que había atacado mi máquina, intercambiamos unos cuantos mensajes por medio del correo anónimo que estaba usando, y despertó mi interés sobre algunos de los *rootkit* a los que tenía acceso. Me habría preocupado de veras si hubiera sido una máquina del trabajo, pero tal como era la situación, parecía un tipo muy inteligente y aprendí unas cuantas cosas. De todas maneras reconstruí la máquina (con software actualizado) en una semana o dos, soy amistoso pero no suicida.

El punto aquí no es que mi sistema fue asaltado, sino que la posibilidad siempre está ahí. Los piratas informáticos pueden a menudo explotar problemas recién descubiertos o sin parche en el software de sistema para instalar un *rootkit* en él, algunos de los cuales son tan rápidos como inteligentes. Añadir *chkrootkit* a la caja de herramientas de su sistema puede ayudarle a detectar este tipo de invasiones y cerrarlas tan rápido como sea posible.

Tipos de rootkit

Los *rootkit* Linux funcionan de varias maneras, normalmente como módulos del núcleo de sistema, como paquetes de software del espacio de usuario que reemplazan binarios de sistema, o como una combinación de ambos. Los *rootkit* del núcleo insertan módulos adicionales que reemplazan las llamadas al sistema con versiones modificadas que capturan información, y a menudo ocultan detalles sobre procesos específicos al usuario, mientras que los *rootkit* de espacio de usuario generalmente reemplazan binarios de sistema tales como *ps*, *login*, *passwd*, etc. con versiones modificadas que también capturan información y ocultan detalles sobre procesos y directorios específicos.

Por ejemplo, el *rootkit* t0rn mencionado en el apartado anterior reemplaza binarios de sistema tales como *ps*, *top*, y *ls* con versiones que no mostrarán nada de lo que esté ejecutando desde su directorio */usr/src/.puta*. En realidad es bastante inteligente.

chkrootkit ejecuta en sistemas Linux que utilizan cualquier núcleo 2.x y se ha usado y probado también en sistemas FreeBSD 2.2.x, 3.x, 4.x y 5.x; OpenBSD 2.x y 3.x; NetBSD 1.6.x; Solaris 2.5.1, 2.6, 8.0, y 9.0; y en varias versiones de sistemas HP-UX, Tru64, y BSDI. En el momento de escribir este libro, *chkrootkit* puede detectar *rootkit* tales como 55808.A Worm, Adore LKM, Adore Worm,

AjaKit, Anonoying, Aquatica, ARK, Bobkit, dsc-rootkit, duarawkz, Ducoci, ESRK, Fu, George, Gold2, Hidrootkit, Illogic, Kenga3, kenny-rk, knark LKM, Lion Worm, LOC, LPD Worm, lrk, Madalin, Maniac-RK, MithRa's Rootkit, Monkit, Omega Worm, OpenBSD rk v1, Optickit, Pizdakit, Ramen Worm, rh-shaper, RK17, Romanian, RSHA, RSTb trojan, Scalper, Sebek LKM, ShitC Worm, Shkit, Showtee, shv4, SK, Slapper A-D, SucKIT, TC2 Worm, t0rn, TRK, Volc, Wormkit Worm, x.c Worm, zaRwT, y ZK.

Un problema básico en la detección de los *rootkit* es que cualquier sistema en el que se haya instalado alguno ya no es de confianza para detectarlos. Esto puede resolverse haciendo un mantenimiento regular de sistema ejecutando *chkrootkit* desde un CD de arranque.

Volveremos a esto más tarde. Por ahora, vamos a instalar *chkrootkit* y a ponerlo en marcha.

Obtener, compilar e instalar *chkrootkit*

chkrootkit es de código abierto y se encuentra disponible totalmente libre en <http://www.chkrootkit.org/download>. La versión cuando se escribió este libro era la 0.46. Las versiones más nuevas son mejores, ya que cada una de ellas añade software y soporte para detectar más y más *rootkit*. El ejecutable de *chkrootkit* es un *script* de intérprete de comandos que ejecuta los binarios y otros *script* que se incluyen como parte del paquete *chkrootkit*.

Tras descargarse el fichero *.tar* con los fuentes, puede compilar *chkrootkit* como se muestra en el siguiente ejemplo:

```
$ tar xzf chkrootkit.tar.gz
$ cd chkrootkit-0.46
$ make
*** stopping make sense ***
make[1]: Entering directory `/home/wvh/src/chkrootkit-0.46'
gcc -DHAVE_LASTLOG_H -o chklastlog chklastlog.c
gcc -DHAVE_LASTLOG_H -o chkwtmp chkwtmp.c
gcc -DHAVE_LASTLOG_H -D_FILE_OFFSET_BITS=64 -o ifpromisc ifpromisc.c
gcc -o chkproc chkproc.c
gcc -o chkdirs chkdirs.c
gcc -o check_wtmpx check_wtmpx.c
gcc -static -o strings-static strings.c
gcc -o chkutmp chkutmp.c
make[1]: Leaving directory `/home/wvh/src/chkrootkit-0.46'
```

El fichero *Makefile* de *chkrootkit* no proporciona un destino de instalación estándar, así que debe copiar sus binarios a alguna parte o bien ejecutarlo directamente desde el directorio en el cual lo compila. Si hace esto último, le sugeriría que eliminara todos los ficheros de código fuente para dificultar a cualquiera que

haya asaltado su sistema el modificar su instalación de *chkrootkit*, no lo hará imposible, pero sí más difícil.

Ejecutar *chkrootkit*

Una vez que ha compilado *chkrootkit*, simplemente ejecútelo desde donde haya puesto los binarios escribiendo *./chkrootkit* o invocando la ruta completa al *script* *chkrootkit*. Debe ejecutarlo como súper-usuario o vía *sudo*. La salida de una ejecución de *chkrootkit* se parece a lo siguiente:

```
# ./chkrootkit
ROOTDIR is '/'
Checking 'amd'... not found
Checking 'basename'... not infected
Checking 'biff'... not found
Checking 'chfn'... not infected
Checking 'chsh'... not infected
Checking 'cron'... not infected
Checking 'date'... not infected
Checking 'du'... not infected
Checking 'dirname'... not infected
Checking 'echo'... not infected
Checking 'egrep'... not infected
Checking 'env'... not infected
Checking 'find'... not infected
Checking 'fingerd'... not found
Checking 'gpm'... not infected
Checking 'grep'... not infected
Checking 'hdparm'... not infected
Checking 'su'... not infected
Checking 'ifconfig'... not infected
Checking 'inetd'... not tested
Checking 'inetdconf'... not found
Checking 'identd'... not found
Checking 'init'... not infected
Checking 'killall'... not infected
Checking 'ldsopreload'... not infected
Checking 'login'... not infected
Checking 'ls'... not infected
Checking 'lsof'... not infected
Checking 'mail'... not infected
Checking 'mingetty'... not infected
Checking 'netstat'... not infected
Checking 'named'... not infected
Checking 'passwd'... not infected
Checking 'pidof'... not infected
Checking 'pop2'... not found
Checking 'pop3'... not found
Checking 'ps'... not infected
```

```

Checking 'pstree'... not infected
Checking 'rpcinfo'... not infected
Checking 'rlogind'... not found
Checking 'rshd'... not found
Checking 'slogin'... not infected
Checking 'sendmail'... not infected
Checking 'sshd'... not infected
Checking 'syslogd'... not infected
Checking 'tar'... not infected
Checking 'tcpd'... not infected
Checking 'tcpdump'... not infected
Checking 'top'... not infected
Checking 'telnetd'... not found
Checking 'timed'... not found
Checking 'traceroute'... not infected
Checking 'vdir'... not infected
Checking 'w'... not infected
Checking 'write'... not infected
Checking 'aliens'... no suspect files
Searching for sniffer's logs, it may take a while... nothing found
Searching for HiDrootkit's default dir... nothing found
Searching for t0rn's default files and dirs... nothing found
Searching for t0rn's v8 defaults... nothing found
Searching for Lion Worm default files and dirs... nothing found
Searching for RSHA's default files and dir... nothing found
Searching for RH-Sharp's default files... nothing found
Searching for Ambient's rootkit (ark) default files and dirs... nothing found
Searching for suspicious files and dirs, it may take a while...
  /usr/lib/jvm/java-1.4.2-sun-1.4.2.08/jre/.systemPrefs
  /usr/lib/perl5/5.8.6/x86_64-linux-thread-multi/.packlist
Searching for LPD Worm files and dirs... nothing found
Searching for Ramen Worm files and dirs... nothing found
Searching for Maniac files and dirs... nothing found
Searching for RK17 files and dirs... nothing found
Searching for Ducoci rootkit... nothing found
Searching for Adore Worm... nothing found
Searching for ShitC Worm... nothing found
Searching for Omega Worm... nothing found
Searching for Sadmind/IIS Worm... nothing found
Searching for MonKit... nothing found
Searching for Showtee... nothing found
Searching for OpticKit... nothing found
Searching for T.R.K... nothing found
Searching for Mithra... nothing found
Searching for OBSD rk v1... nothing found
Searching for LOC rootkit... nothing found
Searching for Romanian rootkit... nothing found
Searching for Suckit rootkit... nothing found
Searching for Volc rootkit... nothing found
Searching for Gold2 rootkit... nothing found
Searching for TC2 Worm default files and dirs... nothing found
Searching for Anonoying rootkit default files and dirs... nothing found

```

```

Searching for ZK rootkit default files and dirs... nothing found
Searching for ShKit rootkit default files and dirs... nothing found
Searching for AjaKit rootkit default files and dirs... nothing found
Searching for zaRwT rootkit default files and dirs... nothing found
Searching for Madalin rootkit default files... nothing found
Searching for Fu rootkit default files... nothing found
Searching for ESRK rootkit default files... nothing found
Searching for anomalies in shell history files... nothing found
Checking 'asp'... not infected
Checking 'bindshell'... not infected
Checking 'lkm'... chkproc: nothing detected
Checking 'rexedcs'... not found
Checking 'sniffer'...
  eth0: not promisc and no PF_PACKET sockets
  vmnet8: not promisc and no PF_PACKET sockets
  vmnet1: not promisc and no PF_PACKET sockets
Checking 'w55808'... not infected
Checking 'wted'... chkwtm: nothing deleted
Checking 'scalper'... not infected
Checking 'slapper'... not infected
Checking 'z2'... chklastlog: nothing deleted
Checking 'chkutmp'... chkutmp: nothing deleted

```

Parece ser que estoy limpio, ¡y han sido un montón de pruebas! Como puede ver, chkrootkit primero comprueba un surtido de binarios de sistema para buscar cadenas que podrían indicar que han sido alterados, después busca indicadores de *rootkit* conocidos, revisa los puertos de red buscando falsos procesos, etc., ya me siento mejor.



Si está ejecutando software adicional de seguridad tal como PortSentry (<http://sourceforge.net/projects/sentrytools/>), podría obtener falsos positivos (es decir, información de problemas que en realidad no lo son) de la prueba "bindshell", que busca procesos que estén monitorizando puertos específicos.

Si quiere ser incluso más paranoico que el comportamiento normal de chkrootkit, puede ejecutarlo con su opción "-x" (experto). Esta opción hace que chkrootkit muestre salida detallada de las pruebas para poder darle la oportunidad de detectar problemas potenciales que puedan ser evidencia de *rootkit* que la versión que está ejecutando puede no haber sido capaz de identificar (todavía).

Automatizar chkrootkit

Ejecutar chkrootkit "de cuando en cuando" es una buena idea, pero ejecutarlo regularmente vía cron es una mejor. Para ejecutar chkrootkit automáti-

camente, inicie sesión como súper-usuario, haga su, o utilice sudo para ejecutar `crontab -e` y añadir `chkrootkit` a la lista de procesos de súper-usuario que se ejecutan automáticamente con `cron`. Por ejemplo, la siguiente entrada ejecutaría `chkrootkit` cada noche a la 1 y mandaría un correo con su salida a `root@hq.vonhagen.org`:

```
0 3 * * * (cd /path/to/chkrootkit; ./chkrootkit 2>&1 | mail -s "chkrootkit \
output" root@hq.vonhagen.org)
```

Resumen

Un problema básico en la detección de un *rootkit* es que cualquier sistema en el que haya sido instalado no es de fiar para su detección. Incluso si sigue las instrucciones de este truco y ejecuta `chkrootkit` vía `cron`, sólo haría falta una pequeña ventana de oportunidad para que el inteligente atacante compruebe la entrada `crontab` del súper-usuario y bien la desactive o altere el propio `chkrootkit`. La combinación de `chkrootkit` con software como Tripwire o Afick puede ayudarle a hacer esta ventana lo más pequeña posible, pero hacer comprobaciones regulares de seguridad de las máquinas dirigidas al exterior desde un CD de arranque que incluya `chkrootkit`, tal como el Inside Security's Insert Security Rescue CD (<http://sourceforge.net/projects/insert/>), es su mejor solución para identificar *rootkit* de tal manera que pueda recuperar sistemas en peligro.

Véase también

- <http://www.chkrootkit.org>
- Insert Security Rescue CD: http://www.inside-security.de/insert_en.html
- Rootkit Hunter: <http://www.rootkit.nl>
- Usuarios de Windows: <http://research.microsoft.com/rootkit/>
- Usuarios de Windows: <http://www.sysinternals.com/utilities/rootkitrevealer.html>



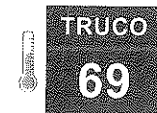
Solución de problemas y rendimiento

Trucos 69 a 77

Le sorprendería lo a menudo que "optimizar rendimiento" realmente se traduce en "solucionar problemas". Si algo no está bien configurado o está roto de alguna otra manera, es probable que el primer indicio de que algo no funciona bien sea un rendimiento pobre, bien del servicio en cuestión o del equipo en el que está ejecutando.

Rendimiento es un término relativo. Es importante saber cómo se ve un sistema cuando está ejecutando con poca o ninguna carga, para poder ser capaz de medir el impacto de añadir incrementalmente más usuarios y servicios.

En este capítulo, le daremos las herramientas y técnicas para solucionar los problemas que encuentre en su camino hacia un rendimiento mejor, para optimizar los recursos que el sistema reserva para sus tareas programadas, y para lidiar con los acaparadores de recursos en sus sistemas y redes.



TRUCO 69 Encuentre acaparadores de recursos con comandos estándar

No necesita un lujoso software de terceros o analizadores de bitácora para encontrar y lidiar con un usuario enloquecido en una orgía de recursos.

Hay ocasiones en las que los usuarios consumen más que su justa porción de recursos de sistema, ya sea CPU, espacio de disco, manejo de ficheros, o ancho de banda. En entornos en los que los usuarios tienen la sesión abierta en la consola (o invocando la utilidad `login` de alguna otra manera), puede usar `pam_limits`, o la utilidad `ulimit` para evitar que se les vaya la mano.

En otros entornos, ninguno de estos métodos es particularmente útil. En servidores de desarrollo, por ejemplo, podría estar hospedando a 50 desarrolladores en una sola máquina donde todos prueban su código antes de pasarlo a las rotativas de producción. Las máquinas de esta naturaleza se configuran generalmente para permitir cosas como ejecutar trabajos *cron*. Mientras que probablemente es técnicamente posible limitar los recursos que la utilidad *cron* puede consumir, esto podría ser buscar problemas, especialmente cuando considera que hay muchos trabajos que ejecutan fuera de *cron* en nombre del sistema, tales como *makewhatis* y *LogWatch*.

En general, los desarrolladores no quieren acaparar recursos. Realmente, no quieren. Hace que su trabajo lleve más tiempo, y que sus compañeros de trabajo descarguen una ración de rabia sobre ellos. Y sobre todo molesta a los administradores de sistemas, a sabiendas de que pueden hacer sus vidas, digamos, "desafiantes". Lo dicho, el acaparamiento de recursos generalmente no es una incidencia diaria, ni siquiera semanal, y difícilmente justifica el coste del software de terceros, ni hacer malabares para configurar cada tipo concebible de consumo de recursos.

Normalmente, se entera de las disputas de recursos bien a través de un correo de alerta de una herramienta de monitorización, o por un correo de un usuario quejándose sobre los lentos tiempos de respuesta o el "cuelgue" de los inicios de sesión. La primera cosa que puede hacer es iniciar sesión en la máquina y ejecutar el comando *top*, que le mostrará el número de tareas que están actualmente en ejecución, la cantidad de memoria en uso, el consumo de espacio de intercambio (*swap*), y lo ocupadas que están las CPU. Además le muestra una lista de los mayores consumidores de recursos, y sus datos se actualizan cada pocos segundos para su comodidad. He aquí una salida de ejemplo de *top*:

```
top - 21:17:48 up 26 days, 6:37, 2 users, load average: 0.18, 0.09, 0.03
Tasks: 87 total, 2 running, 83 sleeping, 2 stopped, 0 zombie
Cpu(s): 14.6% us, 20.6% sy, 0.0% ni, 64.1% id, 0.0% wa, 0.3% hi, 0.3% si
Mem: 2075860k total, 1343220k used, 732640k free, 216800k buffers
Swap: 4785868k total, 0k used, 4785868k free, 781120k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3098	jonesy	25	0	4004	1240	956	S	8.7	0.1	0:11.42	gloton.sh
30033	jonesy	15	0	6400	2100	1656	S	0.7	0.1	0:02.57	sshd
8083	jonesy	16	0	2060	1064	848	R	0.3	0.1	0:00.06	top
1	root	16	0	1500	516	456	S	0.0	0.0	0:01.91	init

Como puede apreciar, el mayor consumidor de recursos es mi *script* *gloton.sh*. Lleva ejecutando unos 11 segundos (como se muestra en la columna *TIME+*), tiene un identificador de proceso 3098, y utiliza 1240K de memoria física. Un campo clave aquí es el campo "NI". Éste hace referencia al valor adecuado (*Nice value*). Los usuarios pueden usar la utilidad *renice* para dar a sus trabajos prio-

ridades menores, ayudando así a asegurar que no se ponen en medio de otros trabajos planeados para ser ejecutados por el programador de tareas del núcleo de sistema. El núcleo ejecuta trabajos basándose en sus prioridades, las cuales se indican en el campo "PR". Como administrador en la posición de intentar arreglar problemas sin pisar el pie a sus necesidades como usuario, un primer paso en el ahorro de recursos podría ser ejecutar *renice* sobre el *script* *gloton.sh*. Necesitará ejecutar *top* como súper-usuario para hacer esto sobre un proceso que no sea de su propiedad. Puede hacerlo pulsando *R* en su teclado, *ps* le preguntará entonces qué proceso desea re-priorizar:

```
top - 21:19:07 up 26 days, 6:38, 2 users, load average: 0.68, 0.26, 0.09
Tasks: 88 total, 4 running, 82 sleeping, 2 stopped, 0 zombie
Cpu(s): 19.6% us, 28.9% sy, 0.0% ni, 49.8% id, 0.0% wa, 1.0% hi, 0.7% si
Mem: 2075860k total, 1343156k used, 732704k free, 216800k buffers
Swap: 4785868k total, 0k used, 4785868k free, 781120k cached
PID to renice: 3098
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3098	jonesy	25	0	4004	1240	956	R	14.3	0.1	0:22.37	gloton.sh

Escribiendo el ID del proceso y pulsando *Intro* hará que *top* le pregunte qué valor adecuado quería poner al proceso. Yo le he asignado 15. En el siguiente refresco de la pantalla, fíjese en el cambio de las estadísticas de mi *script*:

```
top - 21:20:22 up 26 days, 6:39, 2 users, load average: 1.03, 0.46, 0.18
Tasks: 87 total, 1 running, 84 sleeping, 2 stopped, 0 zombie
Cpu(s): 1.3% us, 22.3% sy, 13.6% ni, 61.5% id, 0.0% wa, 0.7% hi, 0.7% si
Mem: 2075860k total, 1343220k used, 732640k free, 216800k buffers
Swap: 4785868k total, 0k used, 4785868k free, 781120k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3098	jonesy	39	15	4004	1240	956	S	12.00	10:31.34	gloton.sh	

Reiniciar un proceso es una precaución de seguridad. Puesto que no sabe lo que hace el código, no sabe la molestia que causará al usuario si lo "mata" instantáneamente. Reiniciar le ayudará a asegurarse de que el proceso no vuelve el sistema inutilizable mientras intenta conseguir más información sobre él.

Lo siguiente que debe comprobar es nuestro viejo amigo *ps*. En realidad hay múltiples maneras de descubrir qué más está ejecutando un usuario dado. Intente ésta:

```
$ ps -ef | grep jonesy
jonesy 28820 1 0 Jul31 ? 00:00:00 SCREEN
jonesy 28821 28820 0 Jul31 pts/3 00:00:00 /bin/bash
jonesy 30203 28821 0 Jul31 pts/3 00:00:00 vim XF86Config
jonesy 30803 1 0 Jul31 ? 00:00:00 SCREEN
jonesy 30804 30803 0 Jul31 pts/4 00:00:00 /bin/bash
```

```
jonesy 30818      1    0 Jul31 ?      00:00:00 SCREEN --1
jonesy 30819 30818    0 Jul31 pts/5    00:00:00 /bin/bash
```

Esto devuelve un listado completo de todos los procesos que contienen la cadena "jonesy". Fíjese que no estoy seleccionando aquí por usuario, de tal manera que si algún otro usuario está ejecutando un *script* llamado "jonesy-es-un-terrible-administrador", me enteraré. Aquí puedo ver que el usuario jonesy está ejecutando además unos cuantos programas más. El PID de cada proceso se lista en la segunda columna, y PID padre (PPID) de cada proceso se lista en la tercera columna. Esto es útil, porque puedo decir, por ejemplo, que el PID 28821 fue iniciado realmente por el PID 28820, así que puedo ver aquí que estoy ejecutando una instancia del intérprete bash dentro de una sesión screen.

Para tener una imagen que ilustre incluso más claramente la relación entre procesos padres e hijos, pruebe este comando:

```
$ ps -fHU jonesy
```

Esto le mostrará los procesos del usuario jonesy en forma jerárquica, de esta manera:

UID	PID	PPID	C	STIME	TTY	TIME	CMD
jonesy	25760	25758	0	15:34	?	00:00:00	sshd: jonesy@notty
jonesy	25446	25444	0	Jul29	?	00:00:06	sshd: jonesy@notty
jonesy	20761	20758	0	16:28	?	00:00:03	sshd: jonesy@pts/0
jonesy	20812	20761	0	16:28	pts/0	00:00:00	-tcsh
jonesy	12543	12533	0	12:11	?	00:00:00	sshd: jonesy@notty
jonesy	12588	12543	0	12:11	?	00:00:00	tcsh -c /usr/local/libexec/sft
jonesy	12612	12588	0	12:11	?	00:00:00	/usr/local/libexec/sftp-serv
jonesy	12106	12104	0	10:49	?	00:00:01	sshd: jonesy@pts/29
jonesy	12135	12106	0	10:49	pts/29	00:00:00	-tcsh
jonesy	12173	12135	0	10:49	pts/29	00:00:01	ssh livid
jonesy	10643	10641	0	Jul28	?	00:00:07	sshd: jonesy@pts/41
jonesy	10674	10643	0	Jul28	pts/41	00:00:00	-tcsh
jonesy	845	10674	0	15:49	pts/41	00:00:06	ssh newhotness
jonesy	7011	6965	0	10:15	?	00:01:39	sshd: jonesy@pts/21
jonesy	7033	7011	0	10:15	pts/21	00:00:00	-tcsh
jonesy	17276	7033	0	11:01	pts/21	00:00:00	-tcsh
jonesy	17279	17276	0	11:01	pts/21	00:00:00	make
jonesy	17280	17279	0	11:01	pts/21	00:00:00	/bin/sh -c bibtex paper;
jonesy	17282	17280	0	11:01	pts/21	00:00:00	latex paper
jonesy	17297	7033	0	11:01	pts/21	00:00:00	-tcsh
jonesy	17300	17297	0	11:01	pts/21	00:00:00	make
jonesy	17301	17300	0	11:01	pts/21	00:00:00	/bin/sh -c bibtex paper;
jonesy	17303	17301	0	11:01	pts/21	00:00:00	latex paper
jonesy	6820	6816	0	Jul28	?	00:00:03	sshd: jonesy@notty
jonesy	6209	6203	0	22:15	?	00:00:01	sshd: jonesy@pts/31
jonesy	6227	6209	0	22:15	pts/31	00:00:00	-tcsh

Como puede ver, itengo mucho ejecutando! Estos procesos parecen bastante benignos, pero esto podría no ser siempre así. Si un usuario está realmente generando muchos procesos de alto consumo de recursos, algo que puede hacer es aplicar *renice* a todos los procesos propiedad de ese usuario de un solo golpe. Por ejemplo, para cambiar la prioridad de todos los procesos que posee el usuario jonesy para que ejecuten sólo cuando no hay nada más ejecutando, utilizaría el siguiente comando:

```
$ renice 20 -u jonesy
1001: old priority 0, new priority 19
```

Haciendo esto a un usuario que haya hecho que la carga del sistema salte a 50 o más puede devolver el sistema a un nivel que lo haga utilizable de nuevo.

¿Qué hacer con los acaparadores de disco?

Los comandos anteriores no le ayudarán con los usuarios que están acaparando espacio de disco. Si los directorios personales de sus usuarios están todos en la misma partición y no está imponiendo cuotas, cualquier cosa, desde un programa desbocado a la afición por las descargas de música puede llenar rápidamente toda la partición. Esto hará que aplicaciones comunes tales como el correo dejen de funcionar por completo. Si su servidor de correo está configurado para montar los directorios personales de usuario y entregar correo en sus carpetas, iesto no tendrá gracia!

Cuando un usuario llama para decir que el correo no funciona, el primer comando que querrá ejecutar será este:

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
fileserver:/export/homes
                 323G  323G   0G  100% /.autofs/u
```

Bien, esto es un sistema de ficheros lleno, por si nunca había visto uno. El comando *df* muestra estadísticas de espacio usado/espacio libre del disco para todos los sistemas de ficheros montados por defecto, o para cualquier sistema de ficheros que reciba como argumento. Ahora, para descubrir la identidad de nuestro "glotón", recurriremos al comando *du* (como súper-usuario):

```
# du -s -B 1024K /home/* | sort -n
```

El comando *du* anterior produce un resumen (opción *-s*) para cada directorio bajo */home*, presentando el uso de disco de cada directorio en bloques de 1024K (1 MB). Conectamos luego la salida con una tubería a la entrada del comando *sort*, al cual le he dicho que lo ordene numéricamente en vez de alfabéticamente

dándole la opción `-n`. con esta salida, puede ver directamente dónde se está usando la mayor cantidad de espacio de disco, y puede entonces tomar la acción oportuna (ya sea contactar al propietario de un enorme fichero o directorio, o eliminar o truncar un fichero de bitácora fuera de control).

Acaparamiento de ancho de banda

Los usuarios que acaparan el ancho de banda raras veces son difíciles de reconocer utilizando las herramientas que ya hemos discutido. Sin embargo, si el culpable no es evidente por alguna razón, puede apoyarse en una verdad fundamental sobre los sistemas tipo Unix que data de décadas: todo es un fichero.

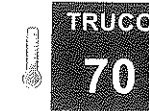
Puede probar cualquier cosa que pueda representarse como un fichero con el comando `lsuf`. Para obtener una lista de todos los ficheros de red (*socket*, conexiones abiertas, puertos abiertos), ordenados por nombre de usuario, pruebe este comando:

```
$ lsuf -i -P | sort -k3
```

La opción `"-i"` de `lsuf` le dice que seleccione sólo ficheros relacionados con la red. La opción `"-P"` le dice que muestre los números de puerto en vez de intentar traducirlos a sus nombres de servicio correspondientes. Conectamos entonces la salida con una tubería a nuestro viejo amigo `sort`, al que le he dicho esta vez que ordene basándose en el tercer campo o "clave", que es el nombre de usuario. He aquí alguna salida:

sshd	1859	root	3u	IPv6	5428	TCP *:22 (LISTEN)
http	1914	root	3u	IPv6	5597	TCP *:80 (LISTEN)
sendmail	16643	root	4u	IPv4	404617	TCP localhost.localdomain:25 (LISTEN)
httpd	1914	root	4u	IPv6	5598	TCP *:443 (LISTEN)
dhcpcd	5417	root	6u	IPv4	97449	UDP *:67
sshd	24916	root	8u	IPv4	4660907	TCP localhost.localdomain:6010 (LISTEN)
nmbd	7812	root	9u	IPv4	161622	UDP *:137
snmpd	25213	root	9u	IPv4	4454614	TCP *:199 (LISTEN)
sshd	24916	root	9u	IPv6	4660908	TCP localhost:6010 (LISTEN)
COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE NODE NAME

Todos estos son servicios comunes, por supuesto, pero en el caso de que pille aquí un puerto que no reconozca, puede continuar utilizando herramientas como un gráfico MRTG, `ngrep`, `tcpdump`, o `snmpget/snmpwalk` para intentar descubrir lo que está haciendo el programa, etc. Además, puesto que `lsuf` le muestra qué procesos mantienen abiertos qué puertos, los problemas que necesiten una atención inmediata pueden ser resueltos utilizando comandos estándar como `renice` o `kill` sobre los procesos ofensivos.



Reduzca tiempos de reinicio con sistemas de ficheros transaccionales

Los discos grandes y los problemas de sistema de ficheros pueden agobiar el proceso de arranque a menos que esté usando un sistema de ficheros transaccional (*journaling*). Linux le da opciones en abundancia.

Los sistemas informáticos pueden sólo montar y usar sistemas de ficheros con éxito si pueden asegurarse de que todas las estructuras de datos en cada sistema de ficheros son consistentes. En términos Linux y Unix, consistencia significa que todos los bloques de disco que están realmente siendo usados en algún fichero o directorio están marcados como que están en uso, todos los bloques borrados no están vinculados a nada que no sea la lista de bloques libres, todos los directorios en el sistema de ficheros realmente tienen directorios padre, etc. Esta comprobación se hace por las aplicaciones de comprobación de consistencia, la más conocida de las cuales es la aplicación `fsck` estándar de Linux/Unix. Cada sistema de ficheros tiene su propia versión de `fsck` (con nombres como `fsck.ext3`, `fsck.jfs`, `fsck.reiserfs`, etc.) que comprende a, y que "hace lo correcto" para ese sistema de ficheros en particular.

Cuando los sistemas de ficheros se montan como parte del proceso de arranque, se marcan como que están en uso ("sucios"). Cuando se apaga un sistema de manera normal, todos sus sistemas de ficheros en disco se marcan como consistentes ("limpios") al desmontarlos. Cuando se reinicia el sistema, los sistemas de ficheros que están marcados como limpios no tienen que ser comprobados antes de montarlos, lo que ahorra mucho tiempo en el proceso de arranque. Sin embargo, si no están marcados como limpios, comienza el laborioso proceso de comprobación de consistencia del sistema de ficheros. Puesto que los sistemas de ficheros de hoy en día son a menudo bastante grandes, y, por tanto, contienen enormes cadenas de archivos, directorios, y subdirectorios, cada uno de ellos utilizando bloques en el sistema de ficheros, verificar la consistencia de cada sistema de ficheros antes de montarlo es normalmente la parte más lenta del proceso de arranque de un ordenador. Evitar las comprobaciones de consistencia de los sistemas de ficheros es por tanto el sueño de todo administrador y un objetivo para todo diseñador de sistemas o de sistemas de ficheros. Este truco explora los conceptos básicos de cómo un tipo especial de sistema de ficheros, conocido como sistema de ficheros transaccional, agiliza los tiempos de reinicio del sistema eliminando en gran parte la necesidad de revisar la consistencia del sistema de ficheros.

Sistemas de ficheros transaccionales 101

Algunos de los más inspirados de entre nosotros podríamos mantener un diario (*journal*) para registrar lo que ocurre en nuestras vidas. Esto resulta útil si

queremos mirar atrás y ver qué nos estaba sucediendo en un punto específico en el tiempo. Los sistemas de ficheros transaccionales operan de una manera similar, escribiendo los cambios planificados a un sistema de ficheros en una parte especial del disco, llamada diario o bitácora, antes de efectivamente aplicarlos al sistema de ficheros. (Esto es difícil de hacer en un diario personal, a no ser que tenga poderes psíquicos.) Hay múltiples razones por las que los sistemas de ficheros registran cambios en un diario antes de aplicarlos, pero la razón principal para esto es garantizar su consistencia.

Utilizar un diario impone consistencia, ya que los conjuntos de cambios planificados están agrupados todos juntos en el diario y se repiten de manera transaccional en el sistema de ficheros. Cuando se aplican con éxito, el sistema de ficheros es consistente, y todos los cambios en el conjunto se eliminan del diario. Si el sistema falla mientras se aplica un conjunto de cambios al sistema de ficheros, las entradas permanecen presentes en el diario y se aplicarán como parte del montaje de ese sistema de ficheros cuando el equipo se recupere de nuevo.

Por tanto, el sistema de ficheros está siempre en estado consistente, o puede casi siempre hacerse consistente rápidamente repitiendo cualquier transacción pendiente.



Digo "casi siempre" porque un sistema de ficheros transaccional no puede protegerle de los bloques erróneos que aparezcan en sus discos o de fallos generales de hardware, los cuales pueden causar una pérdida o corrupción del sistema de ficheros. Véase el capítulo 10 para obtener algunas sugerencias en caso de que `fsck` no funcione para usted.

Sistemas de ficheros transaccionales bajo Linux

Linux ofrece un surtido de sistemas de ficheros transaccionales, pre-integrados en el código del núcleo de sistema primario. Dependiendo de la distribución Linux que esté usando, estos pueden no estar compilados en su núcleo o disponibles como módulos adicionales. Los sistemas de ficheros se activan en el núcleo de Linux en la sección File Systems de su mecanismo preferido de configuración del núcleo, accedido vía `make xconfig` o (para los reaccionarios) `make menuconfig`. Las opciones para el sistema de ficheros transaccional XFS se agrupan en una sección separada, XFS Support. Los sistemas de ficheros transaccionales que estaban integrados en el núcleo de Linux cuando se escribió este libro son los siguientes:

- **ext3:** ext3 añade habilidades transaccionales de alto rendimiento al sistema de ficheros estándar de Linux ext2 en el cual está basado. Los sistemas de ficheros ext2 existentes pueden ser convertidos a ext3 fácilmente, como se explica más adelante en este truco.

- **JFS (Journaled File System, Sistema de Ficheros Transaccional):** Desarrollado originalmente por IBM (*International Business Machines*) para ser usado en sus sistemas OS/2 y AIX. JFS es un sistema de ficheros transaccional de alto rendimiento que asigna espacio de disco según se necesita de unos fondos de almacenamiento disponible en el sistema de ficheros (conocidos como grupos de asignación) y por tanto crea i-nodos bajo demanda, en vez de pre-asignar todo como hacen los sistemas tradicionales Unix/Linux. Esto proporciona una rápida asignación de almacenamiento y además elimina la mayoría de las limitaciones en el número de i-nodos (y por tanto de ficheros y directorios) que pueden crearse en un sistema de ficheros JFS.
- **ReiserFS:** Escrito por Hans Reiser y otros con el soporte financiero de compañías como SUSE, Linspire, mp3.com, y muchas otras, ReiserFS es un sistema de ficheros transaccional de alto rendimiento y eficiente con el espacio que es especialmente ideal para sistemas de ficheros que contienen un gran número de archivos. ReiserFS fue el primer sistema de ficheros transaccional en ser integrado en el código del núcleo de Linux y ha sido por tanto popular y estable por bastante tiempo. Es el tipo de sistema de ficheros por defecto en distribuciones Linux tales como SUSE.
- **Reiser4:** Escrito por Hans Reiser y otros con el soporte financiero de DARPA (*Defense Advanced Research Projects Agency*, Agencia de Proyectos de Investigación Avanzados de Defensa), Reiser4 es el más nuevo de los sistemas de ficheros transaccionales discutidos en este truco. Reiser4 es un sistema de ficheros transaccional de muy alto rendimiento que incrementa más aún la ya extremadamente eficiente asignación de espacio proporcionada por ReiserFS. Está diseñado además para poder ser extendido a través de módulos adicionales que pueden añadir nuevas características sin cambiar el código central.
- **XFS:** Aportado a Linux por Silicon Graphics, Inc. (SGI), XFS (cuyas siglas realmente no quieren decir nada) es un sistema de ficheros transaccional de muy alto rendimiento que dinámicamente asigna espacio y crea i-nodos según se necesitan (como JFS), y soporta una sección especial (opcional) de tiempo real para ficheros que requieren una entrada/salida de alto rendimiento en tiempo real. La combinación de estas características proporciona un rápido sistema de ficheros sin limitaciones significantes en el número de i-nodos (y por tanto de ficheros y directorios) que pueden crearse en un sistema de ficheros XFS.

Cada uno de estos sistemas de ficheros tiene su propio comprobador de consistencia, herramienta de creación de sistemas de ficheros, y herramientas administrativas relacionadas. Incluso si su núcleo de sistema soporta el nuevo tipo de

sistema de ficheros que ha seleccionado, asegúrese de que sus sistemas de ficheros incluyen además sus utilidades administrativas, instaladas por separado por medio del gestor de paquetes de su distribución, o lo pasará mal la próxima vez que reinicie y se necesite una revisión del sistema de ficheros.

El propósito de este truco es explicar por qué los sistemas de ficheros transaccionales son una buena idea para la mayoría del almacenamiento local que está conectado a los sistemas de los que es responsable, y proporcionar algunos consejos sobre cómo integrar sistemas de ficheros transaccionales en sistemas existentes.

No puedo decir realmente más sobre estos aquí, sin convertir este truco en un tomo sobre sistemas de ficheros Linux, el cual ya escribí hace algunos años.

Todos estos sistemas de ficheros transaccionales están bien establecidos y han sido usados en sistemas Linux desde hace unos cuantos años. Reiser4 es el más nuevo de ellos y, por tanto, el menos probado, pero Hans nos asegura a todos que nadie hace ingeniería del software como el equipo Namesys.

Convertir sistemas de ficheros existentes a transaccionales

Los sistemas Linux tradicionales utilizan el sistema de ficheros ext2 para los sistemas de ficheros locales. Puesto que los sistemas de ficheros transaccionales disponibles para Linux utilizan sus propios mecanismos de asignación de i-nodos /almacenamiento, el único sistema de ficheros transaccional que puede comenzar a utilizar sin demasiado esfuerzo es el ext3, que diseñó para ser compatible con ext2.

Para convertir un sistema de ficheros ext2 existente en ext3, todo lo que tiene que hacer es añadir un diario y decirle a su sistema que ahora es un sistema de ficheros ext3, de tal manera que comience a usar dicho diario. El comando para crear un diario en un sistema de ficheros ext2 existente (como súper-usuario o utilizando sudo) es el siguiente:

```
# tune2fs -j /dev/filesystem
```



Si crea un diario en un sistema de ficheros ext2 montado, se generará inicialmente como el fichero `.journal` en la raíz del sistema de ficheros y se ocultará automáticamente cuando reinicie o monte de nuevo el sistema de ficheros como ext3.

Necesitará actualizar `/etc/fstab` para decirle al comando `mount` que monte su sistema de ficheros convertido como ext3 y reiniciar para verificar que todo está bien.

En general si quiere comenzar a usar cualquiera de los sistemas de ficheros transaccionales diferentes de ext3 discutidos en este capítulo con cualquier sistema existente, necesitará hacer lo siguiente:

- Incluir soporte para ese sistema de ficheros transaccional en su núcleo de Linux, hacerlo disponible como un módulo de núcleo adicional, o verificar que ya está soportado en su núcleo existente.
- Asegúrese de que actualiza los contenidos de cualquier disco RAM inicial que utilizó durante el proceso de arranque, para que incluya cualquier módulo del núcleo adicional para el nuevo sistema de ficheros que esté usando.
- Instale las herramientas administrativas asociadas con el nuevo tipo de sistema de ficheros, si no están todavía disponibles en su sistema. Éstas incluyen como mínimo nuevas utilidades `mkfs.tipo-sistema-de-ficheros` y `fsck.tipo-sistema-de-ficheros`, y podría incluir además nuevas utilidades administrativas y de reparación de sistemas de ficheros.
- Convierta manualmente sus sistemas de ficheros existentes al formato del nuevo sistema de ficheros transaccional creando nuevas particiones o volúmenes lógicos que sean como mínimo tan grandes como los existentes, dándoles formato usando el tipo del nuevo sistema de ficheros, y copiando de manera recursiva los contenidos de sus sistemas de ficheros existentes en los nuevos.
- Entre en modo mono-usuario, desmonte sus sistemas de ficheros existentes, y actualice las entradas en `/etc/fstab` para reflejar los nuevos tipos de sistema de ficheros (y los nuevos discos/volúmenes donde estén ubicados, a menos que esté simplemente reemplazando un disco existente con uno o más discos nuevos).

Al migrar los contenidos de las particiones y volúmenes existentes en diferentes formatos de sistemas de ficheros, haga siempre primero una copia de seguridad de todo, y pruebe cada una de las nuevas particiones antes de acabar con su predecesor. Olvidar cualquiera de los pasos en la lista anterior puede convertir su bienintencionada experiencia de mejora de sistema en una pesadilla en el reinicio, si su sistema no arranca correctamente usando sus nuevos y sexy sistemas de ficheros.

Resumen

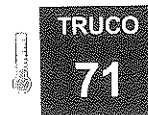
Los sistemas de ficheros transaccionales pueden mejorar significativamente los tiempos de reinicio, proporcionan un uso más eficiente del espacio de disco

disponible en sus volúmenes o particiones, y a menudo incluso incrementan el rendimiento general del sistema. Personalmente tiendo a usar ext3 para los sistemas de ficheros de sistema como / y /boot, ya que esto me permite usar todas las utilidades de reparación ext2 estándar si dichos sistemas de ficheros se volvieran corruptos. Para almacenamiento local en los sistemas SUSE, generalmente utilizo ReiserFS, porque es la opción por defecto y es estupendo para particiones de sistema (tales como sus colas de impresión y correo) debido a su súper eficiente mecanismo de asignación.

Tiendo a usar XFS para particiones físicas en distribuciones Linux diferentes a SUSE, porque lo he usado durante años en equipos Linux y SGI. Según mi experiencia siempre ha sido estable, y la sección de tiempo real de los sistemas de ficheros XFS es realmente genial. Generalmente utilizo ext3 en volúmenes lógicos, porque los mecanismos de asignación dinámica usados por JFS y HFS así como los algoritmos de árboles balanceados de ReiserFS ponen una sobrecarga extra en el subsistema de volumen lógico. Todos ellos funcionan bien en volúmenes lógicos, por supuesto.

Véase también

- man tune2fs
- Página Web de ext3: <http://e2fsprogs.sourceforge.net/ext2.html>
- Página Web de JFS: <http://jfs.sourceforge.net>
- Página Web de ReiserFS/Reiser4: <http://www.namesys.com>
- Página Web de XFS: <http://oss.sgi.com/projects/xfs/>



Optimice y comprenda completamente su sistema con sysctl

En vez de interactuar directamente con ficheros /proc, puede obtener y establecer opciones del núcleo de sistema en un abrir y cerrar de ojos con el comando sysctl.

En la antigüedad, sysctl hacía referencia a un fichero de cabecera o a una llamada que los programadores en C podían usar para cambiar los valores del núcleo de sistema desde un programa. Los ficheros bajo /proc/sys/ a menudo son referidos colectivamente como la interfaz sysctl, porque se puede escribir en ellos, y los cambios hechos a los ficheros serán recogidos por el núcleo en ejecución sin necesidad de reiniciar. Esta característica se implementó en el núcleo de sistema como pronto en la versión 2.0 (pero no me haga mucho caso).

En estos días, sysctl es una llamada al núcleo de sistema, una interfaz, y un comando que permite a los administradores interactuar con el núcleo con facilidad. Además cuenta con un decente fichero de configuración de inicio, de tal manera que no tendrá que recompilar núcleos cada vez que quiera desactivar reenvíos de IP, por ejemplo. Activar y desactivar el reenvío de IP fue una de las primeras cosas para las que utilicé la interfaz sysctl. Activar el reenvío de IP para su router Linux solía hacerse con un comando como éste:

```
# echo 1 > /proc/sys/net/ipv4/ip_forward
```

El contenido del fichero era "0" por defecto, indicando que el reenvío no estaba activado. Redirigiendo la salida de un comando echo con un "1" al fichero lo activamos. Entra en escena el comando sysctl. Ahora todos podemos ver fácilmente cada opción disponible por medio de la interfaz con un simple comando:

```
# sysctl -a
net.ipv4.tcp_keepalive_time = 7200
net.ipv4.ipfrag_time = 30
net.ipv4.ip_dynaddr = 1
net.ipv4.ipfrag_low_thresh = 196608
net.ipv4.ipfrag_high_thresh = 262144
net.ipv4.tcp_max_tw_buckets = 180000
net.ipv4.tcp_max_orphans = 16384
net.ipv4.tcp_synack_retries = 5
net.ipv4.tcp_syn_retries = 5
net.ipv4.ip_nonlocal_bind = 0
net.ipv4.ip_no_pmtu_disc = 0
net.ipv4.ip_autoconfig = 0
net.ipv4.ip_default_ttl = 64
net.ipv4.ip_forward = 0
...
```

En mi sistema de escritorio Debian, esto devolvía más de 400 registros con el formato "clave=valor". Las claves a la izquierda son representaciones con puntos de las rutas de los ficheros bajo /proc/sys/. Por ejemplo, el valor para "net.ipv4.ip_forward" puede encontrarse en /proc/sys/net/ipv4/ip_forward. Aunque, si sabe lo que está buscando, puede especificar lo que quiere como un argumento a sysctl:

```
# /sbin/sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
```

Así que si siempre quiso saber más sobre su núcleo de sistema, considérelolo hecho. ¿Y sobre personalizar los valores del núcleo? Tiene varias opciones. Puede hacer cambios temporales en el núcleo utilizando la opción "-w" o "escribir" un nuevo valor:

```
# sysctl -w net.ipv4.ip_forward=1
```


Por otra parte, si quiere hacer un cambio más permanente, puede poner sus valores personales en el fichero `/etc/sysctl.conf`, lo que asegurará que se apliquen automáticamente cuando el núcleo arranque. (En realidad, no se lee exactamente al arrancar el núcleo per se, pero sí en cierto punto antes de que se muestre el cuadro de diálogo de inicio de sesión en la consola. Exactamente cuándo se activan las variables varía de distribución en distribución, ipero si utiliza `grep` con el parámetro "sysctl" bajo `/etc/init.d`, seguro que lo encuentra rápidamente!)

El fichero de configuración consiste en registros que se ven idénticos a la salida de `sysctl -a`. He aquí un fichero de configuración de ejemplo:

```
# Controls IP packet forwarding
net.ipv4.ip_forward = 0

# Controls source route verification
net.ipv4.conf.default.rp_filter = 1

# Controls the System Request debugging functionality of the kernel
kernel.sysrq = 0

# Controls whether core dumps will append the PID to the core filename.
# Useful for debugging multi-threaded applications.
kernel.core_uses_pid = 1

# Decrease the time default value for tcp_fin_timeout connection.
net.ipv4.tcp_fin_timeout = 30

# Decrease the time default value for tcp_keepalive_time connection
net.ipv4.tcp_keepalive_time = 1800

# Turn off tcp_window_scaling
net.ipv4.tcp_window_scaling = 0

# Turn off the tcp_sack
net.ipv4.tcp_sack = 0

# Turn off tcp_timestamps
net.ipv4.tcp_timestamps = 0

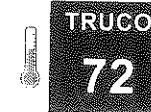
# Increase transport socket buffers to improve performance of nfs (and
networking
# in general)
# 'rmem' is 'read memory', 'wmem' is 'write memory'.
net.core.rmem_max = 262143
net.core.rmem_default = 262143
net.core.wmem_max = 262143
net.core.wmem_default = 262143

net.ipv4.tcp_rmem = 4096      87380   8388608
net.ipv4.tcp_wmem = 4096      87380   8388608

# These are for both security and performance

net.ipv4.icmp_echo_ignore_broadcasts = 1
net.ipv4.icmp_ignore_bogus_error_responses = 1
```

Cuando todo esté dicho y hecho, la parte más difícil en utilizar la interfaz `sysctl` es aprender qué quieren decir realmente todas las variables y cómo se aplican a su situación en particular. Revise además la documentación de los ficheros `/proc` que viene con la distribución de la fuente del núcleo de sistema para empezar.



TRUCO

72

No pierda detalle, con pantallas múltiples

Utilizar dos monitores en un solo equipo le da más espacio para trabajar. Las últimas versiones del sistema X Window hacen esto más fácil que nunca.

Muchos de los trucos en este libro tratan sobre cómo monitorizar mejor el estado de sistema y de proceso, cómo usar la Web para funciones básicas de infraestructura informática, etc. Este truco explica cómo conseguir suficiente espacio de pantalla, de tal manera que pueda realmente ver toda la información conectando dos tarjetas de video y dos monitores a cualquier sistema Linux, y configurando el sistema X Window XFree86 o X.org para lo que se conoce como pantalla multi-cabeza.



Siempre que sea posible, añada una segunda tarjeta gráfica del mismo tipo que la que ya está en su sistema, o reemplace la existente por una que admita dos monitores. Esto le permitirá usar el mismo servidor X para controlar ambas tarjetas gráficas y sus pantallas asociadas. De manera similar, es una buena idea añadir un segundo monitor de exactamente el mismo tamaño y con exactamente la misma resolución máxima que el suyo existente. Esto simplificará el sincronizar modos gráficos a través de los dos monitores (y en las secciones de configuración del sistema X Window para cada pantalla).

Este truco crea dos pantallas separadas, una en cada uno de sus monitores. Un enfoque alternativo sería utilizar la extensión Xinerama del sistema X Window para crear una sola pantalla que abarque dos monitores.

Vea <http://www.tldp.org/HOWTO/Xinerama-HOWTO/> para más información sobre Xinerama. Con dos pantallas separadas no puede mover ventanas de una a otra, si bien puede crear ventanas en una pantalla determinada, especificando la que quiere usar en una línea de comandos de una aplicación X. Encuentro Xinerama desconcertante, porque las ventanas se pueden partir entre las dos pantallas, lo que las hace un poco difíciles de leer debido a la carcasa de mis monitores.

Considero las pantallas separadas más fáciles de usar y de aspecto más limpio. El provecho que usted le saque puede variar.

La información de configuración del sistema X Window se almacena en el fichero `/etc/X11/xorg.conf` si está usando el servidor X11 de X.org, o en `/etc/X11/XF86Config` si está usando un servidor X11 basado en XFree86. Tras añadir el hardware a su sistema y arrancar en modo multi-usuario, no gráfico, tal como el nivel de ejecución 3, el procedimiento para modificar este fichero para que use una pantalla multi-cabeza es tan simple como seguir los siguientes pasos:

Primero, necesita crear dos secciones "Monitor" en el fichero de configuración de su servidor X. Asegúrese de que utiliza un nombre único para el valor de "Identifier" para cada monitor:

```
Section "Monitor"
    Identifier      "Monitor 0"
    VendorName      "Monitor Vendor"
    ModelName       "Model X"
    HorizSync       30.0 - 50.0
    VertRefresh     60.0 - 60.0
EndSection

Section "Monitor"
    Identifier      "Monitor 1"
    VendorName      "Monitor Vendor"
    ModelName       "Model Y"
    HorizSync       30.0 - 50.0
    VertRefresh     60.0 - 60.0
EndSection
```

A continuación, cree una sección "Device" para cada tarjeta gráfica en su sistema. Como con los monitores, asegúrese de que utiliza un valor único para "Identifier" para cada tarjeta gráfica:

```
Section "Device"
    Identifier      "VideoCard 0"
    Driver          "drivername"
    VendorName      "Vendor"
    BusID           "PCI:00:15:0"
EndSection

Section "Device"
    Identifier      "VideoCard 1"
    Driver          "drivername"
    VendorName      "Vendor"
    BusID           "PCI:1:0:0"
EndSection
```

La opción "BusID" permite al servidor definir correctamente, y de manera única, cada pantalla en su fichero de configuración, y su valor puede encontrarse en la salida del comando `lspci`. La opción "BusID" puede encontrarse al comienzo de

cada primera línea de la salida de `lspci` que identifica la tarjeta gráfica. El formato es ligeramente diferente del que necesitará poner en su fichero de configuración: `lspci` informa en hexadecimal, mientras que usted deberá utilizar notación decimal en su fichero. Además la salida del comando `lspci` es `xx:yy.z`, y deberá expresarla como `xx:yy:z`; fíjese que el punto en la salida de `lspci` debe reemplazarse con dos puntos en su fichero de configuración.

```
# lspci | grep VGA
00:0f.0 VGA compatible controller: nVidia Corporation NV11 [GeForce2 MX/MX
400] (rev b2)
01:00.0 VGA compatible controller: nVidia Corporation NV15 [GeForce2 GTS/
Pro] (rev a4)
```



Mi herramienta favorita para convertir de hexadecimal a decimal es la utilidad estándar de Linux `bc`. Puede especificar la base de entrada de `bc` utilizando el comando `ibase=base` y dejar su salida como decimal (valor por defecto). Por ejemplo, lo siguiente muestra cómo convertir un 10 hexadecimal a decimal (de acuerdo, esto no es muy difícil, pero es un ejemplo, y un ejemplo simple aclara más las cosas):

```
$ bc -q
ibase=16
10
16
```

Tras especificar la base de entrada, simplemente introduzca un valor hexadecimal y pulse **Intro**, `bc` mostrará el equivalente en decimal. Pulse **Control-D** para salir de `bc`.

Lo siguiente a añadir al fichero de configuración de su servidor son dos secciones "Screen". Cada sección usará una de las estancias de "Monitor" y "Device" que ha definido anteriormente. La resolución y profundidad de color de las dos podría ser diferente si lo desea, pero normalmente son las mismas:

```
Section "Screen"
    Identifier      "Screen 0"
    Device          "VideoCard 0"
    Monitor         "Monitor 0"
    DefaultDepth    24
    SubSection      "Display"
        Depth       24
        Modes        "800x600" "640x480"
    EndSubSection
EndSection

Section "Screen"
    Identifier      "Screen 1"
    Device          "VideoCard 1"
```

```

Monitor      "Monitor 1"
DefaultDepth 24
SubSection "Display"
    Depth 24
    Modes "1024x768" "800x600" "640x480"
EndSubSection
EndSection

```

Ahora debe ligar todas estas piezas juntas en la sección "ServerLayout" (normalmente al principio de su fichero de configuración):

```

Section "ServerLayout"
    Identifier      "Multihead layout"
    Screen 0        "Screen 0" 0 0
    Screen 1        "Screen 1" RightOf "screen 0"
    InputDevice     "Mouse0" "CorePointer"
    InputDevice     "Keyboard0" "CoreKeyboard"
    InputDevice     "DevInputMice" "AlwaysCore"
EndSection

```

El "0 0" próximo a "Screen 0" significa que esta pantalla comenzará en la posición 0,0. "Screen 1" se ubicará a la derecha de "Screen 0".

Ahora que ha terminado, inicie el sistema X Window utilizando su comando favorito `startx` o `xinit`. Si X no inicia correctamente, revise dos veces las entradas que ha añadido a su fichero de configuración para encontrar errores sintácticos, prestando atención en particular al los valores de "BusID" en las estrofas "Device".



Redirigir la salida del comando `startx` o `xinit` a un fichero puede ayudar a capturar mensajes de error que puede utilizar para depurar sus ficheros de configuración. Ejecutar `xinit >& inicio_x.txt` puede ser extremadamente útil, a menos que pueda leer mucho más rápido que yo.

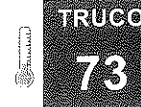
Una vez que X está funcionando correctamente, puede iniciar una aplicación gráfica de tal manera que lo haga en la pantalla de su elección usando la opción "-display", que es aceptada en casi todos los comandos del sistema X Window. Por ejemplo, para iniciar un `xterm` en "Screen 1", ejecutaría el comando `xterm -display :0.1`. Este valor "display" especifica que la aplicación usa "Screen 1" de la pantalla actual (display 0) en el equipo actual. El formato general de un valor "display" es el siguiente:

`equipo:número_monitor.número_pantalla`

Utilizar una pantalla multi-cabeza puede ser un poco desconcertante al principio, especialmente cuando el puntero de su ratón cruza de un monitor a otro,

pero encontrará rápidamente que el estado real de la pantalla adicional es bien merecedor de cualquier esfuerzo de adaptación.

-Lance Tost



TRUCO

73

Maximice los recursos con un gestor de ventanas minimalista

Utilizar gestores de ventanas en lugar de gestores de escritorio puede aumentar el rendimiento de sistemas lentos, o simplemente dejar más recursos de sistema disponibles para computación real.

Las interfaces gráficas de usuario como KDE y GNOME son entornos muy logrados y fáciles de usar, pero todo lo que es delicioso para la vista tiene un precio, ejecutar y gestionar todas esas campanillas y silbidos gráficos requiere un cierto porcentaje de recursos de sistema. Un escritorio típico KDE ocioso en SUSE 9 Enterprise ocupa alrededor de 370 MB of RAM. Para los servidores de hoy en día con múltiples gigabytes de RAM, esto puede no suponer un problema. Sin embargo, si está ejecutando un servidor heredado que contiene menos de un giga de RAM, podría ciertamente beneficiarse del uso de un sistema de gráficos más modesto, conocido como un gestor de ventanas. Los gestores de ventanas se centran en mostrar y gestionar ventanas, no en arrastrar y pegar y otros lujos. Uno de los mejores gestores de ventana en la categoría de "peso ligero" es Fluxbox, un paquete de software de código abierto disponible online y derivado del gestor de ventanas Blackbox, el cual es a su vez un clon de código abierto del gestor de ventanas usado en las viejas estaciones de trabajo de NeXT.

Utilizar Fluxbox puede disminuir la cantidad de RAM requerida por su interfaz gráfica en más de 100 MB, y además elimina los diez trillones de procesos en segundo plano, que los entornos de escritorio como KDE arrancan para dar soporte a cosas tales como arrastrar y pegar, asociaciones automáticas de ficheros, etc. Este truco explica cómo compilar e instalar Fluxbox de tal manera que pueda dedicar más memoria de su sistema a las aplicaciones que realmente desea ejecutar.

Obtener e intalar Fluxbox

Como es habitual, el método más fácil de instalar Fluxbox es vía un RPM empaquetado para su distribución de Linux. Estos pueden encontrarse en la página Web de Fluxbox <http://Fluxbox.sourceforge.net>.

En este ejemplo, compilaremos desde el código fuente de tal manera que podamos pasar unas cuantas opciones para hacer Fluxbox un poco más familiar. Coja el fichero .tar de la página Web, y extraígallo en un directorio de trabajo.

Navegue hasta el directorio recién creado, y ejecute `configure` como sigue:

```
$ ./configure --with kde --with-gnome
```

Esto permitirá a Fluxbox utilizar los iconos del panel de KDE y GNOME. Una vez que el `script` `configure` ha terminado ejecute el siguiente comando como súper-usuario para compilar Fluxbox:

```
# make && make install
```

Esto compilará Fluxbox (lo cual no requiere privilegios de súper-usuario) y lo instalará por usted (lo que sí que requiere privilegios de súper-usuario, puesto que tiene que ser capaz de escribir en subdirectorios de `/usr/local`). El nombre del ejecutable real para Fluxbox es `fluxbox` (la primera en minúscula). Ahora simplemente necesita configurar X para iniciar Fluxbox como su gestor de ventanas.

¡Iníciame Scotty!

Si ha instalado en un sistema SUSE o Red Hat utilizando un RPM, puede simplemente seleccionar Fluxbox como su tipo de sesión desde la pantalla de inicio de sesión. De otra manera, navegue a su directorio personal y encuentre un fichero llamado bien `.xsession` o `.xinitrc`. Si no existe ningún fichero con ese nombre, necesitará crear uno. Cuál de ellos crea depende en gran parte de cómo su sistema inicia X. Vea la documentación de Fluxbox para más información. En este caso, editaremos `.xinitrc`. Ábralo con su editor de texto favorito, e introduzca la siguiente línea:

```
exec /usr/local/bin/fluxbox
```

donde `/usr/local/bin` es el directorio en el cual instaló su ejecutable `fluxbox` (`/usr/local/bin` es normalmente el lugar de instalación por defecto). Necesitará entonces cambiar las propiedades de propietario vía `chmod`:

```
$ chmod 700 .xinitrc
```

Ahora puede cerrar su sesión y volver a iniciarla. Dependiendo de su distribución, `fluxbox` se iniciará automáticamente, o será capaz de seleccionarlo como su tipo de sesión desde el gestor de inicio de sesión. De cualquiera de las maneras, al iniciar sesión será recibido (¡muy rápidamente!) por una pantalla de aspecto muy simple. Haciendo clic con el botón derecho sobre el escritorio se le ofrece un menú con varias opciones en él. Si lo configuró con las opciones de KDE y GNOME como le sugerí, algunas de las herramientas de estos entornos podrían estar directamente a su disponibilidad. La figura 8.1 muestra una pantalla Fluxbox de ejemplo, ejecutando un solo `xterm` con el navegador Firefox abierto, y mostran-

do mi menú Fluxbox por defecto, como resultado de hacer clic con el botón derecho sobre el escritorio.

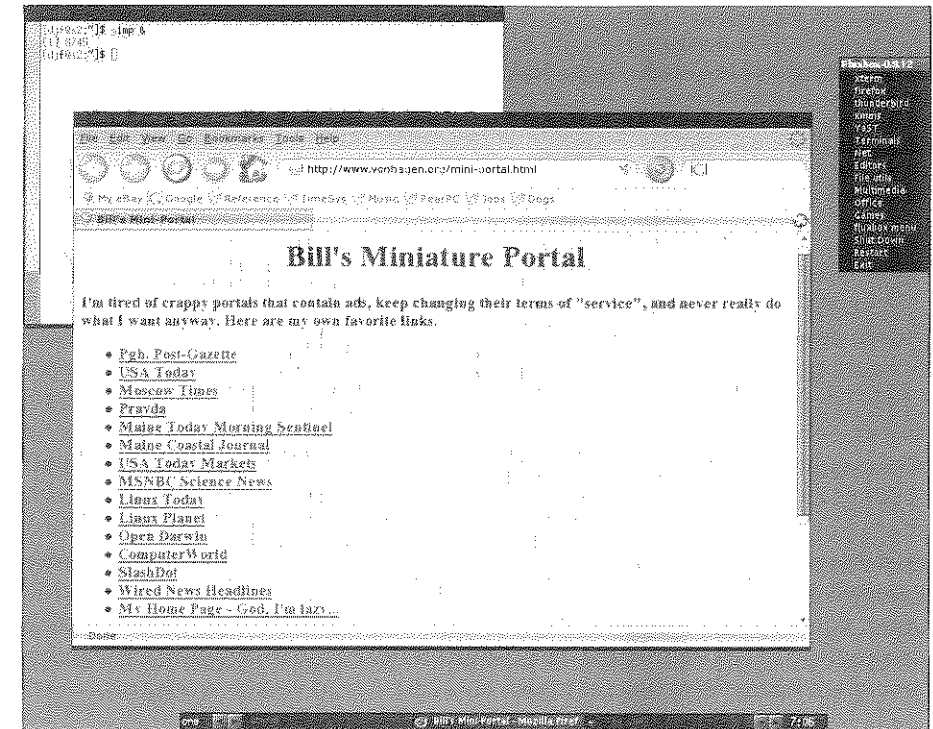


Figura 8.1. Fluxbox en todo su minimalista esplendor.

Configurar Fluxbox

El siguiente paso es comenzar a personalizar Fluxbox a su gusto. Como puede ver en la figura 8.1, Fluxbox es altamente configurable. La mayor parte de la configuración de Fluxbox consiste en personalizar su menú principal. Este es el menú que se muestra en cualquier parte del escritorio que haga clic, y es completamente configurable. El menú se controla vía un fichero de texto llamado `menu`. Este fichero se encuentra en su directorio `.fluxbox`, el cual se crea automáticamente en su directorio personal la primera vez que ejecuta Fluxbox. El formato del fichero es muy simple:

```
[begin] (Fluxbox)
[exec] (xterm) (xterm)
```

```
[exec] (mozilla) {mozilla}
[exec] (Run) {fbrun}
[submenu] (Terminals)
[exec] (xterm) {xterm}
[exec] (gnome-terminal) {gnome-terminal}
[exec] (console) {console}
[end]
[submenu] (Net)
[submenu] (Utilities)
[exec] (Ethereal) {ethereal}
[submenu] (Browsers)
[exec] (mozilla) {mozilla}
[exec] (conqueror) {kfmclient openProfile Web browsing}
[end]
[end]
```

Este ejemplo es una sección de muestra de mi fichero menu de Fluxbox. Como puede ver, es un fichero de configuración bastante simple. La primera línea es el título de mi menú. En este ejemplo, cada elemento del menú está precedido por el comando [exec], el cual le dice a Fluxbox que éste es un comando de sistema que realmente debería ejecutarse. El primer argumento entre paréntesis regulares es el nombre que quiere que se muestre para la aplicación, mientras que el texto entre llaves especifica el comando que ejecutaría desde la línea de comandos para ejecutar el programa. Fíjese que si la aplicación que está intentando añadir a su menú no se encuentra dentro de su ruta por defecto, necesitará especificar la ruta completa al ejecutable. Cada porción secuencialmente más baja de su menú se marca con el comando [submenu]. Especifique el final de un menú con el comando [end]. Los elementos que quiera tener disponibles directamente pueden ponerse al principio, bajo el título.

Los ficheros de inicio de Fluxbox pueden además invocar comandos internos del propio Fluxbox, los cuales se identifican entre corchetes exactamente igual que en la instrucción [exec], como en el siguiente ejemplo:

```
[exit] (Exit)
```

Esto crea un elemento de menú llamado "Exit" que ejecuta el comando interno de Fluxbox "exit". Al desplegar sistemas para usuarios que ejecutan Fluxbox e inician en niveles de ejecución gráficos, podría encontrar los siguientes comandos de menú de Fluxbox bastante prácticos:

```
[exec] (Shut Down System) {sudo shutdown -h now}
[exec] (Reboot System) {sudo shutdown -r now}
[exit] (Log Out)
```

Estos asumen que al usuario se le han otorgado ciertos privilegios en la aplicación sudo, y crea los tipos de elementos de menú estándar, que los usuarios

que puedan no estar familiarizados con Linux normalmente esperan ver en su interfaz gráfica.



Las opciones discutidas en esta sección son tan sólo la punta del iceberg de configuración de Fluxbox. Vea la página de manual de Fluxbox para una lista completa de comandos de configuración y opciones disponibles.

El "Slit"

El "Slit" es una de las mejores características de Fluxbox. Puede pensar en el "Slit" como una versión del salpicadero de Mac OS X que siempre está siempre disponible. Contiene pequeñas aplicaciones acoplables (conocidas comúnmente como "aplicaciones de banquillo" (*dock apps*)) que son capaces de ejecutar en modo retraído, lo cual significa simplemente que pueden ejecutar de manera independiente en segundo plano. Esto se designa normalmente con la opción "-w" cuando se ejecuta la aplicación desde la línea de comandos. Fíjese que no todas las aplicaciones pueden ejecutar de esta manera, pero muchas están especialmente diseñadas para hacerlo de este modo. Yo normalmente arranco todas las aplicaciones de banquillo que quiero ejecutar, poniéndolas en mi fichero .xinitrc, iniciándolas en segundo plano antes de iniciar efectivamente el gestor de ventanas. El orden en el cual las aplicaciones aparecen en el "Slit" se define poniendo sus nombres en el orden deseado en el fichero slitlist de su directorio .fluxbox.

El "Slit" es un modo excepcional de mostrar estadísticas, tales como la utilización de memoria y procesador, utilizando las "aplicaciones de banquillo" adecuadas.

Puede encontrar este tipo de aplicaciones en las dos direcciones siguientes: <http://freshmeat.net> y <http://www.dockapps.org>.

¡Adórnalo!

Hay una amplia comunidad de gente en Internet que dedica un montón de tiempo en crear temas de Fluxbox personalizados. Estos temas pueden encontrarse en la página Web de Fluxbox, así como a lo largo y ancho de la Red. Instalar un tema es tan simple como descargarlo y añadirlo al directorio ~/.fluxbox/styles.

Estos estilos se podrán seleccionar entonces desde el submenú Fluxbox>Menu. Si dicho subdirectorio no existe, busque su directorio share global de Fluxbox (normalmente /usr/local/share/Fluxbox). La ubicación de este directorio puede variar dependiendo de su método de instalación.

Mínimas dificultades

Después de un poco de configuración, podría encontrar que prefiere el simple formato de Fluxbox a gestores de ventanas más pesados como GNOME o KDE. Además de conservar recursos de sistema, Fluxbox es una gran aplicación a usar para extender la vida de un viejo portátil o máquina de escritorio que simplemente no puede llegar a las altas demandas de una solución de escritorio más pesada.



Otro consejo relacionado con interfaces gráficas para ahorrar memoria es iniciar su sistema en un nivel de ejecución no gráfico (normalmente el nivel 3) y entonces arrancar manualmente su gestor de ventanas utilizando los comandos `xinit` o `startx` tras iniciar sesión. Esto elimina la sobrecarga de memoria de los gestores de pantalla `xwm`, `kdm`, o `gdm`, que son los procesos que proporcionan soporte a inicios de sesión gráficos, y pueden ahorrarle otros 80 MB de memoria más o menos. Vea la página de manual de `xinit` para más información.

Véase también

- <http://fluxbox.sourceforge.net>
- <http://www.dockapps.org>
- `man fluxbox`
- `man xinit`
- `man sudo`

—Brian Warshawsky



Truco 74: Retratar sus sistemas usando /proc

El sistema de ficheros `/proc` contiene gran riqueza de información, y, con un poco de programación `script`, puede utilizarla para crear perfiles de sus servidores.

La clave para reconocer irregularidades en su servidor es tener un buen entendimiento y conocimiento de cómo se ven las cosas cuando está "sano". Un lugar estupendo para empezar la búsqueda de información es el sistema de ficheros `/proc`. Este sistema de ficheros es un portal hacia lo más profundo de cómo se ve el núcleo en ejecución y la carga del sistema, y proporciona un retrato completo del hardware en uso en el sistema local.

Cuando instalo un nuevo servidor, una de las primeras cosas que hago es tomar una especie de instantánea de perfil, de tal manera que pueda obtener una buena imagen de cómo se ven los recursos de sistema en un equipo ocioso. Hago también esto justo antes y después de instalar o encender nuevo software o servicios de sistema, de tal manera que pueda obtener una medida del impacto de una aplicación en la disponibilidad de los recursos, y tener así una "chuleta" en la que mirar el hardware instalado en el sistema.

El `script` que uso está muy poco pulido y no se escribió para que funcione en cualquier máquina en la que pueda ejecutarlo, pero funciona en un buen número de servidores Linux con los que me he encontrado. Echemos un vistazo a cada parte del `script`, junto con la salida que produce.

Lo primero que hace el `script` es registrar la información de nombre de equipo y de versión del núcleo, junto con las primeras líneas de salida del comando `top`, de tal manera que pueda ver la carga, número de usuarios/procesos, etc.:

```
#!/bin/bash
echo ""
echo "#####BASIC SYSTEM INFORMATION#####"
echo HOSTNAME: 'cat /proc/sys/kernel/hostname'
echo DOMAIN: 'cat /proc/sys/kernel/domainname'
echo KERNEL: 'uname -r'
top -b | head -8
```

He aquí la salida para esta parte del `script`:

```
#####BASIC SYSTEM INFORMATION#####
HOSTNAME: willy
DOMAIN: pvt
KERNEL: 2.4.21-32.0.1.ELsmp
```

```
22:53:14 up 7 days, 15:36, 12 users, load average: 0.00, 0.02, 0.00
114 processes: 113 sleeping, 1 running, 0 zombie, 0 stopped
CPU states:  cpu  user  nice  system  irq  softirq  iowait  idle
              total 0.0%  0.0%   0.4%  0.0%   0.0%    6.8%  92.6%
              cpu00 0.0%  0.0%   0.9%  0.0%   0.0%    7.8%  91.1%
              cpu01 0.0%  0.0%   0.0%  0.0%   0.0%    7.8%  92.1%
```

La información del nombre de equipo está ahí, así que sabré a lo que estoy mirando cuando haga referencia a la salida de nuevo en el futuro. El dominio listado aquí es en realidad el dominio NIS al que el equipo está vinculado. Dependiendo del entorno, esto puede ser un poco de información útil para resolver problemas, pero si usted está en un entorno NIS, esto ya lo sabe. Lo que probablemente debe estarse preguntando es por qué me he molestado en usar `/proc` para esto en vez de comandos de sistema para obtener la información de nombre de equipo y de dominio. La respuesta es porque he encontrado que utilizar los

ficheros bajo /proc es más fiable que asumir que los comandos de sistema están en su ruta por defecto. Para cosas como hostname es probable que esté ahí, pero se pueden instalar tres herramientas diferentes para la información de nombre de dominio. Un equipo Red Hat típico tiene domainname, ypdomainname, y dnsdomainname. En sistemas Red Hat, estas son todas enlaces simbólicos al comando hostname.

En mi máquina estable Debian, no hay ningún comando domainname en absoluto. Sin embargo el fichero /proc/sys/kernel/domainname está en la mayoría de las máquinas con las que trabajo, así que usarlo hace el *script* más flexible.

A continuación, echemos un vistazo a la parte de *script* que reúne información del sistema de ficheros:

```
echo "##### FILESYSTEM INFORMATION #####"
echo ""
echo "SUPPORTED FILESYSTEM TYPES:"
echo -----
echo 'cat /proc/filesystems | awk -F\t' '{print $2}'
echo ""
echo "MOUNTED FILESYSTEMS:"
echo -----
cat /proc/mounts
```

De nuevo, he aquí la salida:

```
SUPPORTED FILESYSTEM TYPES:
-----
sysfs rootfs bdev proc sockfs pipefs futexfs tmpfs eventpollfs devpts ext2
ramfs iso9660 devfs mqueue usbfs ext3 reiserfs supermount vfat

MOUNTED FILESYSTEMS:
-----
/dev/root / reiserfs rw 0 0
none /dev devfs rw 0 0
none /proc proc rw,nodiratime 0 0
sysfs /sys sysfs rw 0 0
devpts /dev/pts devpts rw 0 0
tmpfs /dev/shm tmpfs rw 0 0
usbfs /proc/bus/usb usbfs rw 0 0
none /dev/shm tmpfs rw 0 0
/dev/hdb1 /mnt/hdb1 ext3 rw,noatime 0 0
/dev/hdb2 /mnt/hdb2 reiserfs rw,noatime 0 0
```

Esta no es información que probablemente cambie en un servidor autónomo, pero en un entorno amplio con muchos montajes NFS y ejecutando montadores automáticos, puede ser una información útil de tener. La información de sistema de ficheros soportada es también práctica si está en una empresa que compila sus propios núcleos de sistema, ya que le permitirá saber si su nuevo administrador

comete el típico error del novato de olvidar añadir soporte para ext3 o vfat al núcleo. El siguiente fragmento es sólo ligeramente más complicado. Resume la información sobre dispositivos IDE, sus números de modelo, los dispositivos de sistema a los que están asignados (hda, hdb, etc.), y, en caso de que no reconozca los modelos, exactamente qué tipo de dispositivo son. He aquí la porción de dispositivos IDE del *script*:

```
echo "IDE DEVICES BY CONTROLLER"
echo -----
for i in `ls /proc/ide | grep ide`
do
    echo $i:
    for j in `ls /proc/ide/$i | grep hd`
    do
        echo ""
        echo "    $j"
        echo "    -----"
        echo "    model: 'cat /proc/ide/$i/$j/model'"
        echo "    driver: 'cat /proc/ide/$i/$j/driver'"
        echo "    device type: 'cat /proc/ide/$i/$j/media'"
        if { -e /proc/ide/$i/$j/geometry }; then
            echo "    geometry: 'cat /proc/ide/$i/$j/geometry'"
        fi
    done
done
```

Y aquí está la salida:

```
##### IDE SUBSYSTEM INFORMATION #####

IDE DEVICES BY CONTROLLER
-----
ide0:

    hdb
    -----
    model: ST3200822A
    driver: ide-disk version 1.18
    device type: disk

ide1:

    hdd
    -----
    model: FX4830T
    driver: ide-cdrom version 4.61
    device type: cdrom
```

Esto me dice que hay dos controladoras IDE: una unidad de CD-ROM, y un disco duro IDE en la máquina. Sé además que la unidad de CD estará disponible

para montaje como `/dev/hdd` (algo que podría ser menos obvio en una máquina con muchos dispositivos IDE). ¡Tenga en mente que podría haber obtenido incluso más información si hubiera querido requerir privilegios de súper-usuario para ejecutar este *script*! Por ejemplo, para ver la configuración de `/dev/hdb`, necesito ser súper-usuario. Y entonces puedo ejecutar este comando:

```
# cat /proc/ide/hdb/settings
```

Esto me dará más información de la que podría querer saber sobre mi disco duro. He aquí una muestra:

name	value	min	max	mode
----	----	---	----	----
acoustic	0	0	254	rw
address	1	0	2	rw
bios_cyl	24321	0	65535	rw
bios_head	255	0	255	rw
bios_sect	63	0	63	rw
bswap	0	0	1	r
current_speed	66	0	70	rw
failures	0	0	65535	rw
init_speed	66	0	70	rw
io_32bit	1	0	3	rw
keepsettings	0	0	1	rw
lun	0	0	7	rw
max_failures	1	0	65535	rw
multcount	16	0	16	rw
nicel	1	0	1	rw
nowerr	0	0	1	rw
number	1	0	3	rw
pio_mode	write-only	0	255	w
unmaskirq	1	0	1	rw
using_dma	1	0	1	rw
wcache	1	0	1	rw

Hay una tonelada de información en los ficheros bajo `/proc`. Los *script* como éste se pueden extender mucho y constituir una herramienta maravillosa para administradores consultores. Envíelo a un cliente y haga que le mande la salida por correo electrónico, o utilícelo para tomar una instantánea de una máquina cuando la configura de tal manera que cuando le llame un cliente esté preparado con la información sobre el equipo en cuestión.

El código

He aquí una copia del *script* completo de una pieza, para revisarlo fácilmente:

```
#!/bin/bash
echo ""
```

```
echo "#####BASIC SYSTEM INFORMATION#####"
echo HOSTNAME: 'cat /proc/sys/kernel/hostname'
echo DOMAIN: 'cat /proc/sys/kernel/domainname'
echo KERNEL: 'uname -r'
top -b | head -8
echo "##### FILESYSTEM INFORMATION #####"
echo ""
echo "SUPPORTED FILESYSTEM TYPES:"
echo -----
echo 'cat /proc/filesystems | awk -F\t ' '{print $2}'
echo ""
echo "MOUNTED FILESYSTEMS:"
echo -----
cat /proc/mounts
echo "IDE DEVICES BY CONTROLLER"
echo -----
for i in `ls /proc/ide | grep ide`
do
    echo $i:
    for j in `ls /proc/ide/$i | grep hd`
    do
        echo ""
        echo "    $j"
        echo "    -----"
        echo "    model: 'cat /proc/ide/$i/$j/model'"
        echo "    driver: 'cat /proc/ide/$i/$j/driver'"
        echo "    device type: 'cat /proc/ide/$i/$j/media'"
        if [ -e /proc/ide/$i/$j/geometry ]; then
            echo "        geometry: 'cat /proc/ide/$i/$j/geometry'"
        fi
        echo ""
    done
done
```



TRUCO

75

Mate procesos de manera correcta

El comando `kill` de Linux le permite poner fin a procesos normalmente o de un mazazo

Si pasa mucho tiempo como usuario o administrador de Linux, tarde o temprano tendrá que terminar un proceso (a menudo simplemente un programa que ya no responde a la entrada de usuario o que simplemente parece no irse). La manera más segura de "matar" un proceso es simplemente utilizar el comando `kill`, sin opciones ni modificadores. Primero use el comando `ps -ef` para determinar el identificador de proceso (PID) del proceso que quiere terminar, y luego simplemente escriba este comando:

```
# kill -pid
```

El comando `kill` estándar normalmente funciona bien, poniendo fin al proceso ofensivo y devolviendo sus recursos al sistema. Sin embargo, si su proceso ha iniciado procesos hijo, simplemente "matar" al padre puede dejar potencialmente a los procesos hijo ejecutando, y por tanto todavía consumiendo recursos de sistema.

Para evitar los llamados "procesos zombi", debería asegurarse de que acaba con todos y cada uno de los procesos hijo antes de poner fin a sus procesos padre respectivos.

Matar procesos en el orden correcto

Puede identificar procesos hijo y sus padres usando el comando Linux `ps -ef` y examinando cada entrada, mirando a la columna etiquetada como PPID (*Parent Process ID*, ID de Proceso Padre). Sin embargo, si sólo está interesado en una familia específica de procesos, usar el comando `grep` hace la vida. Veamos un ejemplo. Si estamos intentando matar el proceso `httpd`, necesitaremos matar a sus procesos hijo antes de que podamos hacerlo con el padre. Como atajo, utilice el siguiente comando para determinar los PID a los que necesitaremos poner fin:

```
# ps -ef | grep httpd
[root@aardvark kids]# ps -ef | grep httpd
root      23739      1  0 Jun06 ?        00:00:07 /usr/sbin/httpd
apache    24375    23739  0 Jul17 ?        00:00:01 /usr/sbin/httpd
apache    24376    23739  0 Jul17 ?        00:00:00 /usr/sbin/httpd
apache    24377    23739  0 Jul17 ?        00:00:01 /usr/sbin/httpd
apache    24378    23739  0 Jul17 ?        00:00:00 /usr/sbin/httpd
apache    24379    23739  0 Jul17 ?        00:00:00 /usr/sbin/httpd
apache    24380    23739  0 Jul17 ?        00:00:01 /usr/sbin/httpd
apache    24383    23739  0 Jul17 ?        00:00:00 /usr/sbin/httpd
apache    24384    23739  0 Jul17 ?        00:00:01 /usr/sbin/httpd
```

La primera columna nos dice el usuario que posee cada proceso, la segunda y tercera columna sus PID y PPID respectivamente. El primer proceso listado, con el PPID 1, es el proceso padre. Cuando un proceso tiene un PPID 1, esto significa que se inició por `init` en el momento del arranque.

Lo siguiente que probamos ahora que tenemos el ID del proceso padre es intentar tirarlo abajo con cortesía, usando el siguiente comando:

```
# kill -1 23739
```

La opción `-1` le dice al comando `kill` que intente terminar el proceso como si el usuario que lo inició hubiera cerrado su sesión. Cuando utiliza esta opción, el comando `kill` intenta además recorrer y matar los procesos hijos que deja detrás. Si bien, esto no siempre funcionará, podría necesitar todavía recorrer y matar procesos hijo manualmente primero, antes de matar al proceso padre.

Para matar más de un proceso a la vez, simplemente separe los PID con espacios en la línea de comando de `kill`:

```
# kill 24384 24383 24380
```

Una segunda opción es enviar una señal `TERM` al proceso padre en un intento de acabar con él y con sus procesos hijo. Esto puede hacerse con el siguiente comando:

```
# kill -TERM 23739
```

De manera alternativa, puede intentar matar todos los procesos dentro del mismo grupo de procesos usando `killall`. El comando `killall` le permite especificar los nombres de los procesos a los que quiere poner fin, en vez de sus PID, lo cual puede ahorrarle un montón de comandos `ps` y vista cansada:

```
# killall httpd
```

Parar y reiniciar un proceso

En cierto punto, podría encontrarse con que simplemente quiere detener y reiniciar un proceso. En vez de escribir la secuencia de comandos para matar su proceso manualmente y después reiniciarlo, pruebe a usar el siguiente comando:

```
# kill -HUP 23739
```

Esto hará que Linux lleve a cabo el apagado del proceso suavemente, y después lo reinicie inmediatamente. Esto es especialmente útil cuando está trabajando en configurar una aplicación que necesita reiniciar su proceso tras efectuar cambios en sus ficheros de configuración:

El último recurso

Si los comandos regulares `kill` o `kill -1` no funcionan, puede siempre echar mano del todopoderoso comando `kill -9`:

```
# kill -9 23739
```

Este comando extremadamente poderoso y peligroso fuerza a un proceso a frenar en seco, sin permitirle limpiar lo que deja detrás. Esto puede conducir a recursos de sistemas no utilizados y generalmente no se recomienda, a menos que todas las demás opciones fallen.

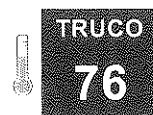
Tras usar el comando `kill -9` (o su sinónimo `kill -s SIGKILL`), asegúrese de utilizar `ps -ef` de nuevo para asegurarse de que no ha dejado ningún proceso

zombi. Sólo puede eliminar un proceso zombi poniendo fin a su proceso padre, lo cual está bien si el proceso padre puede ser finalizado o reiniciado si incidentes, pero problemático si el proceso zombi ha terminado por ser propiedad del proceso `init` (PID 1). No quiere matar al proceso `init`, a menos que sepa lo que conlleva y realmente quiera hacerlo, puesto que matar `init` apagará su sistema. Si tiene procesos zombi cuyo padre es `init`, y están consumiendo cantidades significantes de recursos de sistema, necesitará reiniciar la máquina a cierto punto para poder limpiar la tabla de procesos.

Véase también

- `man kill`
- `man ps`

-Brian Warshawsky



TRUCO

76

Use una consola serie para centralizar el acceso a sus sistemas

Tenga a mano una puerta trasera secreta para emergencias nocturnas.

Imagine el siguiente escenario: son las 3 A.M., y usted es el administrador de guardia. De repente es despertado bruscamente por el buscapersonas vibrando, hasta caerse de la mesilla de noche. Un servidor crítico no responde a sondeos de red, y usted es incapaz de hacerle SSH para determinar cuál es el problema. Se está enfrentando ahora a una dura decisión, nadie quiere vestirse e ir a la oficina a las 3 A.M., pero este servidor es esencial para la presencia online de su compañía. ¿Qué hace? Las buenas noticias son que con una previsión y planificación adecuada, puede evitar del todo este tipo de decisiones con un servidor de consolas.

Un servidor de consolas es un dispositivo al cual puede conectar las consolas de múltiples sistemas. Puede entonces conectarse al servidor de consolas para tener fácil acceso a cualquiera de esos sistemas. Dispositivos que le permiten conectar múltiples puertos serie y conmutar rápidamente entre ellos están fácilmente disponibles desde muchos proveedores diferentes. Una rápida búsqueda de Google por "servidor de consolas serie" listará más proveedores potenciales de los que probablemente quiera saber.

Este truco explica cómo configurar sus sistemas Linux de tal manera que puedan usar puertos serie para salida de consola en vez de las pantallas gráficas tradicionales a las que estamos acostumbrados en Linux. No sólo las consolas serie son baratas comparadas con múltiples pantallas gráficas, sino que son fáciles de acceder remotamente y rápidas, puesto que no hay sobrecarga gráfica.

Las opciones

Antes de que corra a implementar un servidor de consolas, necesita tener en consideración unas cuantas opciones. Hay disponibles varias opciones comerciales que proporcionan muchas variedades diferentes de servidores de consolas. Sin embargo, el método que vamos a discutir aquí es un poco más hágallo-usted-mismo, y pueden en última instancia ser mucho más barato de implementar que una opción comercial.

Otra opción a explorar es si su hardware soporta ya o no el acceso a la consola del puerto serie vía BIOS. Si lo hace, todo podría ser discutible. Sin embargo, este tipo de soporte hardware es bastante raro, así que hay probabilidades de que tenga que decidir entre un caro método propietario, o un fácil de implementar método de código abierto.

Si todavía está leyendo, quiere decir que se ha decidido a ir por la fácil ruta del código abierto. ¡Mejor para usted! Lo primero que debe tener en mente cuando diseñe su servidor de consolas es su despliegue físico. El servidor necesitará mantenerse bastante cerca de sus servidores críticos. Necesitará además tener uno o más puertos serie disponibles. Gran variedad de proveedores proporcionan tarjetas PCI multi-puerto serie, así que encuentre la que le parezca que encaja mejor con su situación y siga intentándolo. Si sólo necesita conectarse a un dispositivo o dos, considere permanecer con los puertos series que normalmente se encuentran en la mayoría de servidores.

Comience por el principio: El gestor de arranque

Comenzaremos ahora el proceso de configurar la consola cliente, la cual, bastante confusamente, es su servidor de producción. Necesitamos configurar el gestor de arranque para que envíe salida y reciba entrada vía el puerto serie. Esto no es tan difícil como podría parecer, así que no tema. Hay varios gestores de arranque disponibles para Linux, pero los más extendidos con diferencia son GRUB y LILO.

En este truco cubriremos la configuración del acceso a consola por medio de GRUB. Si bien LILO es ciertamente un gestor de arranque efectivo y es capaz de llevar a cabo las mismas funciones que GRUB, no contiene tantas características como las que hacen de GRUB una opción atractiva para esta aplicación.

Cuando configuramos el gestor de arranque para redirigir la entrada y salida del sistema, estamos en realidad configurando indirectamente el núcleo de sistema de Linux para que redirija la E/S del sistema. Estas configuraciones se hacen modificando los ficheros de configuración de GRUB, cambiando por tanto la manera en la que GRUB arranca el núcleo de Linux. El fichero de configuración de GRUB se puede encontrar bajo el directorio `/etc` (o a veces `/boot/grub`), y

está acertadamente llamado `grub.conf` (en algunas distribuciones este fichero puede llamarse `menu.lst`).

Antes de sumergirnos en la configuración del gestor de arranque, tomémosnos un momento para examinar un fichero `grub.conf` típico:

```
# grub.conf generated by anaconda
#
# Note that you do not have to rerun grub after making changes to this
file
# NOTICE: You have a /boot partition. This means that
#         all kernel and initrd paths are relative to /boot/, eg.
#         root (hd0,0)
#         kernel /vmlinuz-version ro root=/dev/hda3
#         initrd /initrd-version.img
#boot=/dev/hda
default=0
timeout=5
splashimage=(hd0,0)/grub/splash.xpm.gz
hiddenmenu
title Fedora Core (2.6.11-1.27_FC3)
    root (hd0,0)
    kernel /vmlinuz-2.6.11-1.27_FC3 ro root=LABEL=/
    initrd /initrd-2.6.11-1.27_FC3.img
```

Alguna gente recomienda eliminar las directivas "splashimage", ya que las imágenes gráficas no son adecuadas para consolas serie. Sin embargo, nunca he tenido ningún problema con esto. Que necesite eliminar estas directivas, o no, dependerá en gran parte de la versión de GRUB que esté utilizando. Si es bastante reciente, debería ser capaz de ignorar estas líneas sin ningún problema. De otra manera, simplemente comente o elimine la referencia "splashimage".

Ahora que tiene eso hecho, vamos a modificar el fichero de configuración para que redirija toda la entrada y salida al puerto serie. Los valores estándar para las comunicaciones por puerto serie son 9.600 baudios, sin paridad, y 8 bit. Es importante recordar estos valores, ya que se volverán necesarios más tarde cuando necesite configurar el servidor de consola para comunicarse con el cliente. Para pasar estos valores al núcleo de sistema, añada las siguientes líneas al principio de su fichero `grub.conf`.

```
serial --unit=0 --speed=9600 --word=8 --parity=no --stop=1
terminal --timeout=30 serial console
```

Estas líneas deberían aparecer directamente sobre su directiva "default". La mayoría de las opciones pasadas se explican por sí mismas, pero vamos a mirar las que pueden no estar tan claras. La opción "--unit" le dice al núcleo que redirija todo al primer puerto serie que pueda identificar. Este permanecerá invariable arranque tras arranque, así que no tiene que preocuparse porque pueda cam-

biar. La directiva "--word" se usa para establecer el número de bit que se usarán en las comunicaciones con el servidor de consolas. Esta puede tener el valor 5, 6, 7, u 8. Tome nota de que casi todo se comunica usando 7 u 8 bit, y 8 bit es casi un estándar en toda la industria. Utilizar un número menor para esta opción puede terminar por morderle, ya que los valores ASCII mayores que 127 deberían mostrarse. La opción "--parity" se usa en este caso para desactivar el uso de la comprobación de error de paridad de los datos transmitidos desde un extremo de la conexión *null modem* al otro. La directiva "--stop" se usa para establecer el método por el cual se pone fin a una transmisión de datos *null modem*.

La segunda línea que hemos añadido instruye al núcleo para que use tanto el puerto serie como la consola para mostrar salida. El primero que recibe entrada se convierte en la consola por defecto. Aquí yace una de las características mencionadas anteriormente que hacen de GRUB una excelente elección para este proyecto. Especificando las opciones consola y serie, somos capaces de utilizar efectivamente dos dispositivos como consola.

Una vez que ha efectuado los cambios, reinicie el servidor de tal manera que las nuevas directivas de GRUB puedan tener efecto. Pasaremos ahora a configurar su servidor de consolas para comunicarse con el cliente.

Ponerlo todo junto

Primero, necesita asegurarse de que tiene un cable serie conectando su servidor de consolas al cliente. (Asegúrese de conectar el cable serie al mismo puerto para el que configuró la redirección de E/S del núcleo, lo terminará mirando a todo un montón de nada!) Ahora necesitará un programa para comunicarse vía *null modem*. Hay unos cuantos disponibles, pero para facilidad de uso general y máximo de características, yo recomiendo *minicom*. Para iniciar *minicom*, ejecute el siguiente comando como súper-usuario:

```
# minicom -s
```

Esto le llevará directamente a la pantalla de configuración de *minicom*. Seleccione Serial Port Setup y cambie la configuración para que coincida con lo que ha configurado anteriormente en GRUB. Una vez hecho esto, guarde sus cambios y salga. Debería ser llevado a la pantalla principal de *minicom*. Si no ve nada, pulse **Intro** una vez, y debería ser recibido con el marcador de inicio de sesión de su servidor. ¡Enhorabuena, mientras su red permanezca viva, tiene ahora un acceso remoto de consola a su servidor!

Una vez que todo está montado y *minicom* está instalado, todo lo que tiene que hacer es SSH a su servidor de consolas, iniciar *minicom* con el correcto dispositivo serie, y acceder a la consola de su sistema con problemas. ¡Voilà! ¿Qué puede ser más fácil?

Adónde ir desde aquí

Lo que hemos creado aquí es la aplicación más básica de un servidor de consolas. Como mencioné anteriormente, esto puede expandirse con la adición de una tarjeta multi-puerto serie. Con un poco de tiempo extra, poniendo cables y configurando sus gestores de arranque, puede efectivamente desplegar servidores de consolas por toda su red. Otro truco a tener en mente es el aparentemente simple adaptador RJ45 a DB9.

Estos pequeños personajes le permiten utilizar un hilo de cable de red categoría 5 para conectarse a un puerto serie. De hecho yo los he utilizado en conjunto con paneles de conexión para proporcionarme acceso de consola a los equipos de red desde mi escritorio.

Puede coger uno de esos chismes salvavidas de cualquier compañía de material de red por menos de unos pocos euros.

Otra manera de incrementar la utilidad de su servidor de consolas es incluir un módem conectado a él. Este puede configurarse para aceptar llamadas entrantes, permitiéndole por tanto conectarse a su servidor de consolas por medio de la línea telefónica en el caso de una caída de la red. Le recomendaría encarecidamente esto último, ya que no sacaré ningún partido del servidor remoto si su problema reside en alguna parte de la red.

Sólo necesitará usar un servidor de consolas serie una vez, para probarse a sí mismo que ha invertido bien la media hora más o menos que lleva configurarlo. A las 3 A.M., cuando sea capaz de reiniciar un servidor remotamente y ponerle de nuevo en servicio, como se pone un par de pantalones, estará de acuerdo conmigo en que la previsión para tal ocasión no tiene precio.

Véase también

- <http://www.linuxjournal.com/article/7206>

-Brian Warshawsky



TRUCO

77

Limpie NIS tras la marcha de usuarios

¡No deje que sus mapas NIS se pongan rancios! El mapa de contraseñas de NIS obviamente necesita mantenimiento, pero no olvide eliminar a los usuarios que se hayan marchado de los grupos a los que pertenecían.

Muchos sitios usan NIS, en parte porque ha estado ahí por muchos años y es una manera extremadamente fiable, aceptablemente rápida, y de relativa baja sobrecarga de ejecutar un directorio de autenticación centralizado. A lo largo de los años, se han escrito toneladas de sistemas software para sacar provecho de la

información proporcionada por los servidores NIS para los fines de ofrecer información o seguridad a los sistemas cliente.

Aunque hay herramientas disponibles para cuidar de la mayoría de las tareas de gestión de usuarios cuando los usuarios residen en un equipo local, muchas de estas herramientas no tienen soporte completo para NIS, y las versiones NIS de esas herramientas todavía tienen que aparecer. Como resultado, ciertas porciones de su directorio NIS pueden pasarse de fecha.

El mapa de grupos de NIS es un perfecto ejemplo de este acontecimiento. El comando `userdel` estándar no soporta NIS, y el comando `groupmod` no soporta eliminar un usuario de un grupo, y mucho menos de un grupo NIS.

La mayoría de los comandos específicos de NIS son bien para hacer búsquedas en los mapas (p.e., `ypmatch` e `ypcat`), para obtener información sobre su sistema cliente (p.e., `ypwhich` e `ypdomainname`), u obtener información sobre el servidor NIS (p.e., `ypoll`). No hay ninguna herramienta disponible para modificar los mapas NIS sin tener que abrir un editor y eliminar las entradas a mano.

Por tanto, si no ha estado atento a mantener los mapas para asegurarse de que son siempre consistentes con la realidad, puede acumular montones de cuentas rancias. Muchos sitios están muy atentos a eliminar usuarios del mapa de contraseñas, pero incluso esto es a menudo un proceso manual que implica abrir el mapa en un editor y eliminar la línea correspondiente al usuario que se ha ido. Lo que he encontrado, sin embargo, es que el mapa de grupos es olvidado a menudo, así que podría acabar con 40 ó 50 usuarios asignados a grupos, pero cuyas cuentas ya no existen. Esto hace los datos de ese mapa menos utilizables, y dependiendo de como se usen estos datos, podría causar problemas con el tiempo.

Tome, por ejemplo, un servidor de correo que utiliza el mapa de grupos para crear alias de correo correspondientes a los nombres de grupo. Un mapa de grupos rancio pondrá un puñado de usuarios no existente en sus alias de correo, lo cual hará que la bitácora de su correo crezca fuera de control, registrando errores sobre usuarios no existentes, por no mencionar que escribir a un alias desactualizado, hará que los usuarios finales reciban errores de rebote del servidor de correo.

He escrito un *script* en Perl para cuidarme de limpiar cuentas de usuario que ya no existen. Escudriña el mapa de grupos, y, para cada usuario, comprueba la existencia de esa cuenta de usuario en el mapa de contraseñas. Cualquier usuario que no esté listado en el mapa de contraseñas será pulcramente eliminado del mapa de grupos. Yo llamo al *script* `cleangroup`.

El código

```
#!/usr/bin/perl
```

```
## looks up all members of each group via 'ypmatch $user passwd' and
```



```
## deletes any users from a given group file which aren't found.
## Output goes to STDOUT!

if(${#ARGV} < 0) {
    die "Must specify group file.\n" ;
}
$grpfile = $ARGV[0] ;
open(GRPFILE, "<$grpfile") || die "can't read $grpfile: $!\n" ;

while(<GRPFILE>) {
    chomp ;
    ($group,$pwd,$id,$members) = split(/:/) ;
    @unames = split(/,/, $members);
    foreach $i (@unames){
        if($i ne "root"){
            if(! 'ypmatch $i passwd 2>/dev/null'){
                $members =~ s/\b$i\b//g ;
            }
        }
    }
    $members =~ s/,/,/ ;
    $members =~ s/,,$// ;
    $members =~ s/^,// ;
    print "$group:$pwd:$id:$members\n" ;
}
close(GRPFILE) ;
```

Ejecutar el código

Yo ejecuto `cleangroup` en el directorio que contiene los mapas NIS. Por razones de seguridad, tengo la salida del *script* dirigida a `stdout` en vez de cambiar el mapa *in situ*. Redirijo la salida a un fichero, ejecuto un rápido `diff` para ver qué ha cambiado, y copio el nuevo mapa sobre el antiguo. He aquí los comandos que uso:

```
# ./cleangroup groupmap > newgroupmap
# diff groupmap newgroupmap
```

Esto debería dar líneas de salida similares a las siguientes:

```
104c104
< stuff:*:20205:ken,maria,mike,tier,matt,jonesy,russ,allen
---
> stuff:*:20205:ken,maria,mike,tier,matt,russ,allen
252c252
< things:*:140:dan,chase,chandler,christian,chance,steph,jonesy
---
> things:*:140: dan,chase,chandler,christian,chance,steph
```

Notará que en cada caso la cuenta `jonesy` se ha eliminado, una vez del medio de la lista y otra del final. Todavía no he tenido problemas con este *script*, íasí que espero que lo encuentre tan útil como yo!



Ficheros de bitácora y monitorización

Trucos 78 a 88

Lo único peor que fallos de disco desastrosos, equipos remotos desaparecidos, e incidentes de seguridad insidiosos es el desgarrador sentimiento que acompaña a descubrir que todo esto se podía haber evitado.

Para prevenir la catástrofe, a menudo la mejor herramienta que puede tener es el acceso a datos que le permitan tomar medidas pro-activas. Tanto si es para tener un disco que le diga cuándo está a punto de fenecer como para ser informado de cortes de red o de servicio, las herramientas que reúnen datos y le alertan de anomalías tienen un valor incalculable para los administradores de redes y de sistemas. El objetivo de este capítulo es mostrarle cómo obtener datos de los que actualmente no dispone, y cómo utilizar los que ya tiene de maneras más útiles.



TRUCO

78

Evite fallos catastróficos de disco

Acceda a los diagnósticos integrados en su disco duro usando utilidades Linux para predecir e impedir el desastre.

Nadie quiere entrar tras un fallo de energía sólo para darse cuenta de que, además, debido a un disco duro muerto, ahora tiene que reconstruir servidores completos y echar mano de las copias de seguridad en cinta. Por supuesto, la mejor manera de evitar esta situación es ser alertado de cuándo algo no va bien con su disco duro ATA o SCSI, antes de que éste finalmente falle. Lo ideal sería que la alerta viniera directamente del propio disco duro, pero hasta que no seamos capaces de conectar un RJ-45 directamente a una unidad tendremos que optar por la siguiente mejor opción, que son los diagnósticos integrados en el

disco duro. Desde hace muchos años, las unidades ATA y SCSI han soportado un mecanismo estándar de diagnósticos de disco llamado SMART (*Self Monitoring, Analysis, and Reporting Technology*, Tecnología de Auto-Monitorización, Análisis e Información), orientado a predecir los fallos de disco duro. No pasó demasiado tiempo hasta que Linux tuvo utilidades para sondear la información vital de los discos duros.

El proyecto smartmontools (<http://smartmontools.sourceforge.net>) produce un demonio de monitorización de SMART llamado `smartd` y una utilidad de línea de comandos llamada `smartctl`, la cual puede hacer, bajo demanda, la mayoría de cosas que el demonio hace periódicamente en segundo plano. Con estas herramientas, junto con las utilidades estándar de sistemas de ficheros de Linux tales como `debugfs` y `tune2fs`, no hay muchos problemas de disco duro que no pueda solucionar.

Pero antes de que pueda reparar nada o transformarse en un aparentemente súper-poderoso héroe del disco duro, con poderes prestados del reino de lo sobrenatural, tiene que saber qué sucede con sus unidades, y necesita ser alertado de los cambios en su estado de salud.

Primero, probablemente debería conocer sus unidades un poco, en lo que `smartctl` puede ayudarle. Si sabe que hay tres unidades en uso en el sistema, pero no está seguro de cuál está etiquetando el sistema como `/dev/hda`, ejecute el siguiente comando:

```
# smartctl -i /dev/hda
```

Esto le dirá el modelo y la información de capacidad para esa unidad. Esto es además muy útil para descubrir a qué proveedor tendrá que llamar para reemplazar una unidad si compró ésta por separado. Una vez que sepa qué es qué, puede pasar a tareas mayores.

Normalmente, antes de ni siquiera configurar el demonio `smartd` para hacer monitorización continua a largo plazo de un disco duro, ejecuto primero una comprobación desde la línea de comandos (usando el comando `smartctl`) para asegurarme de que no estoy perdiendo el tiempo configurando la monitorización en un disco que ya tiene problemas.

Pruebe a ejecutar un comando como el siguiente para preguntarle a la unidad sobre su estado de salud en general:

```
# smartctl -H /dev/hda
smartctl version 5.33 [i386-redhat-linux-gnu] Copyright (C) 2002-4 Bruce
Allen
Home page is http://smartmontools.sourceforge.net/
```

```
=== START OF READ SMART DATA SECTION ===
SMART overall-health self-assessment test result: PASSED
```

Bien, esto son buenas nuevas, el disco dice que está en buena forma. Sin embargo, realmente esto no fue mirar demasiado. Vamos a obtener una visión más detallada de las cosas usando la opción "-a" (*all*, todo). Esto nos da un montón de información de salida, así que vamos por partes. He aquí el primer pedazo:

```
# smartctl -a /dev/hda
smartctl version 5.33 [i386-redhat-linux-gnu] Copyright (C) 2002-4 Bruce
Allen
Home page is http://smartmontools.sourceforge.net/

=== START OF INFORMATION SECTION ===
Device Model:          WDC WD307AA
Serial Number:         WD-WMA111283666
Firmware Version:      05.05B05
User Capacity:         30,758,289,408 bytes
Device is:              In smartctl database [for details use: -P show]
ATA Version is:        4
ATA Standard is:        Exact ATA specification draft version not indicated
Local Time is:          Mon Sep  5 17:48:09 2005 EDT
SMART support is:       Available - device has SMART capability.
SMART support is:       Enabled
```

Ésta es exactamente la misma salida que `smartctl -i` le había mostrado antes. Le dice el modelo, la versión de *firmware*, la capacidad, y qué versión del estándar ATA está implementada con esta unidad. Útil pero no realmente una medida de salud per se. Vamos a seguir mirando:

```
=== START OF READ SMART DATA SECTION ===
SMART overall-health self-assessment test result: PASSED
```

Ésta es la misma salida que `smartctl -H` mostró anteriormente. Contento de haber aprobado, pero si lo hicimos a duras penas, esto no es un aprobado para un administrador exigente. ¡Más!

```
General SMART Values:
Offline data collection status (0x05) Offline data collection activity
was aborted by an interrupting
command
from host.
Auto Offline Data Collection:
Disabled.
Self-test execution status:      ( 113) The previous self-test completed
having
the read element of the test failed.
```

Estos son los valores de los atributos SMART que soporta el dispositivo. Podemos ver aquí que la colecta de datos fuera de línea (*offline*) está desactivada. Esto significa que no podemos ejecutar pruebas "fuera de línea" (las cuales se ejecutan automáticamente cuando el disco esta de alguna manera ocioso). Podemos acti-

varla usando el comando `smartctl -o on`, pero puede no ser lo que usted quiera, así que vamos a posponerlo por ahora.

El estado de ejecución de auto-pruebas muestra que una operación de lectura falló durante la última auto-prueba, así que lo tendremos en mente según continuamos mirando a los datos:

```
Total time to complete Offline
data collection:          (2352)  seconds.
Offline data collection
capabilities:              (0x1b)  SMART execute Offline immediate.
                              Auto Offline data collection on/off
                              support.
                              Suspend Offline collection upon new
                              command.
                              Offline surface scan supported.
                              Self-test supported.
                              No Conveyance Self-test supported.
                              No Selective Self-test supported.
SMART capabilities:      (0x0003) Saves SMART data before entering
                              power-saving mode.
                              Supports SMART auto save timer.
Error logging capability: (0x01)  Error logging supported.
                              No General Purpose Logging support.
```

Esta salida es simplemente una lista de las habilidades de la unidad relacionadas con SMART, las cuales está bien conocer, especialmente para unidades más antiguas que podrían no tener todas las características que asumiría que están presentes de otra manera.

El soporte de habilidades y características en las unidades sigue aproximadamente la versión del estándar ATA en uso cuando se construyó la unidad, así que no es seguro asumir que una unidad ATA-4 soportará el mismo juego de características que una unidad ATA-5 o posterior.

Vamos a continuar con nuestro tour por la salida:

```
Short self-test routine
recommended polling time:  ( 2) minutes.
Extended self-test routine
recommended polling time:  (42) minutes.
```

Cuando le diga a esta unidad que haga una breve auto-prueba, le dirá que espere dos minutos para los resultados. Una prueba larga tardará 42 minutos. Si esta unidad fuera lo suficientemente nueva como para soportar otros tipos de auto-prueba (aparte de simplemente "breve" y "extendido"), habría también líneas para ellos. He aquí la siguiente sección de salida:

```
SMART Attributes Data Structure revision number: 16
Vendor Specific SMART Attributes with Thresholds:
```

ID#	ATTRIBUTE_NAME	FLAG	VALUE	WORST	THRESH	TYPE	UPDATED
1	Raw_Read_Error_Rate	0x000b	200	200	051	Pre-fail	Always
3	Spin_Up_Time	0x0006	101	091	000	Old_age	Always
4	Start_Stop_Count	0x0012	100	100	040	Old_age	Always
5	Reallocated_Sector_Ct	0x0012	198	198	112	Old_age	Always
9	Power_On_Hour	0x0012	082	082	000	Old_age	Always
10	Spin_Retry_Count	0x0013	100	100	051	Pre-fail	Always
11	Calibration_Retry_Count	0x0013	100	100	051	Pre-fail	Always
12	Power_Cycle_Count	0x0012	100	100	000	Old_age	Always
196	Reallocated_Event_Count	0x0012	196	196	000	Old_age	Always
197	Current_Pending_Sector	0x0012	199	199	000	Old_age	Always
198	Offline_Uncorrectable	0x0012	199	198	000	Old_age	Always
199	UDMA_CRC_Error_Count	0x000a	200	253	000	Old_age	Always
200	Multi_Zone_Error_Rate	0x0009	200	198	051	Pre-fail	Offline

Los detalles sobre cómo leer esta tabla, a un nivel lo bastante escabroso, están en la página de manual de `sysctl`. Los valores más inmediatos de los que tiene que preocuparse son los etiquetados como "Pre-fail". En estas líneas, un indicador de la necesidad de acción inmediata es si la salida de columna "VALUE" desciende hasta o por debajo del valor en la columna "THRESH". Continuamos:

```
SMART Error Log Version: 1
No Errors Logged
```

```
SMART Self-test log structure revision number 1
Num Test_Description Status Remaining LifeTime(hours)
LBA_of_first_error
# 1 Extended offline Completed: read failure 10% 97
57559262
# 2 Extended offline Aborted by host 50% 97 -
# 3 Short offline Completed without error 00% 97 -
```

```
Device does not support Selective Self Tests/Logging
```

Esta salida es la salida de bitácora de las tres últimas pruebas. La numeración de las pruebas es en realidad a la inversa de como cabría pensar: la del principio

de la lista, etiquetada como "# 1", es realmente la más reciente. En esta prueba podemos ver que hubo un error de lectura, y la dirección LBA del primer fallo es expuesta (57559262). Si quiere ver cómo puede asociar esa prueba con un fichero real, Bruce Allen ha publicado un estupendo HOWTO sobre esto mismo en <http://smartmontools.sourceforge.net/BadBlockHowTo.txt>.

Ahora que ha visto lo que smartctl puede descubrir por nosotros, vamos a comprender cómo configurar smartd para automatizar el proceso de monitorización y hacernos saber si el peligro es inminente.

Afortunadamente, reunir una configuración básica lleva meros segundos, y montar configuraciones más complejas tampoco lleva demasiado tiempo. El proceso smartd obtiene su configuración de /etc/smartd.conf en la mayoría de los sistemas, y para un sistema pequeño (o para una tonelada de sistemas pequeños que no quiere que generen cantidades copiosas de correo) una línea similar a la siguiente le dará lo mínimo esencial:

```
/dev/hda -H -m jonesy@linuxlaboratory.org
```

Esto hará una (muy) simple comprobación del estado de salud de la unidad, y me mandará un correo sólo si falla. Si una comprobación de estado de salud falla, quiere decir que la unidad podría fácilmente fallar en las siguientes 24 horas, ¡así que tenga una unidad extra a mano!

Hay también configuraciones más sofisticadas que pueden alertarle de cambios en el estado que no necesariamente significan una muerte cierta. Echemos un vistazo a una línea de configuración más compleja:

```
/dev/hda -l selftest -l error -I 9 -m jonesy@linuxlaboratory.org -s L/.../7/02
```

Ésta buscará cambios en las bitácoras de auto-prueba y de error del dispositivo, ejecutará una auto-prueba larga cada domingo entre las 2 y las 3 A.M., y me enviará mensajes sobre cualquier atributo excepto sobre el ID 9, el atributo "Power_On_Hours", el cual no tiene importancia para mí para el caso de determinar si un disco está mal (puede comprobar la salida de sysctl -a para determinar el ID de un atributo). El atributo "-I" se usa a menudo con los números 194 ó 231, y normalmente es la temperatura. ¡Sería malo obtener mensajes sobre cambios constantes en la temperatura de la unidad!

Una vez que tiene su fichero de configuración en orden, lo único que queda es iniciar el servicio. Inevitablemente, recibirá más correo del que le gustaría en las primeras ejecuciones iniciales, pero según pasa el tiempo (y lea más de la enorme página de manual) aprenderá a obtener lo que quiere de smartd. Para mí, simplemente la tranquilidad merece las horas que he invertido en obtener una configuración que funcione. Cuando sea capaz de evitar una catástrofe cierta para un cliente o para usted mismo, estoy seguro de que dirá lo mismo.



Monitorice tráfico de red con MRTG

El creador de gráficos de tráfico multi-router (Multi-Router Traffic Grapher) proporciona una rápida instantánea visual del tráfico de red, facilitando el encontrar y resolver congestiones.

Hay muchas razones por las que es una buena idea capturar datos pertenecientes al uso y ancho de banda de su red. Representaciones visuales detalladas de tales datos pueden ser increíblemente útiles en determinar las causas de cortes de red, cuellos de botella, y otros problemas. Recolectar tales datos detallados solía requerir equipos sofisticados y costosos, pero con el advenimiento de Linux y el extendido uso de SNMP, ahora tenemos una nueva herramienta para simplificar y expandir las posibilidades de la monitorización de ancho de banda. Esta herramienta se llama MRTG (Multi-Router Traffic Grapher, creador de Gráficos de Tráfico Multi-Router), y este truco le muestra cómo configurarlo y usarlo.

Requisitos

MRTG tiene unas cuantas dependencias simples que puede necesitar cubrir antes de sumergirse en la instalación. Para empezar, necesita tener un servidor Web funcionando. Normalmente se recomienda Apache, pero podría ser capaz de hacerlo funcionar con otros servidores Web. Necesitará además tener Perl instalado y funcionando en su sistema, y MRTG requerirá tres librerías para construir sus gráficos, la primera, gd, se usa para generar los gráficos que hacen de MRTG lo que es.

La segunda es libpng, que se usa para generar las imágenes de los gráficos. Finalmente, para comprimir estas imágenes, necesitará la librería zlib. Los sitios de descarga para estas tres librerías se pueden encontrar en la página Web de MRTG (<http://people.ee.ethz.ch/~oetiker/Webtools/mrtg/>).

Instalación

Una vez que tenga las dependencias instaladas, puede comenzar la instalación de MRTG. Primero, descargue y desempaquete los fuentes en su lugar de compilación. Inicie la instalación de MRTG con el siguiente comando:

```
$ ./configure --prefix=/usr/local/mrtg-2
```

Si esto produce un mensaje de error, puede tener que especificar dónde ha instalado las librerías previamente mencionadas:

```
# ./configure --prefix=/usr/local/mrtg-2 --with-gd=/path/to/gd \
--with-z=/path/to/z --with-png=/path/to/png
```

Si necesita ayuda para determinar dónde están instaladas esas librerías, ejecute el siguiente comando para cada librería con el fin de encontrar su ubicación:

```
# find / -type f -name libpng
```

Una vez que ha completado la configuración, continúe con un típico make install:

```
# make && make install
```

El siguiente paso es crear el fichero `mrtg.cfg` que MRTG usará para determinar a qué dispositivos de su red consultar. Si tiene que crear esto a mano, las cosas podrían volverse un poco espeluznantes. Sin embargo, afortunadamente para nosotros, MRTG viene con una herramienta de configuración por línea de comandos llamada `cfgmaker` que simplifica mucho la creación del `.cfg`. Documentación detallada sobre `cfgmaker` está disponible en la página Web de MRTG, pero el siguiente ejemplo debería ser suficiente para comenzar:

```
# cfgmaker -global 'WorkDir: /path/to/Web/root/mrtg' \
--output=/etc/mrtg.cfg \
--global 'Options[_]: bits, growright' --output=/etc/mrtg.cfg \
SNMP-community-name@address.router1 \
SNMP-community-name@address.router2 \
Global 'Options[_]: bits, growright' --ifref=descry \
--ifdescr=alias SNMP-community-name@address.switch.1
```

Esto creará el fichero de configuración `/etc/mrtg.cfg`, el cual le dirá a MRTG que cree gráficos de ancho de banda para `router1`, `router2`, y `switch1`. Los gráficos usaran bit como medida primaria en el eje Y creciendo hacia la derecha. Las opciones "-global" añaden entradas que se aplican a esta configuración como un todo, mientras que aquéllas no especificadas como globales se aplican sólo a los dispositivos en los que las especificamos. La ubicación en la que se crea el fichero de configuración se indica con la opción "-output".

Con un fichero de configuración válido en la mano, podemos ahora ejecutar MRTG por primera vez. Cada vez que ejecute MRTG, necesitará especificar la ubicación desde la cual quiere leer el fichero de configuración. Además, a menos que la haya añadido en su ruta por defecto, necesitará escribir la ruta completa al ejecutable.

```
# /usr/local/bin/mrtg-2/bin/mrtg /etc/mrtg.cfg
```

Verá algunos errores las primeras dos veces que ejecute MRTG, pero no les haga caso, simplemente se queja porque no puede encontrar ningunos datos anteriores de MRTG. Tras ejecutar el comando, su raíz Web MRTG debería llenarse de ficheros PNG. Esto es estupendo, excepto porque es bastante molesto mirarlos así, y porque no están exactamente etiquetados en un formato amigable para los

humanos. La solución a este problema puede encontrarse en la herramienta `indexmaker`. `indexmaker` funciona exactamente como la herramienta `cfgmaker`, sólo que en vez de generar ficheros de configuración, genera un borrador HTML con el cual podemos mostrar nuestros gráficos MRTG:

```
# indexmaker -output=/path/to/Web/root/index.html \
--title="My Network MRTG" --sort=title
```

Esto creará un fichero `index.html` que ordena y muestra sus datos en un formato mucho más amigable para el usuario, como se ve en la figura 9.1. Puede modificar entonces el fichero `index` simplemente como haría con cualquier fichero HTML para hacer que muestre cualquier otra información que desee.

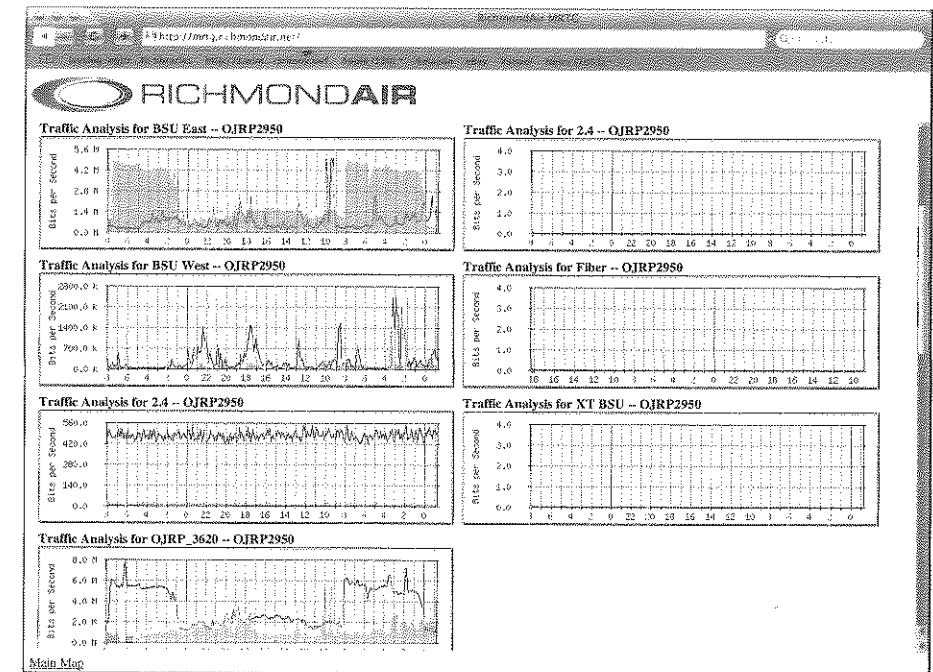


Figura 9.1. Gráficos de tráfico de red creados con datos MRTG.

Automatizar MRTG

Lo único que queda es automatizar el proceso. MRTG no sería muy útil si tuviera que arrancarlo manualmente cada vez, así que tendremos que automatizarlo añadiéndolo a `cron`.

Agregue la siguiente entrada al crontab del súper-usuario para ejecutar MRTG cada cinco minutos:

```
*/5 * * * * /usr/local/mrtg-2/bin/mrtg /etc/mrtg.cfg \
--logging /var/log/mrtg.log
```

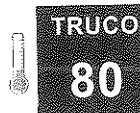
No se impaciente esperando a ver sus bonitos nuevos gráficos. Llevará un día más o menos para que se empiecen a mostrar datos realmente útiles. Si bien, una vez que los ha tenido ejecutando un tiempo, será capaz de obtener todo tipo de pautas útiles sobre la utilización de su ancho de banda. Por ejemplo, podría darse cuenta de que su ancho de banda tiene un pico entre las 8:30 y las 9:00 A.M., y de nuevo después de comer. Esto le ayudará a entender mejor la utilización de su red, y a su vez darle mejor servicio. Puede ser fascinante simplemente mirar cómo se materializa la utilización de su ancho de banda, y utilizar la información para seguir la pista y las pautas de la actividad de red. MRTG creará gráficos anuales así como horarios, mensuales, y diarios. Tener tal información detallada en sus manos puede ayudarle a comprender simplemente cuánto tráfico ha ganado después de que su sitio fue incluido en Slashdot, y cómo se incrementa su popularidad según corre el tiempo.

MRTG tiene un millón de usos, y no se limitan simplemente a seguir la pista a la utilización del ancho de banda. Con un poco de modificación, puede usarlo para medir casi cualquier cosa que quiera. Para más información sobre cómo modificar MRTG para que muestre otras estadísticas, visite la página Web de MRTG.

Véase también

- <http://people.ee.ethz.ch/~oetiker/Webtools/mrtg/>

-Brian Warshawsky



Truco 80 Mantenga una vigilancia constante en los equipos

Monitoree carga u otras estadísticas para múltiples equipos en su escritorio o en la línea de comandos.

rstatd es un servidor de estadísticas del núcleo de sistema basado en RPC que está bien incluido o bien disponible para todos los tipos de Unix que he usado. No es nada nuevo. De hecho, sospecho que su edad podría hacerle pasar desapercibido para el radar de los administradores más jóvenes, que podrían no conocerlo si no ha aparecido en la portada de Freshmeat recientemente. Con un poco de suerte, esta información despertará su interés en esta herramienta tan útil.

Cuando digo que rstatd proporciona "estadísticas del núcleo", me estoy refiriendo a cosas como la carga de CPU, estadísticas de intercambio de páginas, estadísticas de E/S, y similares. Por supuesto, proporcionar esta información a los administradores de una manera que sea útil puede a veces ser desafiante, pero hay unas cuantas herramientas disponibles para ayudar.

Para hacer estas herramientas útiles, debe tener un demonio rstatd ejecutándose. Fíjese que rstatd es dependiente del demonio portmap, el cual ya debería estar ejecutando si está utilizando otros servicios basados en RPC como NIS o NFS. Para hacer una rápida comprobación y asegurarse de que se están ejecutando, puede utilizar el siguiente comando:

```
$ rpcinfo -p
program vers proto port
100000 2 tcp 111 portmapper
100000 2 udp 111 portmapper
100001 3 udp 646 rstatd
100001 2 udp 64 rstatd
100001 1 udp 646 rstatd
```

Sin otros argumentos, esto le mostrará el estado del equipo local. Si pone un nombre de equipo al final del comando anterior, le mostrará el estado de un equipo remoto. ¡Ahora estamos listos para apuntar algunas herramientas a este equipo!

La primera y más destacada de estas herramientas es el comando estándar `rup`, el cual está disponible en Linux y otras plataformas Unix. Es una simple utilidad cliente de rstatd, pero con las herramientas adecuadas puede usarla para producir una salida similar a la producida por el comando `top`, sólo que en vez de monitorizar procesos en el equipo local, puede monitorizar la carga en múltiples máquinas. He aquí un comando que puede ejecutar para tener una lista de equipos, ordenada por promedio de carga y actualizada cada cinco segundos:

```
$ watch -n 5 rup -l host1 host2 host3 host4 host5
host3 up 12 days, 7:33, load average: 0.00, 0.00, 0.00
host4 up 12 days, 7:28, load average: 0.00, 0.00, 0.00
host1 up 12 days, 6:11, load average: 0.05, 0.04, 0.05
host2 up 12 days, 6:11, load average: 0.05, 0.04, 0.05
host5 up 12 days, 7:29, load average: 0.09, 0.06, 0.01
```

Esto está bien si no tiene acceso a ningún tipo de entorno gráfico. Por supuesto, se apodera de su terminal, así que como mínimo necesitará ejecutarlo dentro de una sesión de `screen` o en un terminal virtual separado. Otro problema aquí es que es tan sólo una salida de datos simples y crudos; no le alerta de ningún suceso, como la carga de host4 yéndose a la estratosfera.

Para esto, podemos movernos a clientes gráficos. Un viejo favorito mío es `xmeter`, que fue desarrollado hace tiempo y desde entonces parece haber sido abandonado

y olvidado. Lleva algo de tiempo escudriñar su configuración (no es gráfica), pero viene con una página de manual para ayudar, y una vez que está configurado lo único que tendrá que cambiar es la lista de equipos a monitorizar. Proporciona opciones de configuración para cambiar el color de la salida basándose en valores de umbral, de tal manera que si la carga de una máquina se va un poco fuera de control, el cambio de color probablemente llamará su atención. La figura 9.2 muestra una toma de *xmeter* monitorizando la carga de múltiples sistemas.

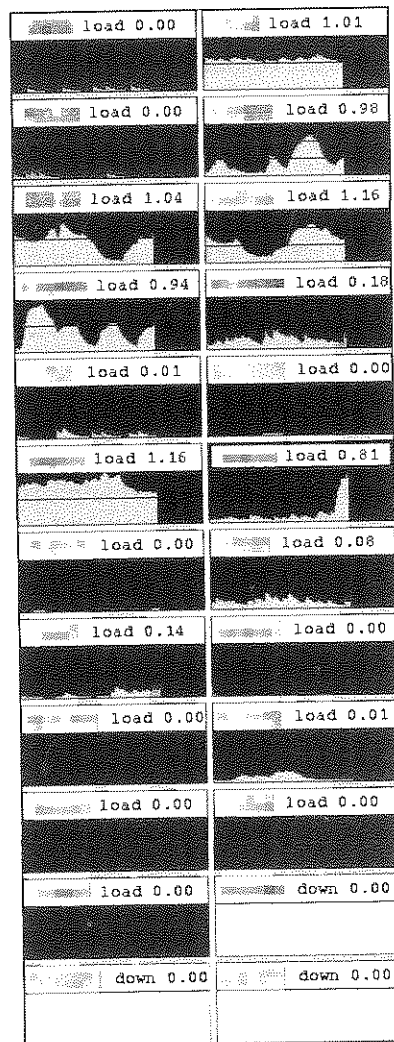


Figura 9.2. Pantalla de *xmeter* monitorizando la carga en múltiples servidores.

Un desarrollo más reciente en el mundo de las herramientas de recolección de datos *rstatd* es *jperfmeter*, que es un monitor basado en Java, multiplataforma, con una interfaz más pulida y una herramienta de configuración gráfica. Todavía no (en el momento de escribir esto) soporta umbrales, y se echan de menos unos cuantos detalles más, pero es una herramienta nueva, así que estoy seguro de que llegará a esto en algún momento.

Hay otras herramientas disponibles para monitorización de estadísticas de servidores remotos, pero puede que quiera investigar también el construir la suya propia, utilizando tanto el módulo *Perl Rstat::Client* o las interfaces *RPC* o *rstat* para otros lenguajes, como Python, Java, o C/C++.



Monitorice remotamente y configure diferentes equipos en red

Usando *SNMP*, puede recolectar información sobre casi cualquier dispositivo conectado a su red.

Para todo lo que tenga una interfaz de red, hay posibilidades de que haya algún tipo de demonio *SNMP* (*Simple Network Management Protocol*, Protocolo Simple de Gestión de Red) que pueda ejecutar sobre él. A lo largo de los años, se han añadido demonios *SNMP* a todo, desde sensores de ambiente para UPS a máquinas de refrescos. El punto de todo esto es ser capaz de acceder remotamente a tanta información sobre el equipo como sea humanamente posible. Y como ventaja adicional, una configuración adecuada puede también permitir a los administradores cambiar valores en el equipo de forma remota.

Los paquetes de demonio *SNMP* están disponibles para todas las distribuciones más utilizadas, junto con paquetes posiblemente separados que contienen un conjunto de herramientas *SNMP* de línea de comandos. Puede que ya se haya encontrado alguna vez en sus viajes con los comandos *snmpwalk* o *snmpget*, o puede haber visto funciones con nombres parecidos en lenguajes de programación de *script* como Perl y PHP.

Vamos a echar un vistazo a una pequeña parte de un paseo por un equipo con *SNMP* activado y a usarla para explicar cómo funciona esto:

```
$ snmpwalk -v2c -c public livid interfaces
IF-MIB::ifNumber.0 = INTEGER: 4
IF-MIB::ifIndex.1 = INTEGER: 1
IF-MIB::ifIndex.2 = INTEGER: 2
IF-MIB::ifIndex.3 = INTEGER: 3
IF-MIB::ifIndex.4 = INTEGER: 4
IF-MIB::ifDescr.1 = STRING: lo
IF-MIB::ifDescr.2 = STRING: eth0
IF-MIB::ifDescr.3 = STRING: eth1
```

```

IF-MIB::ifDescr.4 = STRING: sit0
IF-MIB::ifType.1 = INTEGER: softwareLoopback(24)
IF-MIB::ifType.2 = INTEGER: ethernetCsmacd(6)
IF-MIB::ifType.3 = INTEGER: ethernetCsmacd(6)
IF-MIB::ifType.4 = INTEGER: tunnel(131)
IF-MIB::ifPhysAddress.1 = STRING:
IF-MIB::ifPhysAddress.2 = STRING: 0:a0:cc:e7:24:a0
IF-MIB::ifPhysAddress.3 = STRING: 0:c:f1:d6:3f:32
IF-MIB::ifPhysAddress.4 = STRING: 0:0:0:0:3f:32
IF-MIB::ifAdminStatus.1 = INTEGER: up(1)
IF-MIB::ifAdminStatus.2 = INTEGER: up(1)
IF-MIB::ifAdminStatus.3 = INTEGER: down(2)
IF-MIB::ifAdminStatus.4 = INTEGER: down(2)
IF-MIB::ifOperStatus.1 = INTEGER: up(1)
IF-MIB::ifOperStatus.2 = INTEGER: up(1)
IF-MIB::ifOperStatus.3 = INTEGER: down(2)
IF-MIB::ifOperStatus.4 = INTEGER: down(2)

```

Como puede ver, hay un buen trozo de información aquí, y he cortado las partes que no son importantes en este momento. Además, ésta es sólo una parte de un árbol SNMP (el árbol de interfaces).

Bajo ese árbol yacen valores de configuración e información de estado para cada interfaz en el sistema.

Si examina la lista concienzudamente, verá valores separados para cada interfaz, correspondientes a cosas como la descripción del interfaz (el nombre que le da el equipo), la dirección física, y el tipo de interfaz.

¿Pero qué es este "árbol" del que estoy hablando? Los datos SNMP están en realidad organizados de manera muy similar a los datos LDAP, a los datos DNS, o incluso a la jerarquía de ficheros de su sistema Linux, ¡son todos árboles! Sin embargo, nuestra salida anterior nos ha escondido algunos detalles. Para ver la ruta real en el árbol para cada valor devuelto, añadiremos una opción a nuestro comando anterior:

```

$ snmpwalk -Of -v2c -c public livid interfaces
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifNumber.0 = INTEGER: 4
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifIndex.1 =
INTEGER: 1
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifIndex.2 =
INTEGER: 2
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifIndex.3 =
INTEGER: 3
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifIndex.4 =
INTEGER: 4
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifDescr.1 =
STRING: lo
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifDescr.2 =
STRING: eth0
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifDescr.3 =
STRING: eth1

```

```

.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifDescr.4 =
STRING: sit0
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifType.1 =
INTEGER: softwareLoopback(24)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifType.2 =
INTEGER: ethernetCsmacd(6)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifType.3 =
INTEGER: ethernetCsmacd(6)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifType.4 =
INTEGER: tunnel(131)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifPhysAddress.1 =
STRING:
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifPhysAddress.2 =
STRING: 0:a0:cc:e7:24:a0
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifPhysAddress.3 =
STRING: 0:c:f1:d6:3f:32
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifPhysAddress.4 =
STRING: 0:0:0:0:3f:32
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifAdminStatus.1 =
INTEGER: up(1)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifAdminStatus.2 =
INTEGER: up(1)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifAdminStatus.3 =
INTEGER: down(2)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifAdminStatus.4 =
INTEGER: down(2)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifOperStatus.1 =
INTEGER: up(1)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifOperStatus.2 =
INTEGER: up(1)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifOperStatus.3 =
INTEGER: down(2)
.iso.org.dod.internet.mgmt.mib-2.interfaces.ifTable.ifEntry.ifOperStatus.4 =
INTEGER: down(2)

```

Ahora podemos ver claramente que el árbol de interfaces se coloca por debajo de todos esos otros árboles.

Si reemplazara los puntos separadores con barras ("/"), se vería muy parecido a una jerarquía de directorios, con el valor tras el último punto como el nombre de fichero y todo lo que sigue a los signos "=" como el contenido del fichero. Ahora esto debería empezar a parecer un poco más familiar, más como las salidas de un comando `find` que como algo completamente extraño (espero).

Una estupenda manera de ponerse al tanto de lo que es un dispositivo con SNMP activado (o gestionado por SNMP) es simplemente recorrer el árbol completo de dicho dispositivo. Puede hacer esto apuntando el comando `snmpwalk` al dispositivo sin especificar un árbol, como hemos hecho hasta el momento.

Si bien, asegúrese de redirigir la salida a un fichero, ¡puesto que hay demasiados datos como para digerirlos de una vez!

Para hacer esto, use un comando como el siguiente:

```
$ snmpwalk -Ov -v2c -c public livid > livid.walk
```

Puede ejecutar el mismo comando para *switch*, *router*, cortafuegos, e incluso algunos dispositivos especializados como sensores de contacto de puertas y ventanas y sensores de entorno que miden el calor y la humedad de su sala de máquinas.

El código

Incluso simplemente ceñirse a equipos Linux ofrece una gran riqueza de información.

He escrito un *script* en PHP, ejecutable desde la línea de comandos, que reúne información básica e informa escuchando en puertos TCP, usando sólo SNMP. He aquí el código de dicho *script*:

```
#!/usr/bin/php

<?php
snmp_set_quick_print(1);
$string = "public";
$host = "livid";
check_snmp($host);
spitinfo($host);

function check_snmp($box) //see if this box is running snmp before we throw
//requests at it.
{
    $string="public";
    $infocheck = @snmpget("$box", "$string", "system.sysDescr.0");
    if(! $infocheck)
    {
        die("SNMP doesn't appear to be running on $box");
    }
    else
    {
        return $infocheck;
    }
}

function spitinfo($host)//retrieves and displays snmp data.
{
    $string = "public";
    $hostinfo = @snmpget("$host", "$string", "system.sysDescr.0");
    list ($k)=array(split(" ", $hostinfo));
    $os = $k[0];
    $hostname = @snmpget("$host", "$string", "system.sysName.0");
    $user = @snmpget("$host", "$string", "system.sysContact.0");
    $location = @snmpget("$host", "$string", "system.sysLocation.0");
    $macaddr = @snmpget
```

```
("$host", "$string", "interfaces.ifTable.ifEntry.
    ifPhysAddress.2");
$ethstatus =
    @snmpget("$host", "$string", "interfaces.ifTable.ifEntry.
        ifOperStatus.2");
$ipfwd = @snmpget("$host", "$string", "ip.ipForwarding.0");
$ipaddr = @gethostbyname("$host");
$info=array("Hostname:"=>"$hostname", "Contact:"=>"$user",
    "Location:"=>"$location", "OS:"=>"$os", "MAC Address:"=>
    "$macaddr", "IP Address:"=>"$ipaddr", "Network Status"=>
    "$ethstatus",
    "Forwarding:"=>"$ipfwd");
print "$host\n";
tabdata($info);
print "\nTCP Port Summary\n";
snmp_portscan($hostname);
}

function tabdata($data)
{
    foreach($data as $label=>$value)
    {
        if($label){
            print "$label\t";
        }else{
            print "Not Available";
        }
        if($value){
            print "$value\n";
        }else{
            print "Not Available";
        }
    }
}

function snmp_portscan($target)
{
    $listen_ports = snmpwalk("$target", "public", ".1.3.6.1.2.1.6.13.1.3.
0.0.0.0");
    foreach($listen_ports as $key=>$value)
    {
        print "TCP Port $value (" . getservbyport($value, 'tcp') . ")
listening \n";
    }
}

?>
```

Ejecutar el código

Guarde este *script* en un fichero llamado *report.php*, y hágalo ejecutable (*chmod 775 report.php*).

Una vez hecho esto, ejecútelo con el comando `./report.php`. He incluido un valor para el equipo de destino en el código de este *script* para abreviar las cosas, pero usted probablemente preferiría pasar el equipo al *script* como un argumento de la línea de comandos, o hacerle leer un fichero que contenga una lista de equipos de los que obtener datos. Probablemente querrá además escanear el número de interfaces, y hacer otras cosas geniales que he dejado aparte para ahorrar espacio.

He aquí la salida cuando lo ejecuto con mi sistema Debian de prueba:

```

Hostname:      livid
Contact:       jonesy@linuxlaboratory.org
Location:      Upstairs office
OS:           Linux
MAC Address:   0:a0:cc:e7:24:a0
IP Address:    192.168.42.44
Network Status up
Forwarding:    notForwarding

```

```

TCP Port Summary
TCP Port 80 (http) listening
TCP Port 111 (sunrpc) listening
TCP Port 199 (smux) listening
TCP Port 631 (ipp) listening
TCP Port 649 ( ) listening
TCP Port 2049 (nfs) listening
TCP Port 8000 ( ) listening
TCP Port 32768 ( ) listening

```

Se dará cuenta en el *script* que he usado valores numéricos para buscar en SNMP. Esto es porque, como en otras muchas tecnologías, el texto humanamente legible se corresponde realmente con números, que es lo que las máquinas usan por debajo. Cada registro devuelto por `snmpwalk` tiene un identificador de objeto numérico, OID (*Object ID*).

El cliente usa los ficheros de la MIB (*Management Information Base*, Base de Información de Gestión) que vienen con la distribución Net-SNMP para corresponder los OID numéricos a nombres. En un *script*, sin embargo, la velocidad será esencial, así que querrá saltarse la operación de correspondencia y simplemente obtener los datos. Se dará cuenta además de que he usado SNMP, para hacer lo que normalmente se hace con un escáner de puertos, o con un puñado de llamadas a alguna función como `fsockopen` (en PHP).

Podría haber usado llamadas a función aquí, pero habría sido bastante lento porque habrían estado llamando a cada puerto dentro de un rango, y esperando la respuesta para ver cuáles están abiertos. Usando SNMP, estamos simplemente solicitando la lista de equipos cuyos puertos están abiertos. Sin suposiciones, sin llamadas, y mucho, mucho más rápido.



TRUCO 82

Fuerce a las aplicaciones autónomas a utilizar syslog

Algunas aplicaciones insisten en mantener su propio conjunto de bitácoras. He aquí una manera de reorganizar esas entradas sobre el servicio estándar syslog.

El sueño es éste: trabajar en un entorno en el que todos los servicios de infraestructuras estén ejecutándose en máquinas Linux utilizando software de código abierto fácil de encontrar tal como BIND, Apache, Sendmail, y similares. Hay montones de cosas buenas sobre estos paquetes, y no la menor de ellas es que todos pueden conocer y adoptar los servicios syslog estándar de Linux/Unix. Lo que esto significa es que puede decirles a las aplicaciones, que registren sus actividades utilizando syslog, y luego configurar qué entradas de registros van dónde en un fichero (`syslog.conf`), en lugar de editar ficheros de configuración específicos de aplicaciones.

Por ejemplo, si quiero que Apache guarde registro con syslog, puedo poner una línea como ésta en mi fichero `httpd.conf`:

```
ErrorLog      syslog
```

Esto, por defecto, guardará registro con el servicio syslog "local7". Puede pensar en un servicio syslog como un canal dentro de syslog. Puede configurar syslog para decirle dónde deberían escribirse las entradas que llegan a un canal dado. Así, si quiero que todos los mensajes de Apache que vengan por el canal "local7" se escriban en `/var/log/httpd`, puedo poner la siguiente línea en `/etc/syslog.conf`:

```
local7.*      /var/log/httpd
```

Puede hacer esto para la gran mayoría de aplicaciones de servicio que ejecutan bajo Linux. La gran ventaja es que si una aplicación se porta mal, no tiene que revisar todos sus ficheros de bitácora, siempre puede consultar `syslog.conf` para descubrir dónde están guardando registro sus aplicaciones.

Si bien, en realidad, la mayoría de los entornos no son 100 por 100 Linux. Además, no todo el software es tan amigo de syslog como nos gustaría. De hecho, algún software no tiene ni idea de lo que es syslog, y estas aplicaciones mantienen sus propios ficheros de bitácora, en su propio directorio de registros, sin ninguna opción para cambiar esto de ninguna manera. Algunas de estas aplicaciones son, de otra manera, servicios maravillosos, pero la gente de sistemas es notoriamente implacable en sus exigencias de consistencia en cosas como el registro de eventos. Así que aquí está la chicha de este truco: un ejemplo de servicio que muestra un comportamiento egoísta, y una manera de lidiar con ello.

El FDS (*Fedora Directory Server*, Servidor de Directorios Fedora) puede instalarse de los paquetes binarios de las distribuciones basadas en Red Hat, así como en Solaris y HP-UX. En otras distribuciones Linux, puede compilarse desde los fuentes. Sin embargo, en ninguna de las plataformas FDS sabe nada sobre el servicio `syslog` local. Entra en escena un poco conocido comando llamado `logger`.

El comando `logger` proporciona una interfaz de intérprete de comandos genérica para el servicio `syslog` en su máquina local. Lo que significa que si quiere escribir un *script* de intérprete o en Perl que registre eventos con `syslog` sin escribir funciones específicas de éste, puede simplemente llamar a `logger` desde dentro de dicho *script*, decirle qué escribir y a qué servicio `syslog`, ¡y ya está!

Más allá de esto, `logger` puede además tomar su entrada desde `stdin` (la entrada estándar), lo que quiere decir que puede conectarle con una tubería la información de otra aplicación, y registrará lo que sea que reciba como entrada de la aplicación. Esto es verdaderamente bonito, porque ahora puedo recorrer las bitácoras de FDS en las que estoy interesado y enviarlas a `syslog` con un comando como éste:

```
# exec tail -f /opt/fedora-ds/slaped-ldap/logs/access.log | logger -p local0.debug &
```

Puedo entonces decirle a mi demonio `syslog` que busque todos los mensajes que se han transferido a `logger` por medio de una tubería y enviados a `syslog` en "local 0" y que los ponga en, digamos, `/var/log/ldap/access.log`.

El "debug" al final del nombre de servicio se refiere a lo que, en términos de `syslog`, se conoce como una prioridad. Hay varios niveles de prioridad disponibles para usar por cada servicio `syslog`, de tal manera que una aplicación dada pueda registrar mensajes de diversa severidad como que tienen diferentes prioridades. FDS es un buen ejemplo de una aplicación donde querría utilizar prioridades, el registro de acceso para FDS puede ser extremadamente verboso, así que probablemente querrá separar estos mensajes en su propio fichero de bitácora. Su bitácora de error rara vez se escribe en absoluto, pero los mensajes en ella pueden referirse a la disponibilidad del servicio, así que podría querer que estos mensajes fueran a `/var/log/messages`. En vez de usar otro servicio `syslog` para llevar dichos mensajes a otro fichero, simplemente ejecute un comando como éste:

```
# tail -f /opt/fedora-ds/slaped-ldap/logs/error.log | logger -p local0.notice
```

Ahora vamos a decirle a `syslog` que registre los mensajes en los ficheros adecuados. He aquí las líneas de configuración para los registros de acceso y de error:

```
local0.debug      /var/log/ldap/access.log
local0.notice     /var/log/messages
```

Hay una mejora final que probablemente querrá hacer, y tiene que ver con la salida de `logger`. He aquí una línea que se hace en un fichero de bitácora desde `logger` tal y como lo ejecutamos antes, con sólo la opción "-p" para indicar el servicio a usar:

```
Aug 26 13:30:12 apollo logger: connection refused from 192.168.198.50
```

Bien, esto no es muy útil, ¡porque lista `logger` como la aplicación que registra la entrada en la bitácora! Puede decir a `logger` que se haga pasar por otra aplicación de su elección usando la opción "-t", así:

```
# tail -f access.log | logger -p local0.debug -t FDS
```

Ahora, en vez de mostrar la aplicación remitente como `logger`, lo hará como `FDS`.

Por supuesto, hay alternativas probables a usar `logger`, pero a veces implican escribir demonios Perl o PHP que lleven a cabo básicamente la misma función que nuestra solución. En su camino, puede ser capaz de llegar a una solución mejor para su sistema, pero para el arreglo de "aquí y ahora", `logger` es una buena herramienta a llevar en su cinturón.



TRUCO
83

Monitoree sus ficheros de bitácora

Utilice herramientas existentes o simples *script* caseros para ayudarle a filtrar el ruido de sus ficheros de bitácora.

Si soporta un montón de servicios, un montón de equipos, o ambas cosas, sin duda alguna el problema de hacer un uso eficiente de los ficheros de bitácora es familiar para usted. Seguro, puede tener una herramienta que le envíe la salida de su bitácora cada hora, pero esta información a menudo va a la basura debido a la extrema proporción de ruido. Puede intentar además filtrar la información y usar una herramienta tal como `logwatch` para informarle diariamente sólo de las cosas que son importantes para. Sin embargo, estos informes no ayudan a alertarle del peligro inminente. Por esto, necesita más que una herramienta informativa. Lo que realmente necesita es un monitor de bitácora; algo que vigile los ficheros de bitácora continuamente y le haga saber sobre cualquier cosa extraña.

Los monitores de bitácora en muchos entornos tienen forma humana: los administradores mantienen a menudo varias ventanas abiertas con varios ficheros de bitácora mostrando su contenido (con un comando `tail` continuo normalmente) en ellas, o usan algo como `root-tail` para sacar esas bitácoras de las ventanas directamente a sus fondos de escritorio. Puede incluso enviar su salida a un

cliente Jabber. Esto son cosas maravillosas, pero, de nuevo, no ayuda a filtrar el ruido no deseado en los ficheros de bitácora, y no es muy efectivo si todos los humanos han salido a comer (por decir algo).

Hay un gran número de soluciones a este problema. Una es simplemente asegurarse de que sus servicios están registrando eventos al nivel y con los servicios `syslog` correctos, para después asegurarse de que su demonio `syslog` está configurado para dividir las cosas y registrarlas en los ficheros apropiados. Esto puede ayudar hasta cierto grado, pero lo que queremos es esencialmente tener un "grep", siempre ejecutando y en tiempo real, de nuestros ficheros de bitácora, que nos alertará de las coincidencias que encuentre enviándonos un correo electrónico, actualizando una página Web, o enviándola.

Utilizar log-guardian

Hay un par de herramientas por ahí que puede usar para la monitorización de bitácora. Una es `log-guardian`, que es un *script* en Perl que le permite monitorizar múltiples ficheros de bitácora según varios patrones aportados por el usuario. Puede también configurar la acción que `log-guardian` toma cuando se encuentra una coincidencia. El inconveniente de usar `log-guardian` es que debe tener algún conocimiento de Perl para configurarlo, ya que las acciones proporcionadas por el usuario están en forma de subrutinas Perl, y otros parámetros de configuración como *hash* Perl. Todas éstas se ponen directamente en el mismo *script* o en un fichero de configuración separado. Puede conseguir `log-guardian` desde su página Web <http://www.tifaware.com/perl/log-guardian/>. Una vez descargado, puede poner el *script* donde quiera que almacene las herramientas del sistema local, como bajo `/opt` o en `/var/local`. Puesto que no viene con ningún *script* de iniciación, necesitará añadir una línea similar a ésta en el fichero `rc.local` de su sistema:

```
/var/local/bin/log-guardian &
```

El poder real de `log-guardian` proviene del módulo de Perl `File::Tail`, el cual es un pedazo de código bastante robusto que actúa exactamente como `tail -f`. Este módulo es requerido por `log-guardian`. Para determinar si lo tiene instalado, puede escribir algo como `locate perl | grep Tail`, o ejecutar un "una-línea" rápido de Perl como éste en la línea de comandos:

```
$ perl -e "use File::Tail;"
```

Si esto devuelve un largo, extenso error comenzando con "Can't find Tail/File.pm" o algo similar, necesitará instalarlo usando CPAN, lo cual debería ser absolutamente simple usando el siguiente comando:

```
# perl -MCPAN -e shell
```

Esto le dará un marcador de línea de comando CPAN, donde puede ejecutar el siguiente comando para instalar el módulo:

```
> install File::Tail
```

El módulo `File::Tail` es seguro para usar en ficheros de bitácora que se mudan, mueven, o reemplazan regularmente, y no necesita que reinicie o que ni siquiera piense en su *script* cuando esto ocurra. Es completamente fácil de usar, y sus características más avanzadas le permitirán monitorizar múltiples ficheros de bitácora simultáneamente. He aquí un simple filtro que he añadido al propio *script* `log-guardian` para que encaje con las conexiones `sshd` que lleguen al servidor:

```
'/var/log/messages' => [
{
  label    => 'SSH Connections',
  pattern  => "sshd",
  action => sub {
    my $line = $_[1];
    print $line;
  },
},
],
```

Este es el filtro más simple para `log-guardian` que puede escribir. Busca todo lo que se escriba en `/var/log/messages` que contenga la cadena "shd" e imprime las líneas que encuentre por la salida estándar (`stdout`). Desde ahí, puede enviarla a otra herramienta para un proceso mayor o conectarla con una tubería con el comando `mail`, en cuyo caso podría ejecutar `log-guardian` así:

```
# /var/local/bin/log-guardian | mail jonesy@linuxlaboratory.org
```

Por supuesto, hacer esto enviará cada línea en un mensaje de correo separado, así que podría preferir para simplificar dejarlo ejecutar en un terminal. Será capaz de monitorizar esta salida un poco más fácilmente que los propios ficheros de bitácora, ya que le ha filtrado gran parte del ruido.

Este filtro `sshd` es simplemente un ejemplo, el "patrón" puede consistir en cualquier código Perl que devuelva alguna cadena que el programa pueda usar para encontrar una correspondencia con las entradas de bitácora entrantes, y la "acción" realizada en respuesta a dicha correspondencia puede ser literalmente cualquier cosa que sea capaz de inventar usando Perl. ¡Lo que hace las posibilidades casi infinitas!

Usar logcheck

La utilidad `logcheck` no es un monitor en tiempo real que le alerte al primer signo de peligro. Sin embargo, es una manera muy simple de eliminar el ruido en

sus bitácoras. Puede descargar logcheck de <http://sourceforge.net/projects/sentrytools/>.

Una vez descargado, desempaquete la distribución, haga cd al directorio resultante, y como súper-usuario, ejecute make linux. Esto instalará los ficheros bajo /usr/local. Hay unos pocos ficheros a editar, pero las cosas que necesitan edición son simples líneas sueltas; la configuración es muy intuitiva, y los ficheros están muy bien comentados.

El principal fichero que debe comprobarse completamente para asegurar una configuración adecuada es /usr/local/etc/logcheck.sh. Este fichero contiene secciones que se marcan con etiquetas tales como "CONFIGURATION" y "LOGFILE CONFIGURATION", de tal manera que pueda encontrar fácilmente esas variables del fichero que pudieran necesitar cambios. Probablemente la cosa más obvia a cambiar es la variable "SYSADMIN", la cual le dice a logcheck dónde enviar la salida.

```
SYSADMIN=user@mydomain.com
```

Debería repasar también las otras variables, ya que las variables de rutas y las rutas a binarios también se establecen en este fichero.

Una vez que esté listo, lo siguiente que querrá hacer es editar el fichero crontab del súper-usuario, lo que puede hacerse convirtiéndose en súper-usuario y ejecutando el siguiente comando:

```
# crontab -e
```

Puede programar logcheck para que se ejecute tan a menudo como quiera. La siguiente línea programará logcheck para que se ejecute una vez cada hora, todos los días, a la hora y 50 minutos:

```
50 * * * * /bin/sh /usr/local/etc/logcheck.sh
```

Puede elegir cualquier periodo de tiempo que quiera, pero una vez por hora (o menos, en sitios más pequeños o redes caseras) debería ser suficiente.

Una vez que haya guardado la entrada crontab, comenzará a recibir correo de logcheck con informes que deba saber sobre lo que se encuentre en sus ficheros de bitácora. Comprende a qué entradas de bitácora ir dentro de los informes usando la siguiente metodología:

- Hace corresponder una cadena que haya anotado como significativa poniéndola en /usr/local/etc/logcheck.hacking.
- No hace corresponder ninguna cadena que haya anotado como ruidosa poniéndola en /usr/local/etc/logcheck.ignore.

Estos dos ficheros son simplemente listas de cadenas que logcheck intentará cotejar con las entradas en los ficheros de bitácora que recorre para crear los

informes. En realidad hay también un tercer fichero, /usr/local/etc/logcheck.violations.ignore, que contiene cadenas que se cotejan sólo con las entradas que han sido ya marcadas como violaciones. Hay un ejemplo de esto en el fichero INSTALL que viene con la distribución, que es más perfecto que nada que pudiera ocurrírseme, así que lo reiteraré aquí en castellano:

```
Feb 28 21:00:08 nemesis sendmail[5475]: VAA05473: to=crowland, ctladdr=root
(0/0), delay=00:00:02, xdelay=00:00:01, mailer=local, stat=refused
```

```
Feb 28 22:13:53 nemesis rshd: refused connect from hacker@evil.com:1490
```

La primera entrada proviene de sendmail y es un error bastante común. La línea "stat" indica que el equipo remoto rechazó las conexiones (stat=refused). Esto puede ocurrir por varias razones y generalmente no es un problema.

La última línea, sin embargo, indica que una persona (hacker@evil.com) ha intentado sin éxito iniciar una sesión rsh en mi máquina. Esto es malo (por supuesto, para empezar no debería estar ejecutando rshd).

El fichero logcheck.violations encontrará la palabra "refused" y la marcará para informar de ella; sin embargo, esto informará de que ambas instancias son malas, y recibirá falsas alarmas de sendmail (ambas contienen la palabra "refused").

Para sortear este falso positivo sin eliminar a su vez cosas de las que quiera saber, ponga una línea como ésta en /usr/local/etc/logcheck.violations.ignore:

```
mailer=local, stat=refused
```

Esto cotejará sólo la entrada de bitácora de Sendmail y la ignorará. Cualquier otra entrada será capturada si contiene la cadena "refused".

Por supuesto, probablemente le llevara algún tiempo afinar los informes que logcheck envía, pero el modelo de forzarle a decir a la herramienta que ignore cosas explícitamente asegura que sólo ignora lo que usted le dice que ignore, en vez de asumir cosas sobre su entorno.



TRUCO

84

Envíe mensajes de bitácora a su cliente Jabber

Utilice las características escondidas de syslog y un rápido script para enviar mensajes de syslog directamente a su escritorio.

Por fin tiene un sala de máquinas configurada con bitácora centralizada. Ahora ya no necesita abrir 50 ventanas de terminal diferentes para mostrar la salida de tail sobre los ficheros de bitácora de todos sus servidores Web.

En vez de esto, simplemente abre una sesión en el equipo central de bitácora, hace `tail` sobre el fichero, y se ocupa de sus asuntos.

Pero, ¿y si pudiera tener los mensajes de bitácora realmente importantes, quizás sólo aquéllos que van al servicio `auth.warning`, enviados directamente a su escritorio de una manera que llamaran su atención incluso si se fuera y volviera un segundo después de que el mensaje ya hubiera desaparecido de la pantalla de su sesión `tail`?

En realidad puede llevar a cabo esto de muy distintas maneras, pero mi favorita es enviar cualquier cosa que llega a través de mi filtro `syslog` a mi cliente Jabber. Como probablemente la mayoría ya sabe, Jabber es un protocolo de código abierto de mensajería instantánea soportado por clientes Linux como GAIM y Kopete.

Este truco funciona porque resulta que `syslog` tiene la habilidad de enviar o copiar mensajes a una tubería nombrada (o FIFO). Una tubería en el mundo Linux se parece mucho a una tubería en el mundo de la fontanería: envía algo por un extremo y sale por (o es accesible a través de) el otro extremo. Según esta lógica puede ver que si puedo enviar mensajes de advertencia a una tubería, debería ser capaz de conectar a dicha tubería algún tipo de grifo desde el cual pudiera acceder a dichos mensajes. Esto es exactamente lo que haremos. Por ejemplo, para enviar sólo aquellos mensajes que pertenezcan a intentos de inicio de sesión fallidos (`auth.warning`) a una tubería nombrada, pondría la siguiente línea en `/etc/syslog.conf`:

```
auth.warning      | /var/log/log-fifo
```

Con esto hecho, necesita a continuación crear la tubería nombrada como "log-fifo", lo que puede hacer con el siguiente comando:

```
# mkfifo /var/log/log-fifo
```

La siguiente vez que reinicie su demonio `syslog`, se enviarán los mensajes a "log-fifo". Puede probar rápidamente que está funcionando ejecutando el siguiente comando y mirando la salida:

```
# less -f /var/log/log-fifo
```

Para tener esos mensajes en un cliente Jabber abierto, puede hacer que un *script* lea de "log-fifo", lo envuelva en el XML apropiado, y lo envíe para ser encajado a su cuenta Jabber de destino.

El *script* que yo uso es una versión trucada del *script* Perl original de DJ Adams llamado `jann`, y requiere de el módulo `Net::Jabber`, el cual está fácilmente (si no ya instalado) disponible para la mayoría de las distribuciones. Yo lo denominé `jann-log`.

El código

Este *script* lee la salida de una FIFO y la reenvía como un mensaje Jabber:

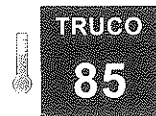
```
#!/usr/bin/perl
use Net::Jabber qw(Client);
use strict;
# Announce resources
my %resource = (
    online => "/announce/online",
);
# default options
my %option = (
    server => "moocow:5222",
    user   => "admin",
    type   => "online",
);
# Default port if none specified
$option{server} = "moocow:5222";
# Ask for password if none given
unless ($option{pass}) {
    print "Password: ";
    system "stty -echo";
    $option{pass} = <STDIN>;
    system "stty echo";
    chomp $option{pass};
    print "\n";
}
# Connect to Jabber server
my ($host, $port) = split(":", $option{server}, 2);
print "Connecting to $host:$port as $option{user}\n";
my $c = new Net::Jabber::Client;
$c->Connect(
    hostname => $host,
    port     => $port,
) or die "Cannot connect to Jabber server at $option{server}\n";
my @result;
eval {
    @result = $c->AuthSend(
        username => $option{user},
        password => $option{pass},
        resource => "GAIM",
    );
};
die "Cannot connect to Jabber server at $option{server}\n" if $@;
if ($result[0] ne "ok") {
    die "Authorisation failed ($result[1]) for user $option{user} on $option{server}\n";
}
print "Sending $option{type} messages\n";
# The message. Change the file name in this 'open' line to
```

```
# the name of your fifo.
open(STATUS, "cat /var/log/log-fifo 2>&1 |")
|| die "UGH: there's issues: $!";
while (<STATUS>) {
    my $xml .= qq[<subject>] .
    ($option{type} eq "online" ? "Admin Message" : "MOTD") .
    qq[</subject>];
    my $to = $host . $resource{$option{type}};
    $xml .= qq[<message to="$to">];
    $xml .= qq[<body>];
    my $message = $_;
    $xml .= XML::Stream::EscapeXML($message);
    $xml .= qq[</body>];
    $xml .= qq[</message>] ;
    $c->SendXML($xml);
    print $xml;
}
```

Ejecutar el código

Sitúe este *script* en un lugar accesible sólo para usted y/o su equipo de administración (por ejemplo, `/var/local/adm/bin/jann-log`) y cambie los permisos de tal manera que pueda ser escrito y ejecutado sólo por su grupo de administración. Abra entonces un cliente Jabber en su escritorio y conéctese a su servidor Jabber. Una vez hecho esto, ejecute el *script*. Debería confirmar que se ha conectado al servidor Jabber y que está esperando mensajes de la FIFO.

Una simple manera de probar su servicio `auth.warning` en el servidor donde `jann-log` está escuchando mensajes es hacer SSH al equipo y usar a propósito una contraseña equivocada para intentar acceder a él.



Monitorice la disponibilidad de servicio con Zabbix

Es bonito tener alguna advertencia antes de que las llamadas pidiendo ayuda le inunden. ¡Sea el primero en saber qué está ocurriendo con los servidores críticos en su red!

Les pasará a todos tarde o temprano: estará ocupándose de sus propios asuntos felizmente, inconsciente de que la red está cayendo de rodillas, hasta que una secretaria reclame que Internet no funciona. Para entonces, todos los jefes se habrán dado cuenta, y todos querrán respuestas. Surge el pánico más absoluto, y usted corre por toda la oficina, haciendo ping a cosas aleatorias para intentar descubrir qué está sucediendo. ¿No sería bonito si tuviera algún tipo de mapa detallado de la red en tiempo real que pudiera monitorizar servicios y decirle qué está ocurriendo? ¡Zabbix al rescate! Zabbix es una herramienta de monitorización

de equipos que puede hacer cosas sorprendentes. Continúe leyendo para ver cómo puede aplicarlo a su propia red.

Dependencias

Zabbix es una bestia complicada, así que asegúrese de que tiene una versión relativamente recién instalada. Si hace tiempo que no actualiza, éste podría ser el momento para hacerlo. Puesto que Zabbix está completamente basado en Web, obviamente necesitará también un servidor Web. Al mismo nivel para este objetivo, Apache o Apache2 es el servidor recomendado a elegir. Asegúrese cuando instale Apache de que lo configura también con `mod-php` activado. Esto asegura que Apache pueda entender el PHP integrado que hace de Zabbix lo que es. A continuación asegúrese de que tiene la librería PHP GD instalada (disponible en <http://www.boutell.com/gd/>). Aunque Zabbix técnicamente funcionará sin ésta, esto no es lo recomendado, ya que es la librería que genera los mapas de red y las gráficas que le hacen tan útil. Finalmente, necesitará una base de datos SQL. Aunque Zabbix soporta tanto PostgreSQL como MySQL, en este ejemplo estaremos usando MySQL.

Zabbix está escrito en PHP, así que asegúrese de que tiene una versión relativamente recién instalada. Si hace tiempo que no actualiza, éste podría ser el momento para hacerlo. Puesto que Zabbix está completamente basado en Web, obviamente necesitará también un servidor Web. Al mismo nivel para este objetivo, Apache o Apache2 es el servidor recomendado a elegir. Asegúrese cuando instale Apache de que lo configura también con `mod-php` activado. Esto asegura que Apache pueda entender el PHP integrado que hace de Zabbix lo que es. A continuación asegúrese de que tiene la librería PHP GD instalada (disponible en <http://www.boutell.com/gd/>). Aunque Zabbix técnicamente funcionará sin ésta, esto no es lo recomendado, ya que es la librería que genera los mapas de red y las gráficas que le hacen tan útil. Finalmente, necesitará una base de datos SQL. Aunque Zabbix soporta tanto PostgreSQL como MySQL, en este ejemplo estaremos usando MySQL.

Instalar Zabbix

Por desgracia, instalar Zabbix no es tan directo como muchas de las aplicaciones que hemos discutido hasta ahora. Algunas partes de su instalación, que remarcaré según avanzamos, son opcionales.

El primer paso para tener Zabbix funcionando es descargar y desempaquetar el código fuente. Puede encontrar éste en su página Web (<http://www.zabbix.com>). Cuando se escribió este libro, la última versión estable era la 1.0. Descargue el archivo de la última versión, desempaquétele en su ubicación de compilación habitual, y navegue al nuevo directorio.

Primero, necesitaremos configurar Zabbix para que haga uso de la opción de base de datos que hemos seleccionado (MySQL) y para usar SNMP. Ejecute el siguiente comando para preparar la instalación:

```
$ ./configure --with-mysql --with-net-snmp
```

Esto no debería llevar demasiado tiempo, ¡así que no agarre una cerveza justo ahora! Antes de pasar al `make`, necesitará tomar un segundo para preparar la base de datos MySQL para Zabbix. Navegue al directorio `create/` y arranque

MySQL, cree la base de datos Zabbix, y concatene los *script .sql* para poblar las tablas:

```
# mysql -u<usuario> -p<contraseña>
Mysql> create database Zabbix;
Mysql> quit;
# cd create/mysql
# cat schema.sql |mysql -u<usuario> -p<contraseña> Zabbix
# cd ../data
# cat data.sql |mysql -u<usuario> -p<contraseña> Zabbix
```

Ahora puede volver a la raíz del directorio Zabbix y ejecutar el comando make.

Una vez que make termine, tómese un momento para copiar los contenidos de directorio bin/ a alguna parte de su ruta por defecto. Yo tiendo a usar /usr/local/bin.

```
# cp bin/* /usr/local/bin
```

Este es un mecanismo de instalación bastante poco sofisticado, pero ya casi ha acabado. Ahora tenemos que establecer unas cuantas variables, de tal manera que PHP sepa cómo acceder adecuadamente a su base de datos. Navegue a frontend/php/include en su directorio fuente de Zabbix y abra el fichero db.inc.php en su editor de texto favorito. Haga los siguientes cambios:

```
$DB_TYPE = "MySQL";
$DB_SERVER = "localhost";
$DB_DATABASE = "Zabbix";
$DB_USER = "<usuario MySQL>"
$DB_PWD = "<contraseña MySQL>"
```

La variable "\$DB_DATABASE" es el nombre de la base de datos que ha creado anteriormente en MySQL para Zabbix. Una vez que se han hecho esos cambios, copie los ficheros PHP a la raíz de su Web:

```
# cp -R frontend/php/* /srv/www/htdocs/
```

Ahora cree el directorio /etc/zabbix y copie los ficheros de configuración de muestra en él:

```
# mkdir /etc/zabbix
# cp misc/conf/zabbix_suckerd.conf /etc/zabbix/zabbix_suckerd.conf
# cp misc/conf/zabbix_trapperd.conf /etc/zabbix/zabbix_trapperd.conf
```

Estos ficheros de configuración de muestra están bien para aplicaciones de corta duración, pero si está planeando desplegar Zabbix a larga escala o a extensión de compañía debería leer la sección de ficheros de configuración en el manual online de Zabbix, disponible en <http://www.zabbix.com/manual/v1.1>

/config_files.php. Hacer esto le ahorrará muchos dolores de cabeza en el futuro. Una vez que se han movido estos ficheros, iya ha terminado la instalación! Todo lo que queda es encender los demonios Zabbix, y asegurarse de que funcionan:

```
# zabbix_suckerd
# zabbix_trapperd
```

Asumiendo que todo fue como estaba previsto, ahora puede apuntar su navegador Web a <http://127.0.0.1> y ver su nueva instalación Zabbix. Cuando llegue a la pantalla de inicio de sesión, introduzca su nombre de usuario y deje el campo de contraseña en blanco. Una vez iniciada la sesión, tómese un momento para cambiar la contraseña por defecto.

Monitorizar equipos

Tras esta instalación, iahora ciertamente merece hacer algo fácil! Afortunadamente, Zabbix parece estar diseñado con la facilidad en mente. Vamos a comenzar añadiendo algunos equipos para monitorizar. La sección superior de la pantalla tiene las barras de navegación que usará para navegar por Zabbix. Haga clic en Hosts para añadir un nuevo equipo a su monitorización. La figura 9.3 muestra los campos disponibles al añadir un nuevo equipo en la pestaña Hosts en Zabbix.

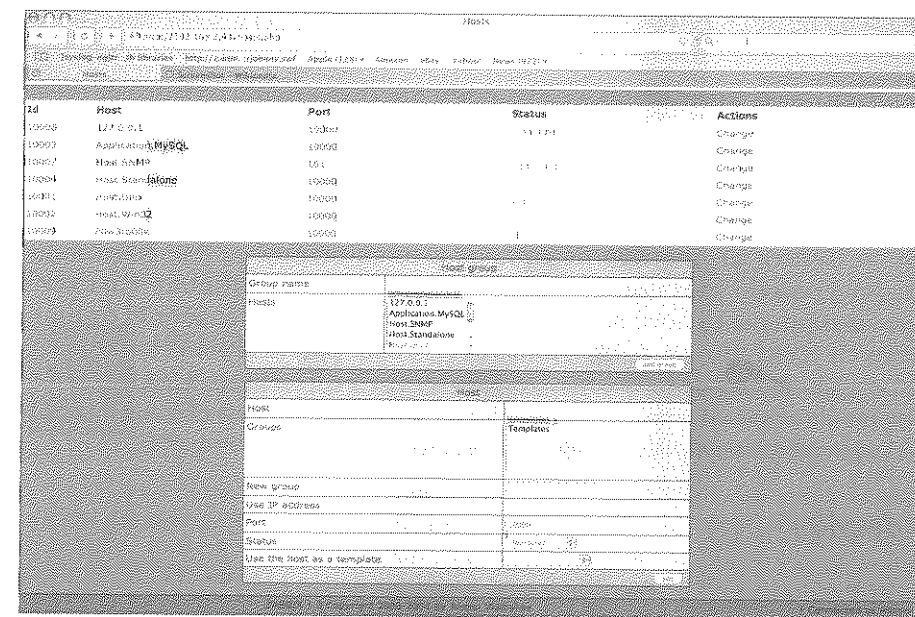


Figura 9.3. La pestaña Hosts de Zabbix.

Verá aquí que tiene varias opciones al añadir sus nuevos equipos. Rellene las opciones para que cubran sus necesidades y haga clic en **Add**. Fíjese que si monitorizara por nombre de equipo en vez de por DNS (lo cual a menudo es una excelente idea), marcar la casilla **Use IP Address** le dará un campo adicional para proporcionar la dirección IP a monitorizar. Por ejemplo, asumamos que queremos configurar Zabbix para que nos notifique si 192.168.2.118 deja alguna vez de servir tráfico FTP. Para hacer esto, en la pestaña **Hosts**, introduciríamos 192.168.2.118 en el campo **Host**. Cambiaríamos entonces el puerto a 22, puesto que estamos interesados en tráfico FTP. A continuación pasaremos a la pestaña **Items**. Necesitaremos escribir una descripción para este elemento, así que lo llamaremos "Home-FTP". Bajo **Type**, seleccionamos **Simple check**. En el campo **Key**, introducimos "ftp." El resto podemos dejarlo como está. Ahora espere unos cuantos minutos, y compruebe la pestaña **Latest Values**. Debería ver una opción aquí para 192.168.2.118 (o para el nombre de equipo si le dio alguno). Puesto que el servidor FTP está ejecutando, obtenemos un valor de 1. Si el servidor no hubiera estado ejecutándose, veríamos 0 en ese campo. Fíjese que a la derecha tendrá la opción para mostrar gráficas, tendencias, y comparar los datos recolectados en el tiempo. Esto permite un análisis detallado de los datos del tiempo de funcionamiento y la disponibilidad de sus servidores. Es además una excelente demostración de las cualidades gráficas de Zabbix.

Hacer un mapa de la red

El último aspecto de Zabbix que vamos a mirar es la característica de creación de mapas (mostrada en la figura 9.4). Esta es una excelente herramienta para proporcionar un mapa de referencia rápida de la red mostrando su estado detallado. Para comenzar, haga clic en el botón inferior **Network Maps**. Cree un nuevo mapa de red rellenando el nombre con el que quiere llamar a su nuevo mapa. Si tuviera un montón de equipos a monitorizar, cambie el tamaño del mapa para hacerlo más grande. Haga clic en **Add** para continuar. Una vez que ha creado su mapa, es hora de añadirle algunos equipos. Seleccione el equipo que creamos en el ejemplo anterior, "Home-FTP". Puede entonces seleccionar las coordenadas que desea para mostrar los iconos que representan "Home-FTP" en el mapa. Puede continuar añadiendo equipos y situándolos en el mapa hasta que tenga una completa representación de su red.

Los detalles

Lo que hemos cubierto aquí es una fracción de las habilidades de Zabbix. Si quiere entrar más en profundidad con él, puede instalar agentes Zabbix en las máquinas que desee monitorizar.

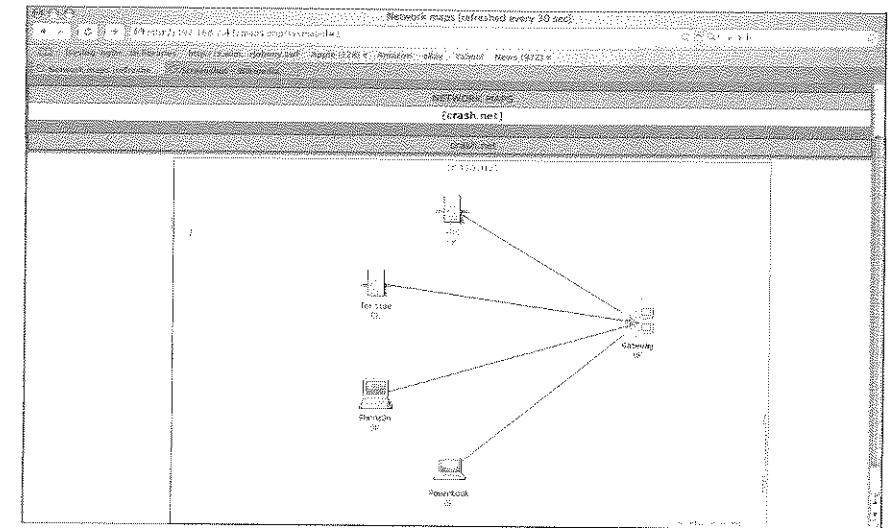
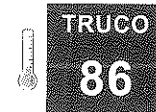


Figura 9.4. Las características de creación de mapas de Zabbix.

Una vez que haya hecho esto, puede monitorizar estadísticas tales como la utilización del CPU, el espacio de disco, y cualquier otra cosa que pueda ser monitorizada vía SNMP. Puede además definir disparadores a medida para que le alerten inmediatamente de situaciones de emergencia. La definición de disparadores es muy detallada, y puede volverse muy elaborada y compleja. Si quisiera aprender más sobre esta herramienta de monitorización de red increíblemente flexible, consulte la página Web de Zabbix en <http://www.zabbix.com> para más información. Hay un foro bastante activo ahí dedicado a ayudar usuarios que lo necesiten y a compartir consejos y trucos de configuración.

—Brian Warshawsky



Afinar el demonio syslog

No puede ver los problemas de los que no se le informa. Configurar correctamente el demonio de bitácora de sistema y los niveles de registro de información asegura que siempre sabrá lo que está sucediendo.

Los sistemas Linux registran la información de arranque, la información de estado de procesos, y una cantidad significativa de información de acceso y error en el fichero de bitácora del sistema, `/var/log/messages`, usando un demonio de sistema conocido como `syslog`. Pero, ¿cuándo fue la última vez que miró este fichero? Si nunca ha dedicado ningún tiempo a afinar el demonio `syslog`, el fichero de bitácora de su sistema probablemente contiene un trágicamente re-

vuelto lío de avisos de finalización de tareas de cron, avisos de arranque, entradas "MARK", y cualquier número de mensajes de bitácora de otro demonio o servicio. Imagine si pudiera configurar syslog para volcar toda esta información donde quiera, y ordenarla a su vez. Bien, es Linux de lo que estamos hablando aquí, ¡así que por supuesto que puede configurar syslog como usted quiera!

Dando sentido a syslog.conf

Un fichero de configuración llamado `/etc/syslog.conf` controla el demonio syslog. Tan falto de imaginación como su nombre, apréndaselo bien porque éste es un fichero con el que necesitará familiarizarse si quiere dominar las entrañas del registro de eventos del sistema Linux. El fichero puede no tener mucho sentido a primera vista, pero aquí utilizaré un simple fichero `syslog.conf` para explicar un poco más la sintaxis:

```
# Log all kernel messages to the console.
# Logging much else clutters up the screen.
# kern.*                               /dev/console
# Log anything (except mail) of level info or higher.
# Don't log private authentication messages!
*.info;mail.none;authpriv.none;cron.none    /var/log/messages
# The authpriv file has restricted access.
authpriv.*                                   /var/log/secure
# Log all the mail messages in one place.
mail.*                                       -/var/log/maillog
# Log cron stuff
cron.*                                       /var/log/cron
# Everybody gets emergency messages
*.emerg                                     *
# Save news errors of level crit and higher in a special file.
uucp,news.crit                             /var/log/spooler
# Save boot messages also to boot.log
local7.*                                    /var/log/boot.log
```

Como puede ver en las líneas no comentadas de este ejemplo, hay tres partes principales para cada línea activa del fichero de configuración. La primera entrada en una línea se llama servicio, que es el subsistema subyacente que crea los mensajes para los ficheros de bitácora. Hay 13 valores predefinidos para los servicios de sistema: `auth`, `authpriv`, `cron`, `daemon`, `ftp`, `kern`, `lpr`, `mail`, `mark`, `news`, `syslog`, `user`, y `uucp`. Además de estos, hay ocho más, nombrados de `local0` a `local7`, que son para que los usen programas al implementar sus propios mensajes syslog. Cada uno de los servicios predefinidos hace referencia a un aspecto específico del sistema. Por ejemplo, "auth" hace referencia al sistema de autorización de Linux, incluyendo programas como `login` y `su`. El servicio "mark" es usado de manera interna por syslog, y debería dejarse solo por ahora. El servi-

cio "daemon" es para otros demonios de sistema que no están listados específicamente. Puede representar todos los servicios disponibles usando el símbolo asterisco (*).

La segunda parte de una línea de configuración es la prioridad, la cual está separada de su servicio asociado por una coma. Cada vez que una parte del sistema manda un mensaje a syslog, ese mensaje es codificado con una prioridad. Básicamente, el programa está dejando saber a syslog lo importante que es este mensaje. De más bajo a más alto, los niveles de prioridad son `debug`, `info`, `notice`, `warning`, `err`, `crit`, `alert`, y `emerg`. Cuanto mayor es la prioridad, más importante es el mensaje. Una vez que se topa con la prioridad "emerg", el sistema está aproximándose rápidamente a un pánico de núcleo de sistema (*kernel panic*) y probablemente no puede usarse. Puede representar mensajes de cualquier prioridad usando el símbolo asterisco. Por ejemplo, "local7.*" significa "mensajes de cualquier prioridad desde el servicio local7."

El tercer y último aspecto de la línea de configuración es la acción. Esto es básicamente tan sólo una corta sección que le dice a syslog qué hacer con la información que ha recibido. Para explicar esto mejor, echemos un vistazo a una línea de ejemplo del fichero de configuración de muestra proporcionado anteriormente:

```
# Log cron stuff
cron.*                               /var/log/cron
```

Pocas cosas son más molestas que recorrer `/var/log/messages` teniendo que vadear todos los mensajes de cron, así que este tipo de opción de configuración viene muy bien. Este ejemplo quiere decir que los mensajes de todas las prioridades emitidos por el servicio cron deberían ser enviados al fichero de bitácora `/var/log/cron`. Como se mencionó anteriormente, el asterisco es una característica comodín que le dice a syslog que aplique la misma regla a cada mensaje de cron, independientemente de su prioridad. Puede hacer cosas parecidas con el comodín asterisco para el servicio, tales como instruir a syslog para enviar todos los mensajes de prioridad "warning" o superior a un fichero de bitácora específico:

```
*.warning                           /var/log/problems
```

Alertas en tiempo real desde la bitácora de sistema

Otras características de comodín que pueden usarse incluyen el signo arroba (@), para enviar mensajes a equipos syslog remotos; un guión (-), para decirle a syslog que no sincronice los discos después de cada mensaje; y un asterisco en la sección de acciones de la configuración, para alertar a todo el mundo que se

encuentre en el sistema sobre algún problema. Por ejemplo, mire el siguiente ejemplo del fichero de configuración de muestra:

```
# Everybody gets emergency messages
*.emerg
```

El asterisco final en esta línea le dice a `syslog` que envíe un mensaje a cada usuario conectado por medio del comando `wall` (*Write to ALL*, Escribir a TODOS los usuarios) para informarles de cualquier condición de emergencia. Estos mensajes aparecerán en todas las ventanas activas de terminal del sistema. Puede pensar en las configuraciones como ésta como un sistema de emisión de mensajes de emergencia de Linux.

Otra línea interesante en el fichero `syslog.conf` de ejemplo mostrado anteriormente en este truco es la línea que se dirige a los mensajes `syslog` del núcleo de sistema. En vez de enviarse a un fichero de bitácora, todos estos mensajes se envían a la consola. Un popular truco usando esta característica es dirigir muchos de los mensajes `syslog` a una consola virtual en vez de a la principal. Yo hago esto a menudo en máquinas que no se usan mucho para trabajo local, pero que aun así tienen monitores. Por ejemplo, especificar esta línea:

```
auth,kern.* /dev/tty5
```

me permite ver los mensajes `syslog` de todos los que inician sesión, y cualquier problema con el núcleo de sistema, simplemente cambiando la máquina a la consola virtual número 5 (**Alt-F5**) y dejándola ahí con el monitor encendido. Ahora, cada vez que pase por esa máquina, puedo seguir la pista de los usuarios que inician y cierran sesión, o cualquier otra cosa para la que la haya configurado. Cuando necesite trabajar en el servidor sobre la marcha, simplemente cambio de nuevo a mi consola primaria (**Alt-F1**), y los mensajes continúan siendo enviados a la consola número 5.

Centralizar las bitácoras para cómodo acceso

Otra interesante opción de `syslog` es el registro remoto de eventos. Aunque el propio `syslog` permite esto, hay una solución más robusta que se puede encontrar en `syslog-ng`, una nueva versión de `syslog`. `syslog` le permite enviar mensajes a equipos remotos, pero lo hace en texto plano por toda la red, así que debería usar esta característica con precaución. He aquí como funciona: añadiendo un signo arroba y un nombre de equipo o dirección IP en la sección de acciones del fichero de configuración, puede especificar que `syslog` envíe sus mensajes a otro servidor `syslog` remoto en espera. Este servidor necesitará tener el demonio `syslog` iniciado con la opción `"-r"` para permitirle escuchar por el puerto 514 los mensajes `syslog` entrantes. La siguiente línea muestra un ejem-

plo de envío de todos los mensajes críticos del núcleo a la máquina remota "aardvark" para salvaguardarlos.

```
kern.crit @aardvark
```

El registro remoto de eventos puede ser extremadamente útil en el caso de una caída de sistema, porque le permite ver mensajes de bitácora a los que no podría acceder de otra manera (ya que el sistema que los emitió está caído). Como se mencionó previamente, estos mensajes se envían en texto plano a lo largo de la red, así que asegúrese de usar el registro remoto de eventos de `syslog` con precaución, y nunca lo haga por Internet. Además, fíjese que si envía ciertos tipos de mensajes a un servidor de bitácora remoto, no se registran localmente, a menos que cree otra entrada que envíe también esos mensajes a la bitácora local, como en el siguiente ejemplo:

```
kern.crit @aardvark
kern.crit /var/log/messages
```

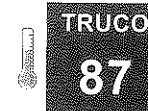


Otro interesante problema potencial de seguridad con el registro de eventos remoto de `syslog` es que iniciar el demonio con la opción `"-r"` para recibir entradas remotas, significa que cualquier equipo puede enviar una mensaje de bitácora a ese equipo. El servicio `syslog` no tiene una manera de identificar equipos específicos de los que debería recibir mensajes, así que simplemente sostiene un gran guante de béisbol electrónico, y acepta cualquier cosa que venga en su camino.

El demonio `syslog` puede personalizarse de varias maneras diferentes, pero está un tanto anticuado en términos tanto de habilidades como de seguridad. A continuación veremos un enfoque más nuevo e incluso más configurable para el registro de eventos de sistema.

Véase también

- `man syslog`
- Brian Warshawsky



Centralice las bitácoras de sistema con seguridad

Proteja sus valiosos ficheros de bitácora de miradas indiscretas.

En el truco anterior, discutimos la configuración del demonio `syslog`. Tan útil e incluso tan necesario como es este servicio de registro de eventos, está, sin

embargo, comenzando a mostrar su edad. En respuesta a esto, una compañía llamada BalaBit ha dedicado tiempo y recursos a traernos la siguiente generación de syslog, syslog-ng, que trata muchos de los problemas que plagan el original. Las mejoras incluyen el uso de TCP en vez de UDP para comunicarse con equipos de bitácora remotos, y una interfaz mucho más configurable para las habilidades de registro de eventos de su sistema. Desde el punto de vista de seguridad, la implementación de TCP es un gran avance, que nos permite utilizar aplicaciones adicionales, tales como stunnel, para crear túneles codificados para proteger los contenidos de los ficheros de bitácora, según se envían al equipo central de bitácora. En este truco, examinamos tal despliegue.

Para empezar

Para implementar registro remoto de eventos codificado, necesitar descargar y compilar tres programas. Vamos a empezar con stunnel. Coja la última instancia del código fuente de <http://www.stunnel.org/download/source>. Una vez que tenga el fichero .tar, desempaquélo y navegue al recién creado directorio. Puede ahora seguir el procedimiento típico de instalación:

```
$ ./configure
$ make
# make install
```

Necesitará ahora coger los fuentes para syslog-ng y libol, una librería requerida por syslog-ng.

Puede descargarse cada uno de ellos de <http://www.balabit.com/downloads/syslog-ng/>. Desempaquete e instale libol primero, luego syslog-ng. La instalación de estas dos aplicaciones usa los tres pasos típicos anteriores.

Una vez que haya instalado con éxito stunnel, syslog-ng, y libol, necesitará crear certificados de codificación para todas las máquinas entre las cuales quiera transferir información de bitácora segura.

Crear sus certificados de codificación

Para transferir datos de bitácora con seguridad entre un equipo remoto y un equipo de bitácora central, la comunicación entre los dos debe ser codificada. Para poder utilizar la codificación con éxito, ambos equipos deben ser capaces de verificar sus identidades y compartir las claves de codificación usadas para leer y escribir los datos codificados. Esta información se proporciona por medio de certificados SSL, los cuales pueden ser otorgados por una tercera entidad o creados por usted mismo para usarlos en su organización. (Para más de lo que probable-

mente quiera saber sobre SSL y certificados, puede ver el "SSL HOWTO" que se encuentra en <http://www.tldp.org/HOWTO/SSL-Certificates-HOWTO/>.)

Llegados a este punto, debe crear múltiples certificados: uno para usarlo con el servidor central de bitácora, y uno para cada cliente que envíe información de bitácora al servidor.

Más adelante en esta sección, instalará el certificado de servidor en su servidor y distribuirá los certificados de cliente a los equipos para los cuales fueron creados.

El proceso de crear certificados varía ligeramente en base a la distribución Linux que esté usando. Para un sistema Red Hat, es como sigue:

```
# cd /usr/share/ssl/certs
# make syslog-ng-server.pem
# make syslog-ng-client.pem
```

Según se genera cada certificado, el *script* le hará varias preguntas sobre su ubicación, nombre de equipo, organización y dirección de correo electrónico. Una vez que haya respondido a todas las preguntas, se generarán sus certificados. Su siguiente paso es verificar que sólo el súper-usuario tiene acceso a ellas:

```
[root@aardvark certs]# ls -l *.pem
-rw----- 1 root root 2149 Aug 14 12:12 syslog-ng-client.pem
-rw----- 1 root root 2165 Aug 14 12:12 syslog-ng-server.pem
[root@aardvark certs]#
```

Hay una última cosa que necesitará hacer antes de que empiece a distribuir sus certificados: extraiga la sección "CERTIFICATE" de cada certificado que va a una máquina cliente y concatene las secciones extraídas en un solo fichero llamado syslog-ng-client.pem, el cual pondrá en su servidor junto con la clave del servidor. Los datos de la clave "CERTIFICATE" en un fichero de certificado son la información entre las siguientes dos líneas:

```
-----BEGIN CERTIFICATE-----
-----END CERTIFICATE-----
```

Copie el fichero syslog-ng-client.pem sobre el directorio /etc/stunnel en el servidor y sitúe una copia del propio certificado de cada cliente en el directorio /etc/stunnel de ese cliente.

Esto puede sonar un tanto complicado, así que vamos a resumir: todo lo que está haciendo aquí es extraer la sección "CERTIFICATE" del fichero de certificado de cada cliente, concatenando esa información en un amplio certificado de cliente que residirá en su servidor (junto con el certificado del servidor), y copiando luego los certificados individuales de cliente a los equipos para los que estaban propuestos.

Configurar stunnel

Ahora, en el lado servidor, edite su fichero `stunnel.conf` para que se vea como sigue:

```
cert = /etc/stunnel/syslog-ng-server.pem
CAfile = /etc/stunnel/syslog-ng-client.pem
verify = 3
[5140]
    accept = ip.de.su.servidor:5140
    connect = 127.0.0.1:514
```

Luego haga cambios parecidos al `stunnel.conf` en el lado cliente:

```
client = yes
cert = /etc/stunnel/syslog-ng-client.pem
CAfile = /etc/stunnel/syslog-ng-server.pem
verify = 3
[5140]
    accept = 127.0.0.1:514
    connect = ip.de.su.servidor:5140
```

Configurar syslog-ng

Una vez que se hayan hecho estos cambios, es hora de comenzar a crear su fichero `syslog-ng.conf`.

La sintaxis de este fichero tiene una empinada curva de aprendizaje y está mucho más allá del alcance de este truco, así que utilice lo que estoy a punto de mostrarle como punto de partida, y trabaje desde ahí. Mucho más detalle puede encontrarse online y en las páginas de manual.

En su servidor central de bitácora, añada lo siguiente a `/etc/syslog-ng/syslog-ng.conf`:

```
options { long_hostnames(off);
          sync(0);
          keep_hostname(yes);
          chain_hostnames(no); };
source src {unix-stream("/dev/log");
            pipe("/proc/kmsg");
            internal( );};
source stunnel {tcp(ip("127.0.0.1")
                    port(514)
                    max-connections(1));};
destination remoteclient {file("/var/log/remoteclient");};
destination dest {file("/var/log/messages");};
log {source(src); destination(dest);};
log {source(stunnel); destination(remoteclient);};
```

Luego, añada lo siguiente a su fichero `syslog-ng.conf` en cada cliente:

```
options {long_hostnames(off);
          sync(0);};
source src {unix-stream("/dev/log"); pipe("/proc/kmsg");
            internal( );};
destination dest {file("/var/log/messages");};
destination stunnel {tcp("127.0.0.1" port(514));};
log {source(src);destination(dest);};
log {source(src);destination(stunnel);};
```

Probar

Una vez que ha hecho todo esto, puede iniciar `stunnel` y `syslog-ng` para ver si todo está funcionando. Sin embargo, antes de que haga esto, asegúrese de que detiene el servicio `syslogd`. No querrá que los dos se pisen el uno al otro. Para probar si su registro remoto de eventos está funcionando, use el comando `logger`:

```
# logger Esto es una prueba
```

Luego, en su servidor de bitácora, busque (o haga `grep`) en `/var/log/messages` (o donde sea que tenga las bitácoras remotas) por "Esto es una prueba". Si obtiene una respuesta, enhorabuena, todo está funcionando bien. ¡Y ahora tiene registro remoto de eventos codificado!

¿Siguiente paso?

Aunque el registro remoto de eventos siempre ha sido un proceso útil e incluso necesario, enviar valiosa información de sistema sin codificar por el vacío siempre ha sido un riesgo de seguridad. Gracias a `syslog-ng` y `stunnel`, no tenemos que preocuparnos más por esto.

Además, la flexibilidad de `syslog-ng` ha crecido a pasos agigantados con respecto a lo que `syslogd` era capaz. Es verdaderamente la siguiente generación de los demonios de registro de eventos de sistema.

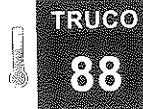
Sin embargo, esta flexibilidad tiene un precio, el fichero de configuración de `syslog-ng` es una bestia compleja. Aunque, si dedica un poco de tiempo a intentar conocerlo, encontrará que no es tan duro como parecía. Le puedo asegurar que la complejidad de la sintaxis es proporcional a su adaptabilidad una vez que lo entienda.

He listado a continuación algunos recursos, que puede consultar online, para obtener ayuda en la configuración de su instancia `syslog-ng`, para que satisfaga sus necesidades.

Véase también

- <http://www.balabit.hu/static/syslog-ng/reference/book1.html>
- <http://www.stunnel.org/examples/syslog-ng.html>

–Brian Warshawsky



TRUCO

88

Controle sistemas y servicios

Consolide *script* y mecanismos de monitorización de cosecha propia usando Nagios.

Monitorizar es una tarea clave para los administradores, tanto si está en un pequeño entorno de 50 a 100 servidores como gestionando muchos sitios globalmente con 5.000 servidores cada uno. En algún momento, intentar mantenerse al ritmo del crecimiento en el número de nuevos servicios y servidores desplegados, y reflejar los cambios a través de muchas soluciones de monitorización dispares, se vuelve un trabajo a tiempo completo!

Los administradores a menudo monitorizan no sólo la disponibilidad de un sistema (usando herramientas simples como `ping`), sino además la salud de los servicios, ejecutando en él los dispositivos de red que interconectan los sistemas, dispositivos periféricos como impresoras, copiadoras, fuentes de alimentación ininterrumpida (UPS, *Uninterruptible Power Supplies*), e incluso aire acondicionado y otros equipos. A menudo, estas herramientas llevan a cabo simples conexiones a servicios, y usan recopilación de datos SNMP y `rstatd` y dispositivos especializados de monitorización de entorno, para obtener una vista completa de los centros de datos.

Aunque hay soluciones de sobra para recopilar y agregar estos datos de alguna manera sensata, he encontrado que Nagios proporciona el perfecto equilibrio entre simplicidad y potencia. Nagios es una solución que satisface bastante bien los requisitos del entorno de computación de su mediana empresa, por razones como éstas:

- **Comprobación de dependencia:** Si todas sus impresoras están en un solo panel, y éste se viene abajo, ¿preferiría obtener una página de cada una de las 50 impresoras no disponibles, o una sola página diciendo "el panel de impresoras está fuera de servicio"? Nagios puede configurarse (o no) para seguir una ruta lógica, de tal manera que un dispositivo inalcanzable dispare la comprobación de los otros dispositivos de los cuales depende el primer fallo. Si una impresora está fuera de servicio, Nagios comprueba primero para asegurarse de que el panel de impresoras funciona, antes de notificar a nadie. Si el panel de impresoras está funcionan-

do y alguien está correteando desenchufando impresoras, tendrá un montón de páginas, pero si el panel está fuera de servicio será notificado del problema más grande no de sus consecuencias. Más aun, si ese panel de impresoras es inalcanzable, porque un *router* entre Nagios y él está apagado o no disponible, obtendrá este mensaje, lo cual puede ahorrarle algún tiempo de análisis y solución de errores y hace las páginas mucho más útiles e interesantes.

- **Programación de periodos de inactividad:** Cuando tiene un montón de herramientas monitorizando su entorno, o una sola herramienta que no le permite programar un periodo de inactividad, su buscaperonas se volverá loco según vaya apagando su entorno, y posiblemente de nuevo cuando lo encienda. Mezcle esto con una situación en la que no haya comprobación de dependencias, y pronto encontrará un grupo de administradores dando vueltas mientras sus buscaperonas yacen vibrando en los cajones de sus escritorios. Con Nagios puede programar periodos de inactividad y evitar problemas.
- **Notificación de recuperación:** Mucha gente utiliza soluciones de monitorización que hacen una simple comprobación de `ping`, que le dice si una máquina es inalcanzable. Sin embargo, si esto fue debido a un fallo temporal de energía, que hizo que el panel perdiera el control momentáneamente, y el agente nunca le informa de que la máquina volvió a estar disponible un minuto después, podría estar gastando mucho tiempo acercándose al sitio por un problema que se ha corregido ya por sí mismo. Nagios le notificará de las recuperaciones.

Un montón de soluciones no proporcionan estos beneficios. Añada soluciones que son difíciles de personalizar, no proporcionan comprobaciones para servicios y aplicaciones especializadas, y son difíciles de integrar con las pocas herramientas que podrían haber hecho ese trabajo bien, y tendrá grandes dolores de cabeza y una tendencia a la baja en la moral de sus administradores.

Nagios entra en escena

He encontrado que Nagios proporciona una manera extremadamente simple de tomar muchos de nuestros diferentes *script*, módulos de notificación, comprobadores de `ping`, y otras herramientas, y ponerlos todos bajo su paraguas como módulos adicionales sin tener que cambiar prácticamente nada. De hecho, la funcionalidad de monitorización que viene pre-configurada con Nagios se maneja completamente por medio de programas Perl, C o de intérprete de comandos a los que Nagios llama en segundo plano. La barrera de entrada era realmente tan baja que en un día tenía una configuración muy básica de Nagios

trabajando, con una interfaz Web, notificaciones por correo electrónico, comprobaciones de servicios y sistemas básicos funcionando. Al final de la semana, había configurado Nagios para ser un poco más discriminante con sus notificaciones (por ejemplo, notificar sólo al administrador de base de datos si el servicio de base de datos deja de estar disponible, pero sólo a los administradores Sun si el servidor de base de datos se viene abajo). Tenía configuradas además dependencias de equipos y servicios, y le había contado nuestros próximos dos periodos de inactividad programados. Ni siquiera encontré módulos adicionales existentes para Nagios que permitieran la jubilación de un par de nuestros *script* de cosecha propia para monitorizar cosas como el archivador NetApp y una base de datos MySQL. ¡Las cosas miraban hacia arriba!

Más aun, la interfaz Web de Nagios, aunque mantiene estadísticas lo bastante útiles para ayudar a señalar cuándo comenzó un problema, o predecir sus necesidades de disco en un servidor de ficheros para el próximo año, puede también integrarse fácilmente con herramientas estándar como MRTG o Cacti.

Si quiere ponerse realmente duro, puede también usar Nagios para recopilar capturas SNMP, o volverse completamente distribuido usando agentes Nagios, en vez de un mecanismo central de sondeo, por toda su sala de máquinas.

La única desventaja de Nagios que he encontrado hasta ahora es que, mientras que la configuración es bastante sencilla, no hay ninguna interfaz gráfica de configuración o automatización, así que todo tiene que hacerse a mano (lo que puede ser un tanto engorroso y ocupar mucho tiempo). Sin embargo, la recompensa está ahí, así que vamos a ver algunos detalles de configuración. Cubriré sólo la configuración más básica, ipuesto que documentar un despliegue completo de Nagios podría constituir otro libro en sí mismo!

Primero necesitará instalar Nagios, bien usando su sistema de gestión de paquetes (para una instalación binaria) o yendo a <http://www.nagios.org> para co-ger los fuentes e instalarlos de acuerdo con la abundante documentación.

Equipos, servicios y contactos, ¡oh, Dios mío!

Empezaremos por lo más simple. Su sala de máquinas consiste en equipos. Estos equipos ejecutan servicios. Si un equipo o un servicio que ejecuta deja de estar disponible, querrá que Nagios se lo notifique a un contacto. Así que, lo primero que hay que hacer es informarle a Nagios sobre estas entidades. Para hacer esto, añadimos entradas en los ficheros `hosts.cfg`, `services.cfg`, y `contacts.cfg`.

Estos ficheros pueden encontrarse bajo `/etc/nagios` si su instalación estaba pre-configurada (como una instalación de un sistema SUSE o Red Hat), o donde sea que le diga que ponga los ficheros de configuración durante la instalación desde los fuentes (`/usr/local/etc/nagios`, por defecto).

He aquí una simple entrada de `hosts.cfg` que le da a Nagios alguna información básica sobre un equipo:

```
define host{
    use                generic-host
    host_name          newhotness
    alias              Jonesy's Desktop
    address            128.112.9.52
    parents            myswitch
}
```

Se dará cuenta de que toda esta información es específica de mi máquina de escritorio. No hay nada en ella sobre cómo comprobar la disponibilidad del equipo, cuándo hacerlo, o cualquier otra cosa. Esto es porque Nagios le permite configurar una entrada borrador "host" para contener toda esa información (ya que es probable que sea idéntica para números más grandes de equipos). El borrador usado en la entrada anterior se llama "generic-host", y puede encontrarse cerca del principio del fichero `hosts.cfg`. El borrador "generic-host" se parece a esto:

```
define host{
    name                generic-host
    notifications_enabled 1
    event_handler_enabled 1
    flap_detection_enabled 1
    process_perf_data    1
    notification_interval 360
    notification_period   24x7
    notification_options  d,u,r
    contact_groups        sysstaff
    check_command          check-host-alive
    max_check_attempts    10
    retain_status_information 1
    retain_nonstatus_information 1
    register              0
}
```

Esta sola entrada tiene todo el levantamiento pesado para el resto de dispositivos que hacen referencia a esta plantilla. Todos ellos se revisarán durante el comando de comprobación "check-host-alive" el cual es un comando `ping` en un *script*. Según el valor de la clave "notification_period", serán monitorizados 24 horas al día, 7 días a la semana. La línea "notification_options" dice que envíe notificaciones si el estado de la máquina es apagada (d), inalcanzable (u), o recuperada (r). La opción "flap_detection_enabled" se activa también aquí. Esta es una característica de Nagios que busca ahorrarle el obtener páginas de servicios o equipos que cambian de estado frecuentemente debido a aberraciones temporales en la conectividad de red, tiempos de respuesta de equipos, o servicios que se reinician a propósito para recoger actualizaciones automatizadas. Tiene que

admitirlo, iponer todos estos detalles en una sola entrada es mejor que ponerlo en cada entrada "host"! Vamos a pasar a los servicios. Una entrada típica de `services.cfg` se parece a lo siguiente:

```
define service{
    use                generic-service
    host_name           ftpserver
    service_description FTP
    is_volatile         0
    check_period        24x7
    max_check_attempts  3
    normal_check_interval 5
    retry_check_interval 1
    contact_groups      sysstaff
    notification_interval 120
    notification_period 24x7
    notification_options w,u,c,r
    check_command        check_ftp
}
```

Esta es la entrada para mi servidor FTP. De nuevo, incluye sólo la información específica a este servidor; todo el resto de información viene de un borrador llamado "generic-service", cuyos valores se aplican a todos los servicios cuyas entradas hacen referencia a él usando la directiva "use generic-service". Fíjese que yo uso un comando de comprobación específico de servicio llamado "check_ftp". El comando "check_ftp" es simplemente un *script* de intérprete de comandos que intenta hacer una conexión al servicio FTP en "ftpserver".

Sin duda se habrá dado cuenta de que tanto la comprobación "host" como la "service" mandan correo a "sysstaff" si hay algún problema. Pero ¿qué es "sysstaff"? en realidad no es un alias de correo (aunque puede usar uno si lo desea). En vez de esto, está configurado dentro del propio Nagios, en los ficheros `contacts.cfg` y `contactgroups.cfg`. ¡Echemos un vistazo! He aquí una entrada para un contacto del fichero `contacts.cfg`:

```
define contact{
    contact_name        jonesy
    alias               Jonesy
    service_notification_period 24x7
    host_notification_period 24x7
    service_notification_options c,r
    host_notification_options d,r
    service_notification_commands notify-by-email
    host_notification_commands host-notify-by-email
    email               jonesy@linuxlaboratory.org
}
```

Esta es mi entrada de contacto. Dice que sea notificado de cualquier fallo de equipo o servicio 24 horas al día, 7 días a la semana. Sin embargo, he trucado mi

entrada de tal manera que, en vez de ser notificado a cada cambio de estado, sea notificado sólo cuando los servicios ("service_notification_options") son críticos (c) y cuando se recuperan (r), y cuando los equipos ("host_notification_options") están apagados (d) y cuando se recuperan (r). Hay una entrada como ésta para todos los que recibirán notificaciones de Nagios sobre el estado de servicio o equipo. Una vez que todos los contactos están definidos, puede agruparlos juntos en forma de grupos específicos de Nagios en `contactgroups.cfg`. He aquí un ejemplo:

```
define contactgroup{
    contactgroup_name    sysstaff
    alias                The Systems Guys
    members               jonesy,bill,joe
}
```

No ha sido tan duro ¿verdad? Simplemente recuerde que cualquiera en el grupo de contacto debe definirse primero como un contacto en `contacts.cfg`.

Llegados a este punto usted tiene sólo una configuración muy simple, pero es suficiente para encender Nagios y hacerle monitorizar los equipos y servicios que ha definido, y notificar a aquellos que ha definido como contactos. Antes de hacer esto, debería ejecutar el siguiente comando para hacer una comprobación de sintaxis:

```
$ nagios -v /etc/nagios/nagios.cfg
```

Esto ejecuta Nagios en modo "verificación", y le hemos pasado como entrada el fichero principal de configuración de Nagios, el cual contiene una línea por cada otro fichero de configuración en uso. Si hay algún problema, Nagios escupirá información de sobra para que encuentre, mire y arregle el problema. En estas etapas tempranas, los problemas más comunes probablemente estarán relacionados con los ficheros de configuración definidos en `nagios.cfg` que todavía no se usan. Por ejemplo, ya que no hemos usado el fichero de configuración de dependencias, querrá comentar cualquier referencia a él en `nagios.cfg`.

Si no ha recibido errores, está en buena forma. Podría ver "advertencias" que le apuntan posibles problemas durante la verificación de configuración, pero en muchos casos estas advertencias son para cosas intencionadas, como contactos que no se han asignado a un grupo de contactos (lo cual no está requerido y no es siempre deseable). Una vez que haya verificado que las advertencias son inofensivas, o arreglado cualquier problema existente y verificado las cosas de nuevo, puede encender Nagios y comenzar a recibir notificaciones vía correo electrónico sobre los equipos y servicios que ha configurado.

Véase también

- <http://www.nagios.org>

Rescate, recuperación y reparación de sistema

Trucos 89 a 100



Ningún sistema informático sobrevive al contacto con el entorno. La calidad de sus habilidades como administrador de sistemas no puede evitar que el hardware falle, sólo puede ayudarle a recuperarse lo mejor posible de discos o controladoras erróneos, y otras calamidades que ahogan su bandeja de entrada con peticiones de soporte (si es que alguien puede mandar algún correo) y concluyen en largas colas de irritados usuarios esperando enfrente de su oficina, como compradores intentando devolver regalos rotos después de Navidad. "Hay copias seguridad de hace 10 minutos, ¿verdad?" les oírás gritar.

La recuperación de datos es más crítica hoy en día que nunca, ya que la pérdida de un solo disco o sistema de ficheros puede significar cientos de gigabytes de datos arruinados. Pero no se preocupe, no todo está necesariamente perdido. Puede salir de muchos fallos de sistema con su sombrero de mago completamente intacto, y quizás incluso ostentando unas cuentas estrella nuevas.

Los trucos en este capítulo le proporcionan un surtido de consejos ganados a pulso sobre cómo lidiar con sistemas que de repente no pueden arrancar por su cuenta, cómo poner en línea sistemas de ficheros tercios que no se dejan acceder o desmontar, e incluso cómo recuperar ficheros o datos borrados de discos duros erróneos. Algunas de las técnicas en este capítulo han recuperado datos de sistemas Linux cuyos discos parecían más bloques de madera, que avanzados dispositivos de almacenamiento.

Como interesante paseo por la recuperación y restauración, este capítulo además incluye trucos sobre cómo eliminar ficheros permanentemente y limpiar discos de tal manera que puedan ser desechados con seguridad, sin donar sus secretos corporativos a la competencia o su colección de música a la SESGAE. Sin

embargo paramos en seco describiendo cómo limpiar físicamente discos duros (por ejemplo, usando un martillo), la mayoría de la gente puede hacerlo (y se nos veía demasiado alegres en las imágenes que entregamos).

TRUCO

89

Resuelva problemas comunes de inicio y arranque

Maliciosos maleantes informáticos, sobre-entusiastas actualizaciones de software, o simples fallos de hardware pueden impedirle el reiniciar o acceder a un sistema. Lo primero que hay que hacer es relajarse e intentar unos cuantos consejos y trucos estándar para poner de nuevo en pie a su achacoso equipo.

Tarde o temprano, normalmente justo antes de que uno de sus usuarios esté a punto de enviar su tesis, o de que tenga una reunión para presentar el documento de estrategia de IT en el que ha estado trabajando durante semanas, encontrará que intentar arrancar uno de sus sistemas acaba en toda una variedad de enigmáticos mensajes de error, un cursor intermitente, o una interfaz gráfica de usuario que no aceptará ninguna entrada por teclado o ratón. En otras palabras, en absoluto el inicio de sesión estándar de Linux al que está acostumbrado. Por supuesto, tiene copias de seguridad de sus ficheros críticos en alguna otra parte, pero si su sistema no está ejecutando por una u otra razón, las copias de seguridad son tan sólo una distante manta de seguridad. Lo más seguro es que sus datos probablemente estén todavía presentes en el equipo antiguamente conocido como "su máquina de escritorio", pero simplemente no puede arrancar el equipo para llegar a ellos. ¿Qué se supone que debe hacer?

Dependiendo de los tipos de errores que esté viendo, podría necesitar cualquier cosa, desde un curso intensivo en configuración de BIOS, un Doctorado en el uso de `fsck`, y sus amigos, o alguna manera de arrancar su sistema y acceder a sus datos rápidamente. Este truco discute algunos de los consejos y trucos estándar para intentar hacer que su máquina ejecute por su cuenta. Si los consejos de este truco no son suficientes, vea el truco siguiente para el gran martillo, que consiste en crear un CD de arranque que contenga una distribución Linux que proporcione las herramientas que necesite para reparar un equipo Linux con achaques. Puede aplicar entonces las herramientas proporcionadas por ese CD para reparar sus sistemas de ficheros, recuperar particiones, y llevar a cabo los otros trucos listados al final de este truco, y que le permitirán tener su sistema de vuelta y arrancando por su cuenta.

Compruebe la configuración de la BIOS

Si su sistema no arranca en absoluto, lo primero que debe comprobar es si está realmente encontrando el dispositivo desde el cual usted espera que arranque. Si

ha añadido recientemente un disco a su sistema o cambiado su configuración hardware de alguna manera, hay posibilidades de que simplemente sus valores de BIOS no sean los correctos. Por ejemplo, yo tengo un servidor a 64 bit con un surtido de unidades extraíbles que arranca de un disco interno. Por alguna razón, cada vez que añado, elimino, o cambio una de las unidades extraíbles, la BIOS olvida que se supone que debe arrancar de una unidad SATA interna, e insiste en intentar arrancar de uno de mis ficheros de música o de uno de los discos que contienen directorios de usuario.

Los síntomas estándar de un sistema que se siente confundido sobre sus valores de arranque son un cursor parpadeante, después de que el sistema ha intentado iniciar el proceso de arranque, o un mensaje diciendo algo como "Dispositivo de arranque no encontrado" (*No bootable devices found*). Para asegurarse de que su sistema está realmente intentando arrancar del dispositivo correcto, tendrá que investigar sus valores de BIOS (*Basic Input/Output System*, Sistema básico de Entrada/Salida).

En muchos sistemas, o bien hay un "pantallazo" de arranque que oculta el comando necesario para entrar en la BIOS, o la pantalla se enciende una vez que esa información se ha mostrado. La mayoría de los sistemas modernos le permiten acceder a la configuración de su BIOS pulsando la tecla **Supr** (la que está con el mismo grupo de teclas que **Inicio**, **Fin**, **Av Pág**, y **Re Pág**) tan pronto como el sistema se enciende.

El sistema todavía intentará llevar a cabo algunas comprobaciones iniciales, pero después mostrará la pantalla de configuración de BIOS. Si pulsar **Supr** no proporciona acceso a la BIOS de su sistema, otras teclas/combinaciones de teclas populares para intentar (en orden) son **F2**, **F1**, **F3**, **F10**, **Esc**, **Control-Alt-Esc**, **Control-Alt-Insert**, y **Control-Alt-S**. Una de estas debería darle acceso a la BIOS de su sistema, aunque intentarlas todas puede resultar tedioso y consumir mucho tiempo.

La mayoría de los equipos x86 modernos incluye uno de entre un pequeño número de diferentes tipos de BIOS. Dos de los tipos de BIOS más populares son las diferentes pantallas de Award BIOS mostradas en las figuras 10.1 y 10.2.

En la BIOS mostrada en la figura 10.1, la configuración de arranque se almacena en la pantalla **Advanced Settings**, a la cual puede navegar usando la tecla **Flecha abajo**. Pulse **Intro** para mostrar esta pantalla una vez que su nombre esté resaltado.

En la pantalla **Advanced Settings**, use la tecla **Flecha abajo** para navegar a la entrada **First Boot Device**, y pulse **Intro** para mostrar sus opciones. Use las teclas de flecha (o teclas del cursor) para seleccionar la entrada correspondiente a su unidad de arranque real y pulse **Intro**. Puede pulsar entonces la tecla **Esc** para salir de esta pantalla, y **F10** para guardar los nuevos valores, salir de la pantalla de configuración de BIOS, y reiniciar.

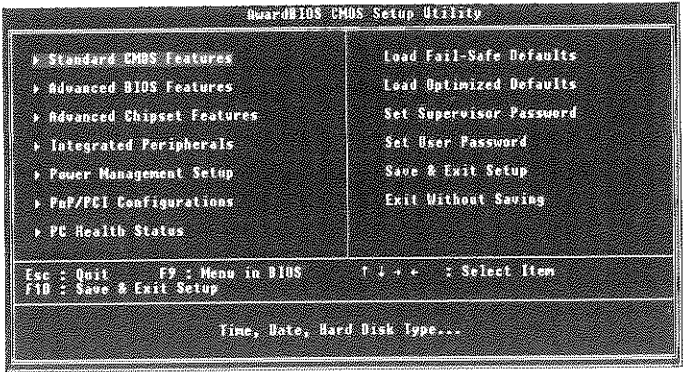


Figura 10.1. BIOS Award con menús verticales.

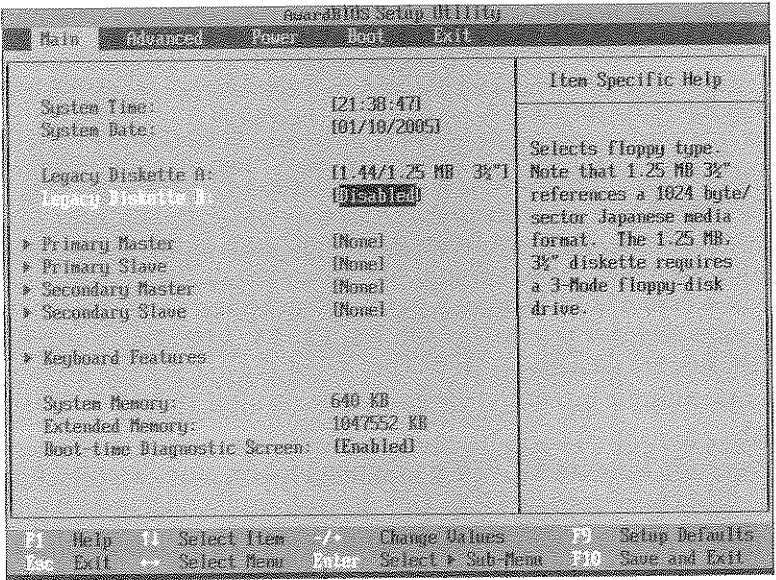


Figura 10.2. BIOS Award con menús horizontales.

En la BIOS mostrada en la figura 10.2, la configuración de arranque se almacena en la pantalla **Boot**, a la que puede navegar usando la tecla **Flecha derecha**. Pulse **Intro** para mostrar esta pantalla una vez que su nombre esté resaltado. En la pantalla **Boot**, use la tecla **Flecha abajo** para navegar a la entrada **Hard Drive**, y pulse **Intro** para mostrar una lista de unidades disponibles. Puede entonces remarcar la unidad correcta usando las teclas de flecha y pulsar **Intro** para seleccionarla. Una vez que se ha seleccionado el disco duro correcto, puede

usar el símbolo más (+) para mover esa entrada para que sea el primer dispositivo de arranque, y pulsar entonces **F10** para guardar los nuevos valores, salir de la pantalla de configuración de BIOS, y reiniciar.

Si la configuración de arranque de BIOS para el sistema en el que está teniendo problemas parece estar correcta, probablemente este no es el origen de su problema, y debería cambiar estos valores sólo como último recurso. Cambiar demasiadas variables a la vez es una reacción normal a un sistema que no arranca, pero raras veces es la correcta.

Dependiendo de los tipos y la configuración de las unidades en su sistema, podría tener que experimentar con la configuración del dispositivo de arranque en la BIOS antes de que su sistema arranque correctamente. Si la BIOS no encuentra una unidad que usted sabe que está físicamente presente, la unidad puede haber fallado, en cuyo caso no hay mucho que pueda hacer sin técnicas de recuperación hardware específicas de la unidad, que están fuera del alcance de este libro.

Si la BIOS encuentra la unidad pero no puede leer la tabla de particiones del disco usando el CD de rescate, véase "Recupere particiones perdidas" más adelante en este capítulo para obtener sugerencias sobre cómo crear de nuevo la tabla de particiones. Si la tabla de particiones está bien, pero no puede montar o reparar una o más particiones, consulte "Recupere datos de discos averiados" (también en este capítulo), para sugerencias sobre recuperar datos del disco.

Solucionar problemas de nivel de ejecución o del sistema X Window

La mayoría de las distribuciones Linux de hoy en día proporcionan algún tipo de servicio gratuito de actualización online. Esto es estupendo para mantener su sistema actualizado con el software más nuevo, más inteligente y más reluciente disponible para su distribución. Sin embargo, si obtiene una actualización falsa, puede incapacitar su sistema, y algunas de las actualizaciones falsas que más he visto son actualizaciones del sistema X Window (para X.org, en el pasado XFree86).

Desgraciadamente, la solución que corrige el problema de otro puede poner a su interfaz gráfica de rodillas, sin aceptar entrada de ratón ni de teclado. Si no puede hacer que su sistema X Window se muestre o responda a la entrada de ratón o teclado, pruebe lo siguiente:

- Cambie a otra consola virtual pulsando **Control-Alt-F1** o **Control-Alt-F2**. Inicie sesión ahí, y edite `/etc/inittab` para iniciar en otro nivel de ejecu-

ción hasta que pueda corregir el problema. La línea específica de `inittab` que está buscando es:

```
id:5:initdefault:
```

- Necesita cambiar el 5 a otro nivel de ejecución (normalmente 3). Algunas distribuciones, como Ubuntu y Gentoo, simplemente requieren detener el gestor de arranque, lo cual normalmente significa eliminar el servicio `xdm`, `gdm`, o `kdm` del proceso de arranque. Una vez que haya hecho esto, reinicie.
- Vaya a otra máquina y haga SSH o telnet al sistema donde está teniendo problemas. Una vez iniciada sesión, haga `su` y edite `/etc/inittab` para iniciar en otro nivel de ejecución (normalmente 3) hasta que pueda corregir el problema. Reinicie.
- Si no puede hacer ninguna de las sugerencias anteriores (por ejemplo, si ninguna otra máquina está a mano o ha desactivado las consolas y `getty` virtuales para optimizar el rendimiento), use la información proporcionada más tarde en este truco, para reiniciar en modo mono-usuario. Puede editar entonces `/etc/inittab` para iniciar en otro nivel de ejecución hasta que pueda corregir el problema. Reinicie.

Una vez que esté en un nivel de ejecución no gráfico, puede llevar a cabo tareas de reparación tales como ejecutar utilidades de reparación de sistemas de ficheros, reparar la configuración de su sistema X Window, etc.

Regenerar un fichero de configuración por defecto de X Window

Si puede arrancar su sistema con éxito en un nivel de ejecución no gráfico pero no puede iniciar el sistema X Window automática o manualmente, su fichero de configuración puede simplemente estar agitado (en términos técnicos). Si esto ocurrió porque instaló una versión actualizada del sistema X Window, porque su sistema de ficheros raíz tuvo un problema y el fichero fue borrado, o porque ha "afinado" sus ficheros de configuración hasta un punto en el que X ya no inicia, puede empezar desde cero generando un fichero de configuración por defecto de X Window, que pueda usar entonces como punto de partida para corregir los problemas que está viendo. Las implementaciones X.org y XFree86 del sistema X Window proporcionan una opción `"-configure"` que le permite generar un fichero de configuración por defecto. Dependiendo de qué servidor X Window tenga en su sistema Linux, inicie sesión como súper-usuario, y ejecute uno de los siguientes comandos para generar un fichero de configuración por defecto:

```
# Xorg -configure
# XFree86 -configure
```

Estos comandos hacen que el servidor X pruebe su hardware de gráficos y genere un fichero de configuración por defecto de X Window en el directorio `/root` llamado `xorg.conf.new` o `XF86Config.new`. Puede probar entonces este fichero de configuración genérico iniciando su servidor X con el siguiente comando:

```
# X -config /root/filename
```

Si el servidor X inicia correctamente, reemplace su fichero de configuración X por defecto con el nuevo y (tras crear una copia de seguridad) reanude el uso normal o el afinado. Un fallo común es que X no inicie porque no puede detectar su ratón. Si esto ocurre, compruebe la sección "InputDevice" del fichero de configuración que creó para el valor de la opción "Device".

Si éste es simplemente `/dev/mouse`, intente cambiarlo por `/dev/input/mice` y reiniciar X, usando el fichero de configuración actualizado.



Si está teniendo problemas iniciando o configurando x en general, su tarjeta de video puede que use un conjunto de chips que todavía no está soportado por la versión del sistema X Window que está usando. Si esto ocurre, puede intentar usar un menor denominador común como una alternativa. VESA (Video Electronic Standards Association, Asociación de Estándares Electrónicos de Video) está soportado por la mayoría de las tarjetas y debería hacer funcionar X a menores resoluciones en casi cualquier sistema con habilidades gráficas. Para usar VESA, simplemente establezca la línea "Driver" en su sección "Device" para que sea "vesa".

Arrancar en modo mono-usuario

Si está teniendo problemas arrancando en un nivel de ejecución específico, podría necesitar arrancar en modo mono-usuario para poder reparar su sistema. Esto puede ocurrir por un gran número de razones, más comúnmente por problemas de consistencia de sistemas de ficheros, pero además por cosas como el fallo de alguno de los *script* de bajo nivel de inicialización de sistema.

Si está usando el cargador de arranque GRUB, pulse cualquier tecla para interrumpir el proceso de arranque estándar de GRUB, use las teclas de flecha para seleccionar el núcleo que quiere arrancar, y pulse la tecla E para editar las opciones de arranque para ese núcleo. Seleccione la línea que contiene las opciones reales de arranque (normalmente la primera línea), pulse E de nuevo para editar esa línea de comandos, y añada el comando "single" al final de la línea de comandos. Puede ahora pulsar B para arrancar con estas opciones de arranque, y su sistema pasará por el proceso estándar de arranque pero acabará bien en

una sesión de intérprete de comandos de súper-usuario, o pidiéndole su contraseña de súper-usuario antes de iniciar el intérprete.

Si todavía está usando el cargador de arranque LILO, puede hacer lo mismo introduciendo el nombre de la estrofa de arranque que quiere arrancar (normalmente "linux"), seguida de un espacio y la directiva "-s". De nuevo, debería obtener una sesión de intérprete de comandos de súper-usuario o una petición de contraseña de súper-usuario en pocos segundos.

Si está teniendo problemas iniciando un intérprete de comandos mono-usuario, puede haber todavía algún problema en algún aspecto de bajo nivel de su proceso de arranque, o (¡ups!) puede haber olvidado o ser incapaz de facilitar la contraseña de súper-usuario. En este caso, consulte "Sáltese la secuencia estándar de init para hacer reparaciones rápidas" (más adelante en este capítulo) para ver una manera rápida de saltarse el proceso `/sbin/init` e iniciar un intérprete de comandos inmediatamente.

Resolver problemas de consistencia de sistemas de ficheros

Cuando un sistema no arranca porque alega que una de sus particiones, o más, es inconsistente y necesita por tanto ser reparada, está de suerte. Es difícil ver la corrupción de disco como algo bueno, pero supera a alguna de las alternativas. Como mínimo su sistema encontró el sector de arranque, arrancó de la unidad correcta, y llegó al punto en el que encontró suficientes aplicaciones para intentar comprobar sus sistemas de ficheros.

Uno de los problemas más comunes al arrancar un sistema es resolver los problemas de consistencia de sistemas de ficheros encontrados durante el arranque. Cuando apaga un sistema de manera normal, el sistema desmonta automáticamente todos sus sistemas de ficheros, marcándolos como "limpios", de tal manera que puede reconocer que están en un estado consistente cuando arranque el sistema la siguiente vez. Si un sistema falla por alguna razón, los sistemas de ficheros no se marcan como limpios, y por tanto debe comprobarse su consistencia y corrección la próxima vez que arranque el sistema. Cada diferente tipo de sistemas de ficheros tiene su propia verificación de consistencia y sus propias utilidades de reparación. En la mayoría de los casos, su sistema las ejecutará automáticamente como parte del proceso de arranque, y corregirá cualquier problema de consistencia de sistema de ficheros que estas utilidades detecten. A veces, sin embargo, no tiene tanta suerte, y tendrá que ejecutar estas utilidades manualmente para corregir problemas serios de sistemas de ficheros. De manera similar, si está usando el sistema de ficheros XFS, todo lo que la utilidad de reparación *vanilla* hace es devolver `TRUE`, ya que espera que el sistema de ficheros XFS pueda repetir correctamente el diario (*journal*) y arreglar cualquier problema como parte del proceso de montaje. Si no es este el caso, puede

encontrarse en modo mono-usuario si las particiones de arranque y raíz están OK. Si no, vea el truco siguiente para informarse sobre cómo obtener un CD de rescate, porque lo va a necesitar.

Los detalles sobre cómo ejecutar manualmente la utilidad de comprobación de consistencia de cada sistema de ficheros están fuera del alcance de este truco, pero como mínimo es útil saber qué utilidad usar si tiene que reparar manualmente un sistema de ficheros. La tabla 10.1 muestra las utilidades de consistencia de sistemas de ficheros a utilizar para reparar manualmente varios tipos de sistemas de ficheros Linux.

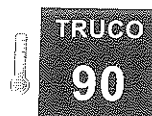
Tabla 10.1. Utilidades de reparación para diferentes sistemas de ficheros Linux.

Filesystem	Utility
<i>ext2, ext3</i>	<i>e2fsck</i>
<i>JFS</i>	<i>jfs_fsck</i>
<i>reiserfs</i>	<i>reiserfsck</i>
<i>XFS</i>	<i>xfs_check, xfs_repair</i>

En el caso del sistema de ficheros XFS, `xfs_check` es un *script* de intérprete de comandos, que simplemente identifica problemas en un sistema de ficheros específico, con los cuales tiene entonces que aplicar la utilidad `xfs_repair` para corregirlos.

Véase también

- Página Web de RIP: <http://www.tux.org/pub/people/kent-robotti/looplunix/rip/>



TRUCO
90

¡Rescátame!

¿Así que ha probado todos los consejos y trucos para conseguir que su equipo arranque por su cuenta y nada ha funcionado? En ese caso, un sistema de arranque Linux en un CD puede ser su nuevo mejor amigo.

Un fallo hardware, la corrupción de un sistema de ficheros, actualizaciones sobre-entusiastas, y un significativo retoque del proceso de inicio de su sistema están entre las cosas que pueden hacer que su sistema no pueda arrancar con éxito. Asumiendo que ha llegado a este punto y las sugerencias en el truco ante-

rior no funcionaron, su siguiente mejor alternativa es descargar, grabar, y arrancar de lo que se conoce como un "disco de rescate".



Siempre es una buena idea tener un disco de arranque de rescate a mano. Descargue y grabe uno antes de tener problemas, de tal manera que tenga uno para usar si alguna vez lo necesita.

Un disco de rescate es una pequeña distribución Linux que arranca y ejecuta desde CD y proporciona el núcleo de sistema y las utilidades de sistema operativo que necesita para acceder a su hardware, así como las herramientas que necesita para resolver problemas con la interacción entre dicho hardware y el sistema de escritorio o servidor que está intentando arrancar. Las cosas que un disco de rescate debe proporcionar caen dentro de cuatro categorías generales:

- Un núcleo de sistema y controladores para los dispositivos de almacenamiento conectados a su sistema y, preferiblemente, como mínimo una de las interfaces de red disponibles en ese sistema.
- Utilidades de reparación de disco para varios tipos de sistemas de ficheros, incluyendo utilidades de gestión de volúmenes lógicos (LVM).
- Utilidades de sistema tales como `mount` que le permitan acceder a los datos de los sistemas de ficheros en la máquina achacosa, herramientas de arranque como GRUB que le permitan verificar (y opcionalmente actualizar) el proceso de arranque del sistema, etc. Esto a menudo incluye las herramientas usadas para recuperarse de problemas de sistema, como `gpart` y `ddrescue`.
- Utilidades estándar, como un editor de texto para corregir y actualizar los ficheros de texto usados por el sistema durante el proceso de arranque (tales como `/etc/inittab`), los ficheros de configuración usados por varios servicios, y los `script` de inicio de sistema en el directorio `/etc/rc.d` o `/etc/init.d` (dependiendo de su distribución).

Aunque hay por ahí discos de rescate de sobra, incluyendo muchas distribuciones Linux gráficas "Live-CD", mi favorito personal durante años ha sido el disco RIP (*Recovery Is Possible!*, ¡Recuperar es posible!) de Kent Robotti, disponible en <http://www.tux.org/pub/people/kent-robotti/looplinux/rip/>. Este es un disco de rescate relativamente pequeño (25 MB) que no ofrece una interfaz gráfica, pero sí proporciona un juego completo de utilidades de reparación de sistemas de ficheros actualizadas para ext2, ext3, JFS, reiserfs, reiser4, y XFS, así como las utilidades LVM2 para montar y gestionar volúmenes lógicos. Como disco de rescate no gráfico, está orientado a administradores de sistemas experimentados

que se sientan cómodos con la línea de comandos, que es lo que debería ser si intenta rescatar o recuperar datos de un sistema enfermo.

Descargar y grabar el disco de rescate

Las dos imágenes ISO en la página de RIP difieren en términos del gestor de arranque que usan: una usa GRUB, y la otra usa el gestor de arranque estándar ISOLINUX. Yo prefiero usar la última porque es más simple. Así que siempre tomo el fichero `RIP-15.isolinux.iso`, que es una imagen que puede grabar directamente en un CD y usarla para arrancar su sistema.

La utilidad Linux estándar de grabación de CD se llama `cdrecord`. Anteriormente al núcleo 2.6 de Linux usar una copiadora de CD IDE con `cdrecord` requería el uso de un módulo de núcleo adicional que proporcionara la emulación SCSI para IDE, ya que `cdrecord` esperaba identificadores SCSI al especificar el dispositivo de destino de la salida. Con el núcleo 2.6, las utilidades de grabación de CD pueden usar unidades de CD ATA directamente, sin ningún módulo especial.

Una vez que ha tomado el fichero, necesitará identificar la(s) grabadora(s) de CD. Para hacer esto, haga su a súper-usuario y ejecute el comando `cdrecord -scanbus`. Esto hace que `cdrecord` pruebe el sistema buscando dispositivos apropiados y muestre la información que necesitará suministrar para poder escribir en ellos. He aquí un ejemplo:

```
# cdrecord -scanbus
Cdrecord 2.0 (i686-pc-linux-gnu) Copyright (C) 1995-2002 J#rg Schilling
Linux sg driver version: 3.1.24
Using libscg version 'schily-0.7'
scsibus0:
 0,0,0 0) 'TOSHIBA ' 'DVD-ROM SD-R1202' '1026' Removable CD-ROM
 0,1,0 1) *
 0,2,0 2) *
 0,3,0 3) *
 0,4,0 4) *
 0,5,0 5) *
 0,6,0 6) *
 0,7,0 7) *
```

Una vez que ha identificado el dispositivo asociado con su copiadora de CD, grabe la imagen en un CD-ROM virgen usando un comando como el siguiente:

```
# cdrecord -v dev=0,0,0 speed=4 RIP-15.isolinux.iso
```

Este comando producirá una salida muy detallada (debido al uso de la opción "-v") y esperará nueve segundos antes de realmente comenzar a escribir en el disco, por si acaso cambia de idea. Una vez que comienza la escritura, el coman-

do `cdrecord` le muestra una línea de estado que continúa actualizándose hasta que se ha escrito el fichero completo en el CD.

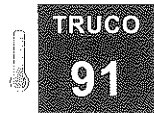
Usar el CD de rescate

Una vez que ha creado el CD de rescate, sólo necesita ponerlo en el sistema enfermo y reiniciar. Si su sistema no está configurado para arrancar desde la unidad de CD antes de arrancar de una partición de disco duro, puede necesitar cambiar la secuencia de arranque de su sistema en la configuración de BIOS para poder conseguir que el sistema arranque desde el CD. Una vez que ha arrancado desde el CD de rescate, puede llevar a cabo tareas como las siguientes, rápida y fácilmente:

- Ejecutar comandos estándar de reparación de sistema para reparar la consistencia del sistema de ficheros.
- Configurar la interfaz de red de su sistema de tal manera que pueda hacerlo funcionar en su red.
- Crear archivos de ficheros y directorios críticos y transferirlos a otros sistemas usando la utilidad `ncftp` proporcionada en el disco de rescate.
- Corregir otros problemas de arranque.

Véase también

- Página Web de RIP: <http://www.tux.org/pub/people/kent-robotti/looplinux/rip/>



TRUCO
91

Sáltese la secuencia estándar de init para hacer reparaciones rápidas

Llegue lo más profundo que se puede al resolver problemas de inicio.

Si está teniendo problemas arrancando un sistema en modo mono-usuario, los gestores de arranque LILO y GRUB proporcionan un atajo estupendo para ayudarle a obtener un marcador de intérprete de comandos en un sistema achacoso. Este truco es especialmente útil si su contraseña o su fichero `shadow` están dañados o desaparecidos, o, Dios no lo quiera, en realidad ha olvidado la contraseña de súper-usuario en uno de sus sistemas.

Por defecto, los sistemas Linux usan el proceso `/sbin/init` para iniciar otros procesos, incluyendo el intérprete de comandos del súper-usuario que obtiene

cuando arranca el sistema en modo mono-usuario. Si bien, tanto LILO como GRUB le permiten especificar un binario alternativo a ejecutar en lugar del proceso `init`, usando la opción de arranque `"init=comando"`. Especificando `/bin/bash` como el comando con el que empezar, puede obtener un rápido marcador de intérprete de comandos en su máquina, sin tener que ejecutar `init` o pasar por cualquiera de los otros pasos del proceso normal de inicio de su sistema.



El intérprete de comandos que se inicia cuando ejecuta `/bin/bash` directamente no tiene control de trabajos (**Control-Z**) y no responde a interrupciones (**Control-C**), así que tenga mucho cuidado con los comandos que ejecuta desde este intérprete. No ejecute comandos que no terminen automáticamente o que no le soliciten sub-comandos que le permitan salir y volver al intérprete.

Si está usando el gestor de arranque GRUB, pulse cualquier tecla para interrumpir el proceso de arranque estándar de GRUB, use las teclas de flecha para seleccionar el núcleo de sistema que quiera ejecutar, y pulse la tecla **E** para editar las opciones de arranque para ese núcleo. Seleccione la línea que contiene las opciones de arranque reales (normalmente la primera línea), pulse la tecla **E** de nuevo para editar esa línea de comandos, y añada el comando `"init=/bin/sh"` al final de dicha línea. Puede entonces pulsar **B** para arrancar con esas opciones de arranque. Debería ver un marcador de intérprete de comandos en pocos segundos.

Si todavía está usando el gestor de arranque LILO, puede hacer lo mismo introduciendo el nombre de la estrofa de arranque que quiere arrancar (normalmente `"linux"`), seguida de un espacio y el comando `"init=/bin/sh"`.

De nuevo, debería obtener un marcador de intérprete de comandos en pocos segundos.

Tras obtener el marcador de intérprete, debería montar de nuevo `/proc` para asegurarse de que comandos como `ps` (y cualquier otro que use el sistema de ficheros `/proc`) funcionen correctamente. Puede hacer esto ejecutando el siguiente comando como súper-usuario (o vía `sudo`):

```
# mount -t proc none /proc
```

Si necesita crear ficheros en su sistema (por ejemplo, si está creando un archivo de ficheros que quiera migrar a otro sistema "por si acaso"), debe además montar de nuevo su sistema de ficheros raíz en modos de lectura/escritura, ya que en este punto tan temprano del proceso de arranque está montado como sólo-lectura. Para hacer esto, ejecute el siguiente comando como súper-usuario (o vía `sudo`):

```
# mount -o remount,rw /
```

Puede ejecutar ahora comandos como los de reparación de sistemas de ficheros, iniciar su interfaz de red manualmente ejecutando `/sbin/ifconfig` con una dirección IP estática, o cualquier otro comando que necesite hacer para poder reparar su sistema actual o migrar datos a otros sistema.



TRUCO

92

Descubra por qué no puede desmontar una partición

Si no puede desmontar un disco porque está ocupado, puede usar los comandos `lsdf` y `fuser` para encontrar ficheros abiertos o molestos procesos adjuntos.

La popularidad de las unidades extraíbles y su utilidad para cosas como hacer copias de seguridad hacen del montaje y desmontaje de particiones una actividad bastante común mientras un sistema está ejecutando. Otra actividad, no tan común pero sí más crítica, del administrador de sistemas es la necesidad de desmontar una unidad en una emergencia, como cuando uno de sus usuarios ha eliminado accidentalmente su tesis o el código fuente de su producto de próxima generación, o el disco comienza a tener errores de escritura y necesita iniciar la recuperación cuanto antes.

En cualquier caso, es verdaderamente irritante cuando no puede desmontar una partición porque algún proceso desconocido está usándola de una u otra manera.

Apagar el sistema simplemente para desmontar una unidad, de tal manera que pueda retirarla o repararla, es claramente desproporcionado. ¿No hay una manera mejor? Por supuesto que la hay, continúe leyendo.

Escena

Una de las reglas más básicas de Linux/Unix es que no puede desmontar una partición mientras un proceso está escribiendo o ejecutando en ella.

Intentar hacer esto devuelve un informativo pero bastante inútil mensaje como el siguiente:

```
$ sudo umount /mnt/music
umount: /mnt/music: device is busy
umount: /mnt/music: device is busy
```

En algunos casos, poner fin a los procesos asociados con una partición es tan fácil como recorrer todas sus ventanas buscando procesos suspendidos o en segundo plano, que estén escribiendo en la partición en cuestión o usándola como su directorio actual de trabajo, e interrumpirlos.

Sin embargo, en sistemas gráficos multiusuario, esto no es siempre tan directo como le gustaría. Como progreso hacia una solución final a esta frustración, especificaciones Linux de propósito específico tales como Carrier-Grade Linux (CGL) requieren alguna funcionalidad de "desmontaje forzado" en el núcleo de sistema (<http://developer.osdl.org/dev/fumount/>), y el comando `umount` incluye una opción de forzado (`-F`) para sistemas de ficheros NFS.

Esto está muy bien, pero aquellos de nosotros que estamos usando distribuciones Linux de vainilla en discos locales, todavía necesitamos una solución práctica que no requiera poner un parche a cada núcleo de sistema o matar moscas a cañonazos con un apagado inmediato.



Versiones recientes del comando `umount` proporcionan una opción `-l` para desmontar "perezosamente" (*lazily*) un sistema de ficheros de manera inmediata, e intentar limpiar entonces las referencias a él, según finalizan los procesos asociados. Esto es ciertamente interesante y puede ser útil, pero generalmente prefiero saber qué está sucediendo si no puedo desmontar un sistema de ficheros que pienso que debería ser capaz de desmontar. Podría funcionar de manera diferente en su ordenador.

Linux proporciona dos comandos que puede usar para identificar procesos que están ejecutando en un sistema de ficheros, de tal manera que pueda (con un poco de suerte) ponerles fin de una manera u otra: `fuser` (*find user process*, encontrar proceso de usuario) y `lsdf` (*list open files*, listar ficheros abiertos). La diferencia clave entre los dos es que el comando `fuser` simplemente devuelve los identificadores de proceso (PID) de cualquier proceso asociado con el fichero o directorio especificado como argumento, mientras que el comando `lsdf` devuelve un listado completo de procesos que proporciona un surtido de información sobre los procesos asociados con su(s) argumento(s). Ambos son bastante útiles, y cuál utilice depende de usted.

Las dos siguientes secciones muestran cómo usar cada uno de estos comandos para ayudarle a encontrar los molestos procesos que le impiden desmontar una partición.



La página Web de desmontaje forzado del Open Source Development Lab, referenciada al final de este truco, proporciona un crudo y genial *script* llamado `funmount` que intenta combinar automáticamente un número de pasos de `fuser` con los comandos `umount` apropiados para "hacer lo correcto" por usted cuando necesite desmontar forzosamente una partición determinada. Merece la pena echarle un vistazo.

Encontrar procesos que están usando un sistema de ficheros

El comando `fuser` devuelve los PID de todos los procesos asociados con el dispositivo o el sistema de ficheros montado que se especifica como argumento, junto con una lacónica información que resume la manera en la que cada proceso está usando el sistema de ficheros.

Para buscar todos los procesos asociados con un dispositivo o sistema de ficheros montado, necesita especificar la opción `-m`, seguida del nombre del sistema de ficheros o de su punto de montaje. Por ejemplo, el siguiente comando `fuser` busca procesos asociados con el sistema de ficheros montado en `/mnt/music` en mi sistema:

```
$ fuser -m /mnt/music
/mnt/music:      29846c 31763c
```

Cada identificador de proceso devuelto por el comando `fuser` está seguido por una sola letra que indica cómo el proceso específico está usando el sistema de ficheros. La más común de ellas es la letra `"c"`, que indica que el proceso está usando un directorio en ese sistema de ficheros como su directorio de trabajo actual (*current*).

En el ejemplo anterior, puede ver que ambos procesos listados están usando el sistema de ficheros como su directorio de trabajo actual.

Una vez que tiene este tipo de salida, puede usar el comando `grep` para buscar cada uno de los identificadores de proceso especificados y ver que están haciendo realmente, como en el siguiente ejemplo:

```
$ ps auxww | grep 29846
0 1000 29846 7797 16 0 9992 2284 wait Ss pts/13 0:00 /bin/bash
4 0 29912 29846 16 0 24608 1364 finish T pts/13 0:00 su
0 1000 31763 29846 16 0 10292 2480 - S+ pts/13 0:00 vi playlist.m3u
0 1000 31789 30009 17 0 3788 764 - R+ pts/14 0:00 grep -i 29846
```

Por defecto, el comando `fuser` devuelve todos los procesos activos. Sin embargo, como podemos (por accidente) ver en el listado de procesos anterior, hay además un proceso su finalizado que es hijo del proceso que `fuser` identificó, el cual podría impedirnos desmontar el sistema de ficheros en cuestión. Para proporcionar un listado de salida de `fuser` más detallado, generalmente debería ejecutar el comando `fuser` como súper-usuario (o vía `sudo`), y especificar además la opción `-a` para asegurar que se listan todos los procesos, independientemente de sus estados, como en el siguiente ejemplo:

```
$ sudo fuser -am /dev/mapper/data-music
/dev/mapper/data-music: 29846c 29912c 29916c 31763c 32088
```

Como puede ver, `fuser` recoge ahora el identificador de proceso de su.



Si tiene mucha prisa, puede también especificar la opción `"-k"` del comando `fuser`, la cual mata cualquier proceso que encuentra. Generalmente es una buena idea intentar encontrar los procesos en cuestión y ponerles fin limpiamente. En algunos casos podría simplemente querer matar todos los procesos tan rápido como sea posible (por ejemplo, cuando espera recuperar posteriormente ficheros borrados y quiere evitar actualizaciones del sistema de ficheros).

Listar ficheros abiertos

El comando `fuser` devuelve los PID que requieren una posterior interpretación para descubrir qué ficheros está realmente usando en el sistema de ficheros especificado (aunque el indicador de estado añadido a cada PID le da una rápida idea de cómo cada proceso está usando el sistema de ficheros). En contraste, el comando `lsuf` devuelve información más detallada sobre los procesos que tienen ficheros abiertos en un sistema de ficheros especificado, y podría decirle todo lo que necesite saber de un solo vistazo. Por ejemplo, lo siguiente es la salida del comando `lsuf` en el mismo sistema de ficheros usado en los ejemplos anteriores:

```
$ lsuf /mnt/music
COMMAND PID USER FD TYPE DEVICE SIZE NODE NAME
bash 29846 wvh cwd DIR 253,0 64 131 /mnt/music/test
vi 31763 wvh cwd DIR 253,0 64 131 /mnt/music/test
vi 31763 wvh 4u REG 253,0 12288 133 /mnt/music/test/.playlist.m3u.swp
```

La primera columna ("COMMAND") muestra cada comando que está asociado con el fichero, directorio, o punto de montaje que especifique como argumento. La última columna ("NAME") identifica el fichero o directorio con el que cada comando está realmente asociado. La columna "FD" muestra los descriptores de fichero activos asociados con el proceso o, en el caso de un intérprete de comandos o de un comando, el hecho de que esté usando el directorio especificado como su directorio de trabajo actual (*cwd*, *current working directory*).

Como con `fuser`, cuando `lsuf` es ejecutado por un usuario estándar su salida muestra sólo procesos activos. Para obtener una salida más completa, generalmente debería ejecutar el comando `lsuf` como súper-usuario (o vía `sudo`), como en el siguiente ejemplo:

```
$ sudo lsuf /mnt/music
COMMAND PID USER FD TYPE DEVICE SIZE NODE NAME
bash 29846 wvh cwd DIR 253,0 64 131 /mnt/music/test
su 29912 root cwd DIR 253,0 17 128 /mnt/music
bash 29916 root cwd DIR 253,0 17 128 /mnt/music
vi 31763 wvh cwd DIR 253,0 64 131 /mnt/music/test
vi 31763 wvh 4u REG 253,0 12288 133 /mnt/music/test/.playlist.m3u.swp
```